Riassuntone Sistemi Operativi

Appunti presi a lezione e nozioni integrate dal libro

Santo Lo Bianco 1000061604

CAPITOLO 1 INTRODUZIONE

I sistemi operativi saranno argomento cardine del programma cardine di questa materia. Essi sono una parte fondamentale del software presente nei dispositivi che ci circondano tutti i giorni.

Poiché si parlerà poco dell'hardware, è bene fare un'introduzione parlando dei componenti che costituiscono un pc. Come visto precedentemente(architettura degli elaboratori :>), un calcolatore è formato da:

- Uno o più processori (punto cardine dell'elaborazione, divisi in ALU e CU);
- Memoria centrale, banco di lavoro dei pc;
- Dischi, dediti allo stoccaggio dei dati in modo persistente;
- Mouse, stampante e altre periferiche I/O.

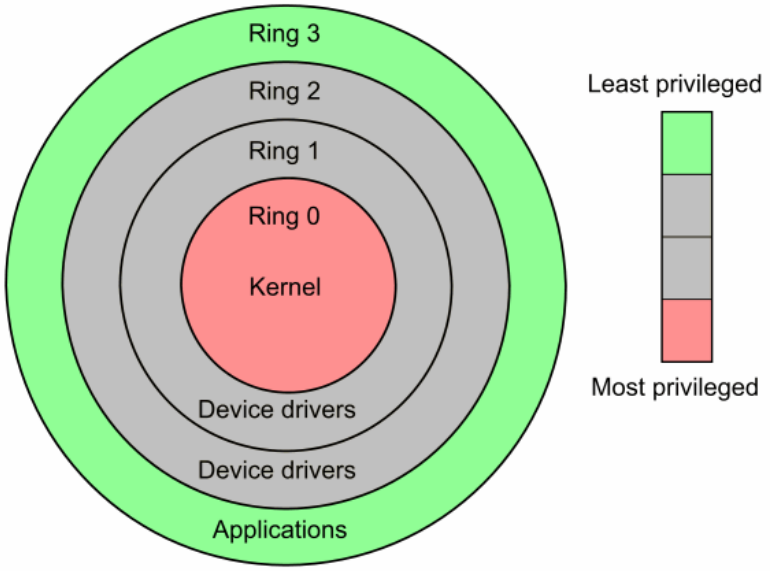
Dato che gestire tutti i dettagli di basso livello è molto complesso, ad esempio muovere la testina del disco, si introducono i Sistemi Operativi (SO).

Un sistema operativo è uno strato intermedio software che permette di svolgere tutte queste operazioni automaticamente.

Più volgarmente il sistema operativo è un grande programma che si occupa di impartire ordini alle componenti fisiche prima citate, in modo che l'utente possa usare il computer facilmente e senza conoscenza dei dettagli tecnici.

Possiamo controllare l'hardware attraverso due modalità:

- **Modalità Kernel(supervisor)**: modalità che ci permette di avere un'esecuzione più veloce e un controllo diretto sulle singole risorse. È la modalità privilegiata in cui opera il nucleo(KERNEL) del sistema operativo. => **gestore file system, scheduler processi, driver di dispositivo, ecc. .**
- **Modalità Utente**: modalità limitata, destinata al solo utilizzo di app sviluppate per l'utente. Questa modalità prevede un accesso limitato all'hardware, questo garantisce sicurezza ma non prestazioni, in termini di velocità. => **browser web, app messagistica, editor di testo, ecc. .**



Il sistema operativo effettua delle ASTRAZIONI ovvero che da qualcosa che non ha uno scopo apparente, attraverso questa tecnica può assumere dei significati. Quindi quello che fa il sistema operativo è quello di offrire dei servizi di astrazione.
Un esempio di astrazione molto facile da capire è il file.
Il file è più semplicemente una sequenza di byte, che attraverso il sistema operativo assume un significato.
Il SO deve gestire in modo controllato e ordinato tutte le risorse hardware e software, e da sistemi più moderni viene richiesto di gestire più programmi contemporaneamente e più utenti.
Necessita quindi di un'allocazione controllata e ordinata di risorse hardware (processori, memoria, unità di I/O) e software(file,database), questo perché poi effettua il multiplexing nel tempo e nello spazio.

Una delle attività del sistema operativo è quella di nascondere all'utente la componentistica hardware e presentarla con programmi frutto di astrazioni, con il quale è più comodo lavorare.

Multiplexing nel tempo: l'uso di una risorsa viene alternata tra vari programmi, nel tempo, in modo che non ci sia starvation(mancanze) per delle unità.

Multiplexing nello spazio: l'uso di una risorsa viene suddiviso fisicamente tra più programmi, in modo che ogni programma usi la partizione della risorsa, senza che esso se ne accorga.

I sistemi operativi si sono evoluti con l'hardware che li accoglieva, l'evoluzione hardware e sistema operativo integrato è stata sinergica e reciproca.

Possiamo quindi individuare 5 generazioni di computer con i relativi sistemi operativi.

Essi sono:

1) **Prima generazione**: **computer a valvole termoioniche**, la programmazione avveniva cablando circuiti elettronici o in linguaggio macchina e gli output erano gestiti attraverso queste valvole. Qui si ebbe la prima intuizione di sistema operativo;
2) **Seconda generazione**: **sistemi batch basati su transistor**, ovvero sistemi dove insiemi di lavori o job venivano raggruppati ed eseguiti in sequenza automatica. Si sviluppava in FORTRAN e aveva un sistema operativo(che fungeva solo da scheduler);
3) **Terza generazione**: nascono **i circuiti integrati**(chip con molti transistors), si hanno i primi sistemi operativi e soprattutto la multiprogrammazione e sistemi timesharing. Tutto ciò preveda dei sistemi operativi più robusti rispetto alla seconda generazione, ed è qui che nasce UNIX, MINIX e infine LINUX(molto usato per il suo kernel[open source]). Si ricorda che la programmazione avveniva attraverso linguaggi come COBOL, FORTRAN, BASIC e assembly per parti critiche;
4) **Quarta generazione**: attraverso alla tecnologia **Large Scale Integration**(centinaia di transistor in un singolo chip) si aprì la strada per i **microprocessori** tipo 8080 ecc. . Un'intuizione molto importante fu quella di collegare i floppy disk al processore e scrivere il sistema operativo lì. IBM inizialmente fu riluttante ed è per questo che solo con MS-DOS, si ebbe il successo di quest'idea. Allora, i comandi venivano scritti tramite linee di comando, ma ciò cambio con l'avvento della GUI(Graphic user interface), studiata nei laboratori della XEROX, e poi rubata e implementata dalla Apple. Successivamente tutti i sistemi operativi si avvicinarono all'uso dell'interfaccia grafica, abbandonando il vecchio modo di concepire il computer.

5) **Quinta generazione**: **computer mobili**. L'ultima generazione, infatti, prevede i telefoni mobili, sia quelli più tradizionali(a tasti per intenderci) e sia gli smartphone il cui sistema operativo iniziale era Symbian

OS. Con l'introduzione di Android, un sistema operativo open source sviluppato da google, si abbandonò symbian. Attualmente un concorrente di Android è iOS il cui proprietario è l'Apple.

Il processore è l'unita del pc dedita alla gestione del flusso di esecuzione e all'esecuzione delle istruzioni, segue un ciclo ben predefinito, e una sua semplificazione per gli elaboratori moderni è la seguente:

PRELIEVO ⟶ DECODIFICA ⟶ ESECUZIONE

- Prelievo dell'istruzione dalla memoria centrale;
- Decodifica dell'istruzione, linguaggio oggetto;
- Esecuzione dell'istruzione.

La CPU al suo interno ha una piccolissima unità di memoria, impiegata per contenere informazioni essenziali sullo stato del sistema, essenziali per mandare avanti l'esecuzione. Questa unità di memoria si chiama **banco dei registri**, e si divide in **registri di uso generale** e **registri speciali**.

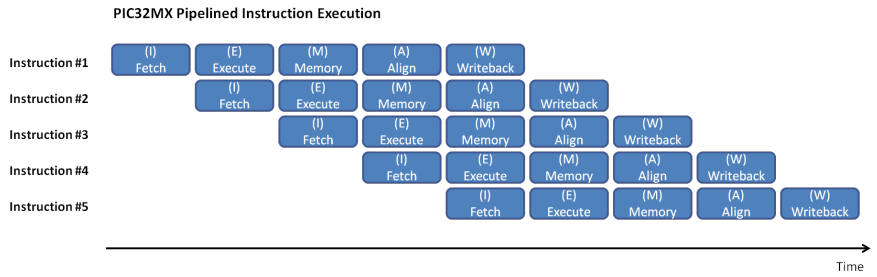
Registri di **uso generale**: unità di memoria usata per contenere dati temporanei, possono essere usati dai programmi liberamente.

Registri di **uso speciale**: unità di memoria usata per contenere informazioni essenziali sullo stato dell'elaborazione e del sistema. Ogni registro ha un compito specifico, infatti abbiamo:

- PROGRAM COUNTER(PC) ⟶ contiene l'indirizzo della prossima istruzione;
- STACK POINTER(SP) ⟶ punta alla cima dello stack;
- PROGRAM STATUS WORD(PSW) ⟶ contiene dei flag che descrivono lo stato della CPU;
- ISTRUCTION REGISTRER(IR) ⟶ contiene l'istruzione corrente.

Progettazioni avanzate prevedono l'impiego di **pipeline** e **cpu superscalare**.

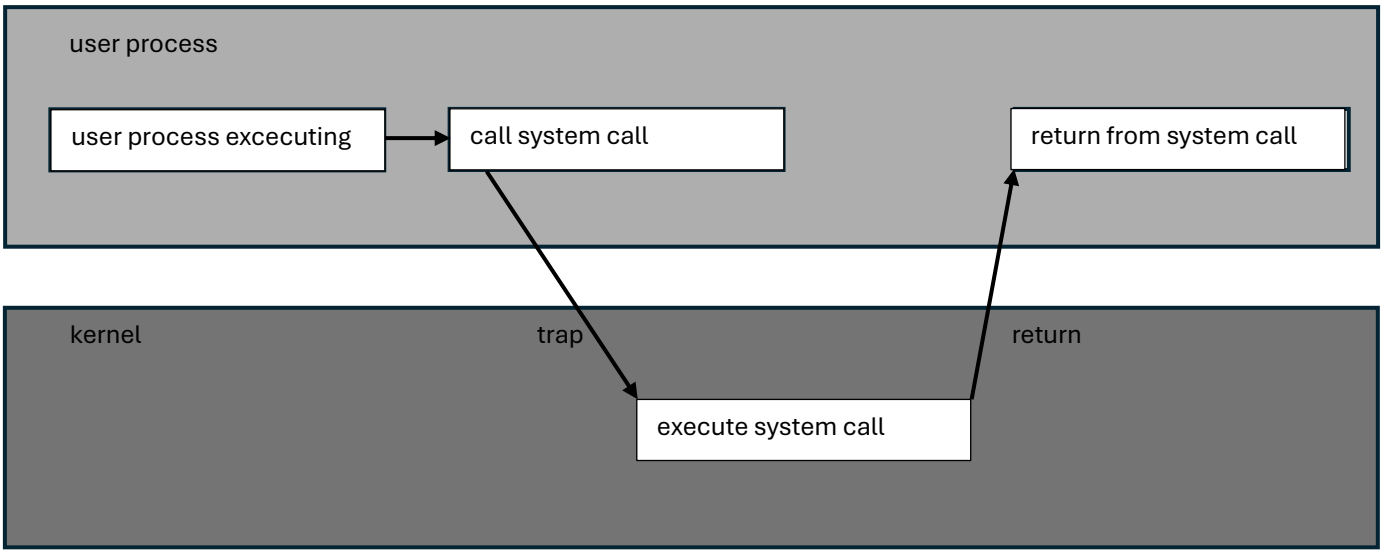
PIPELINE: è una tecnica che permette di sovrapporre più istruzioni, sfruttando il vantaggio che l'istruzione si può dividere in più fasi e che quando un'istruzione si trova in una determinata fase dell'elaborazione, in fasi differenti possiamo avere altre istruzioni.



CPU SUPERSCALARE: CPU dotata di più unità funzionali, che eseguono in parallelo più istruzioni. Hanno la stessa memoria, lo stesso flusso di esecuzione, le stesse risorse a disposizione, diviso per più unità.

L'esecuzione prevede, come citato prima, due modalità, ovvero USER MODE e KERNEL MODE. Se il programma ha bisogno di usare una risorsa protetta, deve fare una **chiamata di sistema(system call)** per passare il controllo al kernel.

Esistono queste due modalità al fine di proteggere il sistema, infatti in user mode, se avviene un bug, non si danneggia l'intero sistema.



Una **TRAP** è un meccanismo di interrupt software: serve a passare da user mode al kernel mode in modo controllato, quando un programma chiede un servizio al sistema operativo.

Quando un programma invoca una system call, per gestirla la CPU genera una TRAP che causa un salto controllato al Kernel. -> viene interrotto il flusso del programma e la trap intrappola la richiesta nel Kernel -> TRAP indica catturato, intrappolare.

Viene utilizzato il termine TRAP perché nei primi sistemi operativi si dovevano distinguere gli interrupt causati da software(system call) da quelli causati dall'hardware(es. interrupt tastiera).

Quando viene innescata una TRAP, si ha un costo di esecuzione alto, soprattutto in confronto alle istruzioni normali. Questo perché?

Il cambio di contesto da user a kernel mode richiede operazioni come:

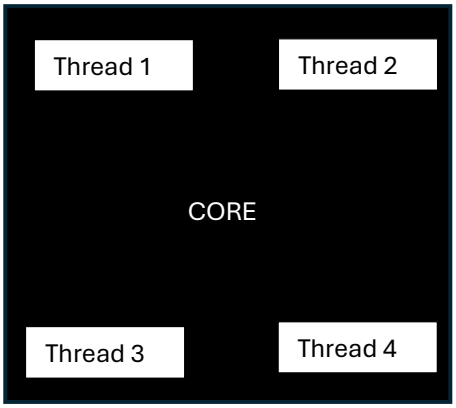
- Salvataggio dello stato del processo;
- Cambio dello stack;
- Aggiornamento dei registri di sistema.

Questo passaggio viene effettuato due volte, sia quando passiamo alla modalità kernel, che sia quando torniamo alla modalità kernel. => Il **context switch** tra processi influenza molto questo costo alto.

Possiamo adesso parlare dei vari tipi di elaborazione multipla dei processi, essa infatti può avvenire a secondo di come viene strutturata la nostra architettura.

Le seguenti architetture dei processori sono cruciali per l'elaborazione:

- **MULTITHREADING**, architettura che prevede che il singolo core gestisca più thread mantenendo lo stato di più contesti di esecuzione. CPU gestisce i thread in modo indipendente, quindi la congestione di un non implica che il resto si blocchi.

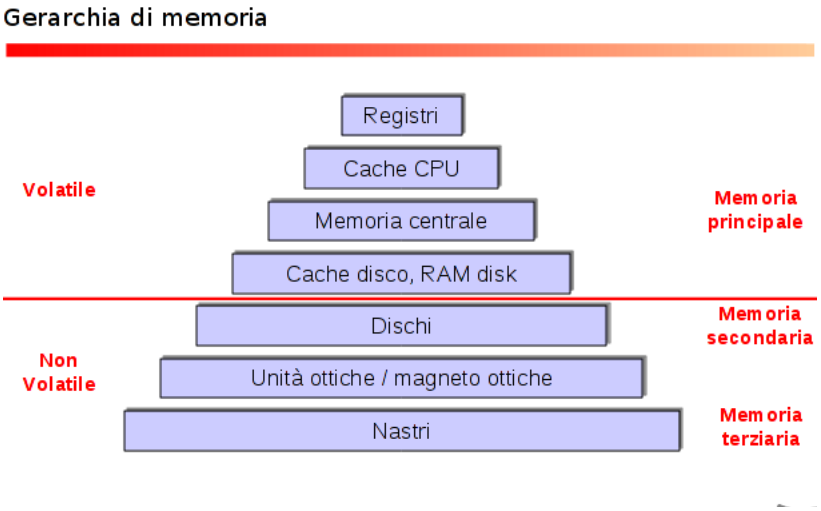


- **MULTIPROCESSORI**, architettura che prevede più processori fisici, indipendenti ma coordinati dal sistema operativo. Si ha un throughput maggiore, ovvero numeri di processi che vengono elaborati in parallelo, e una sicurezza maggiore dato che nonostante il malfunzionamento di una delle CPU, il sistema non si arresta.
- **MULTICORE**, architettura che prevede che un singolo chip con molti core(unità di elaborazione). Le prestazioni sono simili ad un multiprocessore solo che risolvono il collo della bottiglia della memoria perché i core comunicano più velocemente.
- **GPU**, architettura specializzata con centinaia o migliaia di core molto semplici, progettati per elaborazioni in parallelo massiccio e non seriale (calcoli eccessivamente grossi preferibili a molte istruzioni che si susseguono).

Procedendo con lo studio dei componenti dei moderni elaboratori, dobbiamo necessariamente parlare delle memorie.

Nei nostri PC abbiamo diversi tipi di memorie, che cambiano in termini di velocità e dimensione a secondo dell'uso a cui sono finalizzate.

Abbiamo appunto una gerarchia di memoria:



Se si hanno conoscenze basilari sulle memorie si può già intuire che alla base della piramide ci sono tipi di memoria che possono contenere più dati, ma più lente, cosa inversa della cima.

Velocità e capacità nel caso delle memorie sono inversamente proporzionali, ed è per questo che esiste la RAM.

La **RAM** o **Random Access Memory(core Memory)**, è una memoria di tipo volatile ovvero che quando c'è assenza di corrente, perde tutti i dati. Essa, infatti, non è utile allo stoccaggio dei dati in modo permanente ma bensì funge da banco di lavoro per il nostro elaboratore.

Attraverso la RAM effettuiamo **l'astrazione della memoria**, offrendo una visione semplice e uniforme della memoria e con il supporto del Sistema Operativo si fornisce al singolo processo l'illusione che ha per sé tutta la memoria continua e privata, anche se la memoria è condivisa tra tutti i processi.

Inoltre, attraverso questa memoria e le memorie cache, che sono presenti fisicamente vicino la CPU, si approssima un'immagine diversa da ciò che è realmente la memoria, mostrandola veloce con sé introducendo anche ritardo, generato dal dato non trovato.

Proseguendo per assonanza fonetica, anche se con significato totalmente diverso, possiamo parlare delle **ROM** o **Read Only Memory**, memorie non volatili, che una volta scritte solitamente dal produttore, non possono più essere modificate. Esse solitamente vengono utilizzate per contenere firmware o istruzioni essenziali per l'avvio del sistema operativo, come il BIOS che viene eseguito in fase di boot.

Infine, abbiamo le memorie che contengono grandi quantità di dati in modo permanente, sono:

- **Unità elettroniche**, unità di memoria il cui immagazzinamento dei dati funziona attraverso il passaggio di corrente in circuiti elettronici. Di questa categoria ne fanno parte le seguenti memorie: SSD, SD e le chiavette USB. Tra le memorie di massa sono le più veloci
- **Unità magnetiche**, unità di memoria il cui immagazzinamento dei dati avviene attraverso la polarizzazione di aree di un disco che gira intorno ai 7000 giri per minuto. Di questa categoria ne fanno parte gli Hard Disk. Essi sono sistemi che prevedono due motori, uno per fare girare i dischi e l'altro per muovere la puntina che polarizza l'area.
- **Unità ottiche**, unità di memorie la cui lettura e scrittura dei dati avviene attraverso incisioni effettuate con un laser su un disco. A seconda della tipologia del laser cambia la quantità di dati immagazzinati, es. bluray, dato che utilizza onde a corto raggio, permette di fare incisioni più fini e quindi di immagazzinare più dati. Di questa categoria ne fanno parte CD, DVD e Blu-Ray.

Non esiste solo CPU e memoria in un computer, infatti uno dei punti fondamentali che rende aperta la macchina al mondo esterno è la **periferica**, che sia di input o output.

Le periferiche interagiscono intensamente con il sistema operativo, dato che l'affluenza di dati che passa da una periferica è molto alta.

Le periferiche le possiamo dividere in due parti: il **dispositivo** e il **controller** della periferica.

Il dispositivo presenta un'interfaccia semplificata che è complicata pilotarla, per questo si utilizzano interfacce standardizzate tipo SATA, USB, ecc.

Il **controller** è un chip, che controlla fisicamente la periferica ed esegue i comandi impartiti dalla CPU. Ogni controller è diverso per tipologia ed è per questo che per ognuno di essi c'è un **software apposito chiamato driver.**

Il controller comunica con il driver ed esso a sua volta con il sistema operativo, ciò garantisce un'interfaccia uniforme e astratta verso le periferiche. Questo, quindi, permette al sistema operativo di gestire dispositivi diversi, senza dover conoscere i dettagli hardware di ogni controller.

Più nel dettaglio le operazioni dei driver dei dispositivi sono due:

- **mappatura in memoria**, sia che le periferiche siano gestite come normali locazioni di memoria e sia che abbiano partizioni di memoria appositi
- **traduzione istruzioni di Input/Output**.

Ci sono tre alternative per la comunicazione tra il sistema operativo e le perifiche:

- **busy waiting**, ovvero che il sistema operativo attraverso i controller manda delle chiamate di sistema e resta in attesa controllando lo stato delle periferiche, per attivare eventuali routine(inefficiente perché la CPU resta occupata anche in assenza di eventuali elaborazioni);
- **meccanismo di interrupt**, se sono permessi, le periferiche mandano degli interrupt request, il sistema operativo sospende la corrente elaborazione e attiva la routine della periferica individuata attraverso il vettore degli interrupt. Al termine la CPU riprende i lavori precedentemente sospesi;
- **DMA o direct access memory**, si esegue in modalità utente, non c'è bisogno di coinvolgere la CPU per il trasferimento in memoria del singolo dato. Questo permette un vantaggio in termini di prestazioni.

Un'ulteriore componente importante nei nostri calcolatori, sono i **bus**. I dati, le istruzioni e qualsiasi indirizzo passano dai bus, motivo per cui è necessario che siano efficienti nel trasporto.

Ogni periferica comunica con la CPU attraverso bus standardizzati quali PCIe, USB, SATA, che definiscono modalità e velocità di trasmissione.

PCIe -> schede video, SSD, schede di rete;

USB -> dispositivi esterni;

SATA -> dischi rigidi.

Fisicamente per collegare i bus al processore, si utilizza il **DMI** (direct media interface), un bus sviluppato per collegare il processore al chipset della scheda madre.

Attraverso il DMI transitano dati proveniente da PCIe, USB, ecc., dati anche non gestiti direttamente dalla CPU.

Ma quali sistemi operativi girano nelle diverse tipologie di dispositivi:

MainFrame -> OS/360 e recentemente Linux;

Server -> FREEBSD, Linux, Solaris e Microsoft Server;

Personal Computer -> Windows, Linux, UNIX, MACOS, FREEBSD e molti altri;

Computer palmare, smartphone -> Android, iOS, iPadOS;

Dispositivi IOT -> Linux, QNX e VxWorks;

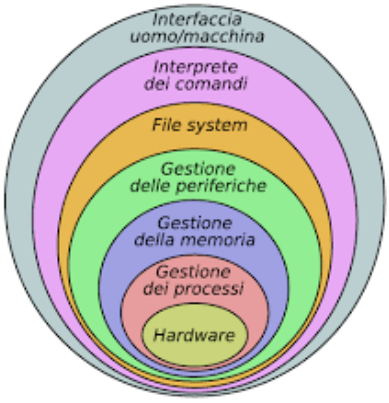
Real time -> eCos, QNX e VxWorks;

I sistemi operativi variano per la gestione dei processi, indirizzi e dei file.

A seconda delle esigenze si possono seguire preferire alcuni approcci architetturali piuttosto che altri.

Di seguito ne sono citati alcuni:

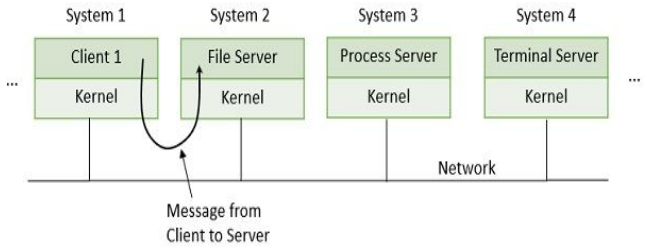
- **Approccio monolitico** -> Organizzazione più comune per un sistema operativo, viene eseguito come un programma singolo in modalità kernel. Il sistema operativo è diviso in tante procedure, poi chiamate all'occorrenza all'interno di un grande programma. È un approccio efficiente dividere il sistema in procedure, ma solo in fase di esecuzione, perché è meno flessibile in fase di compilazione, dato che deve essere compilato tutto il sistema operativo. C'è assenza di occultamento delle informazioni e quindi tutte le procedure sono visibili da altre. Possono essere collegate estensioni per i driver dei dispositivi I/O, ecc., queste si chiamano librerie condivise in UNIX o DLL(dynamic-link libreries) in windows.
- **Sistemi a livelli** -> Organizzazione del sistema operativo concepita da W. Dijkstra e dai suoi studenti durante lo sviluppo di THE. Un sistema a livelli prevede una suddivisione gerarchica, dove ogni livello è costruito sul precedente. L'idea è di separare le funzionalità in moduli indipendenti, ciascuno con le proprie responsabilità. Il livello più basso gestisce l'hardware. Ciò costituisce un sistema più solido, dato che errori nei moduli superiori non costituiscono compromissione d'integrità in quelli inferiori, più facile da mantenere aggiornato e riparabile, dato che si opera solo con il modulo in questione, più sicuro, dato che un livello superiore impone sicurezza e controllo sui livelli superiori ed è di maggiore leggibilità. Può risultare meno prestante, dato che ci sono chiamate nidificate e al relativo overhead di esse.



- **Microkernel** -> Con l'approccio a livelli, i progettisti sono in grado di stabilire dove porre fine al livello kernel e far iniziare quello utente. L'approccio Microkernel è nato dall'esigenza di sicurezza e affidabilità del sistema operativo. Ad esempio, un errore in un driver non dovrebbe compromettere l'intero sistema. Per questo motivo, si è scelto di trasferire il maggior numeri di processi dal kernel allo spazio utente . Quindi il design di questo approccio architetturale prevede un microkernel e il resto dei processi organizzato in moduli separati eseguiti in modalità utente. Ad esempio, il sistema operativo MINIX 3 adotta una struttura a 4 livelli:
    - Microkernel
    - Livello driver
    - Livello server
    - Programmi utente

La comunicazione tra processi avviene attraverso lo scambio dei messaggi espliciti, gestiti attraverso il microkernel che funge da mediatore.

- **Microkernel client-server** -> Rivisitazione dell'approccio microkernel che prevede l'individuazione di componenti del sistema, ognuna con un ruolo ben definito, ovvero:
    - Client, processi che richiedono servizi(applicazioni che richiedono l'accesso al file system);
    - Server, processi che offrono servizi(un file server che gestisce le operazioni su file).



Client-Server System Structure

Il microkernel funge da mediatore fornendo i meccanismi di comunicazione tramite messaggi(inter-process-comunication). Il funzionamento è simile a ciò che accade con http, infatti, il programma client effettua una richiesta che arriva al server mediante il kernel. Il server esamina la richiesta e poi effettua la prestazione desiderata.

Così ogni server è isolato e in caso di errore non blocca il sistema, può essere aggiornato o ravviato senza che si riavvi il kernel.

- **Sistemi a moduli** -> l'organizzazione a moduli del sistema è fortemente influenzata dall'idea della programmazione ad oggetti, però applicata al kernel.

  I moduli in questo sistema implementano un aspetto specifico, essi possono invocare qualsiasi altro modulo, anche se stessi, senza uso di messaggi. Inoltre, i moduli sono spesso caricabili e scaricabili dinamicamente, durante l'esecuzione del sistema operativo e non necessitano di un riavvio. Anche se si pensa che un bug possa essere cruciale, linux dimostra che in realtà seguendo questo approccio non ci si imbatte in questa instabilità.

- **Macchine virtuali** -> il sistema operativo è un fornitore di astrazioni, infatti estremizzando questo concetto, possiamo ottenere: memoria virtuale, CPU virtuale, e infine intere macchine virtuali.
  Una macchina virtuale è quindi un ambiente completamente isolato(usate per i test) e simulato, che emula il funzionamento di una macchina reale. Il riutilizzo delle macchine virtuali è stato favorito a seguito dell'espansioni di grandi basi di dati e del cloud computing. Data l'elevata richiesta computazionale, si è scelto di usare macchine virtuali, che favoriscono l'omogeneità nei sistemi e nelle reti.
  Importante è l'ottimizzazione delle risorse hardware, dato che possono essere eseguiti più sistemi operativi sulla stessa macchina.
  L'**hypervisor** è il perno delle macchine virtuali, infatti, è il software che consente di crearle, gestirle ed eseguirle. Esso consente di emulare l'hardware per ciascuna macchina, assegnare risorse fisiche e gestire l'isolamento tra le varie macchine virtuali e sistema host.
  Si hanno due tipi di hypervisor:
    - **Tipo 1**: software che eseguito direttamente sull'hardware, **senza sistema operativo sottostante**, ciò contribuisce ad una maggiore efficienza e sicurezza(meno codice soggetto a bug)(esecuzione nativa sulla macchina fisica). Un esempio di hypervisor tipo 1 è MICROSOFT HYPER-V.
    - **Tipo 2**: programma **eseguito su un sistema operativo esistente** contribuisce un uso più semplice che tuttavia va ad impattare sulla perdita d'efficienza. Un esempio di hypervisor tipo 2 è VirtualBox.

Nel caso dell'emulazione il codice del software ospitato viene interpretato ed esegue il codice sulla macchina simulata(possiamo riprodurre l'hardware originale di un dispositivo non più in circolo).

Nel caso della simulazione il comportamento del sistema viene modellato e riprodotto senza eseguire il vero codice, ma simulandone gli effetti funzionali.

Nel caso della para-virtualizzazione, il sistema operativo guest è consapevole di essere in esecuzione in una macchina virtuale e collabora con l'hypervisor per migliorare le prestazioni.

I sistemi operativi non sono tenuti a sapere che vengono eseguiti in un ambiente virtuale, solo nella para-virtualizzazione.

CAPITOLO 2 PROCESSI, THREAD, IPC E SCHEDULING

Il concetto fondamentale del sistema operativo è il processo: astrazione del programma in esecuzione. Capire questo concetto è fondamentale per comprendere come anche sola una CPU possa gestire in modo pseudo-parallelo molti programmi.

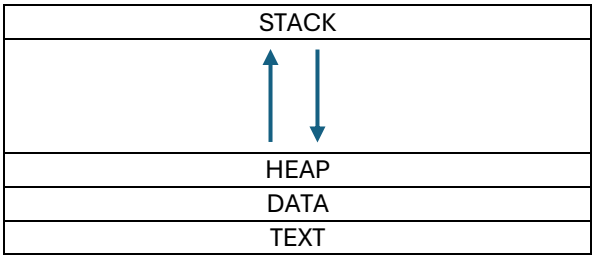
Il processo è semplicemente un'istanza di un programma in esecuzione, che contiene valori del contatore di programma, dei registri e delle variabili. La differenza tra processo e programma è impercettibile, infatti è necessario capire che il programma in realtà è l'entità passiva, ovvero il codice residente in memoria, mentre il processo è l'entità attiva, ovvero l'istanza di un programma in un determinato intervallo di tempo.

Lo pseudo-parallelismo(diverso dal parallelismo che si ottiene attraverso l'hardware) è un parallelismo ottenuto grazie al software, che ci permette di eseguire più processi "contemporaneamente".

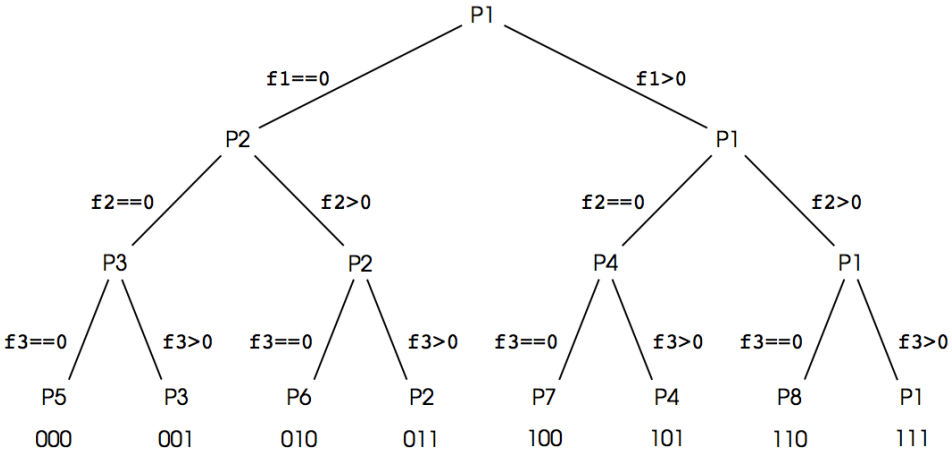
Ogni processo pensa di avere la CPU tutta per sé, ma in realtà il sistema operativo suddivide il tempo di esecuzione tra vari processi, in modo tale che durante operazioni molto lunghe, come lettura o scrittura in memoria, la CPU sia occupata a elaborare altri processi.

Il processo è associato a delle informazioni, tali sono:

- **Spazio di indirizzamento del processo**: parte di memoria assegnata al processo. All'interno dello spazio di indirizzamento possiamo trovare dati, codice, heap e stack;



- **copia dei registri della CPU**;
- **file aperti**: informazioni sui file aperti dal processo(attraverso il file descriptor), offset corrente all'interno del file e informazioni sul tipo di apertura.
- **allarmi pendenti**: fotografia di eventi in sospeso che il processo deve ancora gestire;
- **processi imparentati**: dato che i processi vengono gestiti in modo gerarchico, come in un albero genealogico. I processi possono creare altri processi che saranno i processi figlio, dato ciò è importante sapere la parentela, visto che vengono elargite informazioni sulla natura del processo.



Un processo è individuato univocamente dal proprio identificatore, chiamato anche **Process Identifier(**PID), che viene assegnato automaticamente dal sistema operativo. Attraverso il PID si schedula, gestisce e monitora il processo a cui è associato.

Il **Process Control Block**(PCB) contiene tutti i riferimenti e le informazioni necessarie a gestire un processo. In un certo senso, si può dire che il PCB è la carta d'identità del processo, dato che contiene informazioni su: chi è, cosa sta facendo, quali risorse usa e in che stato si trova.

La **Tabella Dei Processi** contiene tutti i riferimenti ai processi attivi o in esecuzione nella macchina.

La creazione dei processi è fortemente influenzata dal tipo di sistema e a seconda delle situazioni; infatti, per sistemi operativi molto semplici(sistemi operativi per elettrodomestici) può essere utile creare tutti i processi in fase di inizializzazione del sistema, ma per sistemi complessi, la creazione di un processo può assumere forme diverse.

La creazione di processi in sistemi operativi complessi può avvenire nei seguenti modi:

- **in fase di inizializzazione del sistema**: vengono creati molti processi, sia che essi siano attivi e sia che essi siano in background. I processi in background, che diventano attivi a seconda dello stato del sistema, vengono chiamati **daemon(demoni)**.
- **da parte di un altro processo**: dato un lavoro molto complesso un processo, attraverso chiamate di sistema, ne può creare un altro, e dividere il carico del lavoro con i processi creati. In questo modo più processi possono essere eseguiti parallelamente o pseudo-parallelamente per completare il lavoro più in fretta. In UNIX un processo può crearne un altro attraverso la chiamata di sistema **fork()**, che crea una copia del processo padre, con lo stesso PCB a meno del PID. La memoria su cui lavorano è diversa, anche se lo spazio di indirizzamento apparentemente è lo stesso, a meno che non si lavora in modalità copy-on-write. Attraverso la funzione **exec()**, un processo figlio, può cambiare il proprio codice ergo può lavorare su un programma diverso. In windows tutto ciò è gestito attraverso la chiamata CreateProcess().
- **su richiesta dell'utente**: ad esempio, all'apertura di una finestra si può creare un processo per la gestione di quella singola finestra.
- **inserimento di un job in modalità batch**: in sistemi ove sono previste operazioni automatizzate, all'inserimento di un nuovo job, si crea un nuovo processo incaricato a eseguirlo.
-

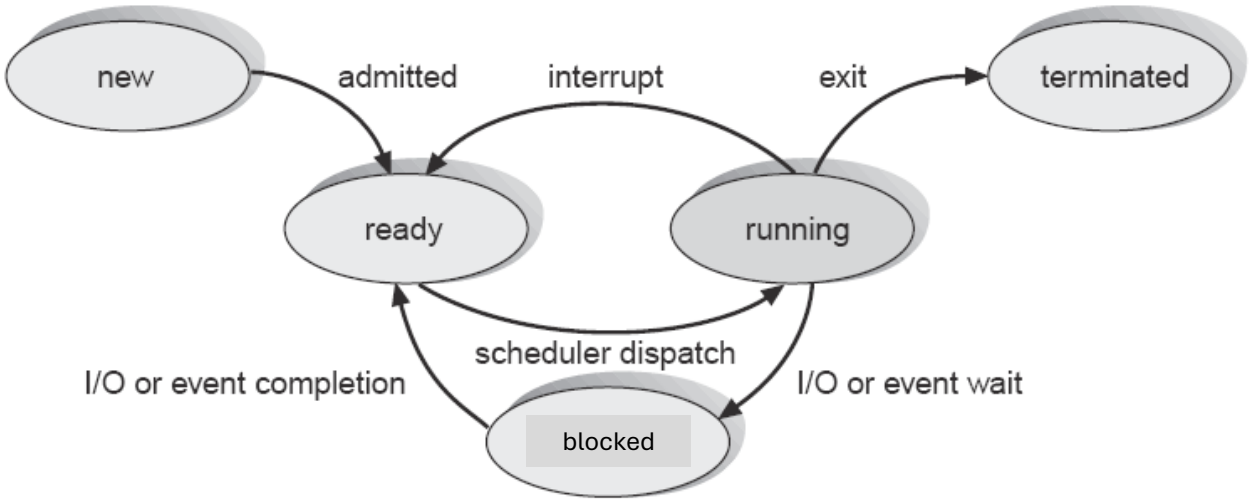
Prima o poi un processo termina e svariate possono essere le cause, tra cui:

- **uscita normale**(condizione volontaria): il processo termina dato che non ha più istruzioni che devono essere eseguite(la maggior parte delle volte terminano così i processi). Essi terminano con la chiamata **exit(0)** in UNIX e **ExitProcess** in Windows.
- **errore critico**(condizione involontaria): il processo termina a causa di errore critico, ad esempio quando si incorre in mancanza della risorsa indicata dall'utente.
- **uscita su errore**(condizione volontaria): il processo stesso, se riconosce errori nel codice, può attivare la chiamata di sistema **exit(code)** e terminare sé stesso.
- **terminato da un altro processo**(condizione involontaria): un processo denominato processo killer, se autorizzato, attraverso le chiamate di sistema **kill()** in UNIX e **TerminateProcess** in windows, può terminare un altro processo.

Un breve inciso sulla discendenza dei processi è utile a capire come funzionano. I processi possono generarne di altri attraverso le chiamate fork()(UNIX) e CreateProcess(WINDOWS), con l'unica differenza che l'albero in UNIX è molto rigido: un processo è figlio di uno e uno solo genitore. Invece, in windows l'appartenenza al genitore può variare nel tempo, perché il genitore detiene un token detto **handle**, che può passare ad un altro processo, corrompendo l'albero genealogico dei processi.

Solitamente un processo non è in un ambiente isolato, dove l'esecuzione da parte della CPU è unica e indisturbata, ma si trova in un ambiente concorrenziale, dove ogni processo vuole essere elaborato, prima di altri e dove alcuni collaborano tra loro. L'evoluzione dell'esecuzione, quindi, può risultare graduale e cooperante. È importante quindi conoscere i vari stati di un processo, che sono:

- **in esecuzione**: ovvero che la CPU lo sta eseguendo;
- **pronto**: ovvero che la CPU sta eseguendo altri processi, ma il processo in questione è pronto ad essere elaborato, e si trova nella coda dei processi pronti(coda con priorità);
- **bloccato**: un fattore esterno quale ad esempio una risorsa è mancante, ergo il processo non è pronto per l'esecuzione(anche se la CPU è libera, il processo non può essere eseguito);
- **new(addizionale):** il processo è stato appena creato e se ammesso entrerà nella coda dei processi pronti;
- **terminato(addizionale)**: il processo è stato terminato, sia correttamente o sia in stato di errore.



Abbiamo 6 transizioni di stato, le quali sono fortemente influenzate dal nostro sistema. Esse sono:

- **ammesso**(new->ready): un nuovo processo, appena creato, passa nella coda dei processi pronti;
- **scheduler dispatch**(ready->running): lo **scheduler** sceglie nella coda dei processi pronti, il processo che verrà eseguito dalla CPU;
- **interruzione**(running->ready): lo **scheduler** sospende il processo attualmente in esecuzione, inserendolo nuovamente nella coda dei processi, per sostituirlo con un nuovo processo, così garantendo un'equità nell'uso della CPU;
- **attesa evento I/O**(running->blocked): dato che il processo esige risorse I/O per proseguire, viene sospeso e passa nello stato bloccato, in attesa delle risorse richieste. Un altro processo verrà selezionato per essere eseguito;
- **completamento evento I/O**(blocked->ready): a seguito di eventi I/O, se il processo avrà tutte le risorse a disposizione per proseguire tornerà nello stato pronto, in attesa che lo scheduler lo selezioni per l'esecuzione.

Lo **scheduler**, precedentemente menzionato, è il software del sistema operativo incaricato a gestire l'assegnazione della CPU ai vari processi. Nel dettaglio, decide:

- processo che deve essere eseguito;
- tempo di assegnazione della CPU;
- eventuale sostituzione con un altro processo.

La scelta del processo che si manda in esecuzione, viene implementata attraverso le chiamate di sistema gestite dal **clock di sistema**. Il clock di sistema è un particolare dispositivo hardware che permette di misurare e gestire il tempo.

Attraverso il clock si trasmettono interruzioni periodiche, che si rivelano utili al sistema operativo per gestire processi e attivare lo scheduler.

Come viene mantenuta l'illusione che una CPU gestisca più processi sequenziali e gli interrupt utilizzati per il cambio di processo?

Abbiamo prima parlato dei PCB e della tabella dei processi, dicendo che all'interno del PCB sono conservate tutte le informazioni relative ad un processo.

Quando un processo deve essere sostituito si opera nel seguente modo:

- viene conservato nello stack il contatore di programma, ecc.;
- hardware carica il nuovo contatore di programma legato all'interrupt;
- vengono salvate in memoria tutte le informazioni legate allo stack attuale;
- viene impostato il nuovo stack secondo le informazioni dettate dalla procedura di servizio, individuata attraverso il confronto dell'ISR(interrupt servirce routine) e il vettore degli interrupt;
- viene eseguita la procedura di servizio;
- viene interrogato lo scheduler, per determinare il prossimo processo da eseguire;
- viene ripristinato il PCB del processo selezionato;
- si esegue il processo.

Come citato precedentemente, i processi pronti vengono inseriti in una coda con priorità, che è un sottoinsieme della tabella dei processi. La coda contiene solo i riferimenti dei processi pronti ad essere elaborati.

Oltre alla coda dei processi pronti, sono presenti anche le **code dei dispositivi**.

Il singolo dispositivo ha a disposizione una coda che contiene i riferimenti dei processi bloccati a causa della risorsa I/O.

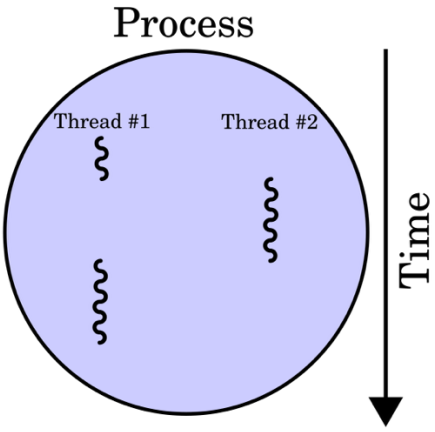
Si fa presente che all'interno del PCB sono presenti 4 puntatori:

- 2 per la coda dei processi pronti(precedente e successivo);
- 2 per la coda dei dispositivi(precedente e successivo).

Il modello dei processi prevede che entità indipendenti, raggruppino risorse e abbiano un unico flusso di esecuzione, detto **thread di controllo**.

Tuttavia, è più utile, rendere condivisibile a più flussi di esecuzione lo stesso spazio di indirizzamento e questo è possibile attraverso i thread. I thread(dall'inglese filo) rappresentano un'unità più piccola del processo(miniprocessi) e permettono di dividere le operazioni in più task.

Un processo può avere uno o più thread che condividono la stessa memoria e le stesse risorse ma vengono eseguiti indipendentemente.



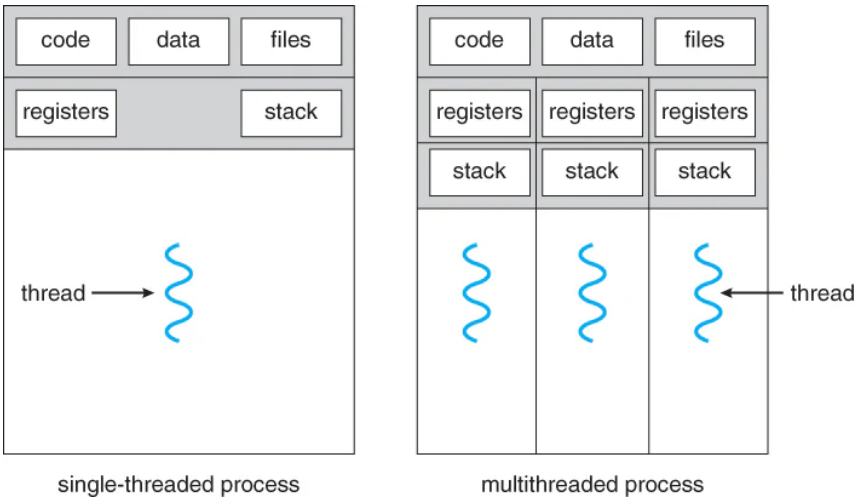
Il loro uso è fortemente influenzato dalle loro proprietà, quali sono:

1) forte cooperazione e condivisione, non esiste la competizione come nei processi
2) leggerezza, permettono una creazione e distruzione molto più veloce;
3) sovrapposizione in ambienti multicore.

In un ambiente **multithreading**, il processo assume il ruolo di contenitore delle risorse condivise, e i thread rappresentano le varie entità di elaborazione all'interno del processo.

I thread condividono lo spazio di indirizzamento e altre risorse, invece, i processi condividono memoria fisica, stampanti, dischi e altre risorse. Per questo motivo, i thread possono essere visti come tanti processi eseguiti in parallelo e spesso sono detti **processi leggeri**.

Un thread, come un processo, è caratterizzato da: PC, registri, stack, stato e da tutte le risorse condivise.



Non c'è protezione tra thread dello stesso processo perché:

1) è impossibile;
2) non sarebbe utile, dato che appartenendo allo stesso processo non dovrebbero entrare in conflitto.

È importante quindi non raggruppare nello stesso processo thread che potrebbero entrare in conflitto, ad esempio se creati da utenti diversi.

I thread come i processi hanno degli stati, quali sono: pronto, bloccato e in esecuzione.

Come citato, i thread condividono la memoria, ma questo è solo limitato alla parte del codice del processo, variabili globali e in generale allo spazio di indirizzamento. Essi non condividono i registri e lo stack, infatti, hanno copie diverse dei registri, al fine di garantire che l'esecuzione sia indipendente e una prevenzione da alterazioni involontarie dello stato di un altro thread.

Anche i thread hanno a disposizione lo scheduler, con le stesse politiche di scheduling dei processi, per l'assegnazione delle CPU.

Preferire un'elaborazione multi-thread è comune, dato che i cambi di contesti(context switching) nei thread sono più veloci. Il **cambio di contesto** è il processo con cui il sistema operativo salva lo stato di esecuzione di un processo e carica quello di un altro.

Per i processi, il cambio di contesto è molto più lento piuttosto che nei thread, perché invece di salvare solo le informazioni relativi ai thread, si salva l'intero spazio di indirizzamento.

Anche l'**MMU** o MEMORY MANAGEMENT UNIT, contribuisce al costo di context switching tra processi, poiché deve gestire la memoria virtuale e tradurre gli indirizzi logici in fisici.

Nei sistemi multithreading, i processi partono dal singolo thread che ha la capacità di generarne altri, attraverso la chiamata **thread_create**. Talvolta i thread possono essere gerarchici, ovvero c'è una relazione genitore-figlio, ma spesso sono tutti uguali.

Quando un thread termina il proprio lavoro, attraverso la chiamata **thread_exit**, esso non è più schedulabile.

In generale ci sono varie chiamate che hanno ruoli diversi, e sono:

CHIAMATA	**DESCRIZIONE**
thread_create	Crea un nuovo thread
thread_exit	Termina il thread
thread_join	Attende che un thread specifico termina(si sincronizza con la fine di un altro thread)
thread_yield	Il thread rilascia volontariamente la CPU ad altri thread
thread_attr_init	Viene inizializzata una struttura dati con valori di default, prima di creare un thread
thread_attr_destroy	Libera le risorse associate alla struttura dati. Pulisce la struttura dati dopo l'uso.

Capire che i thread collaborano piuttosto che competere è fondamentale, specialmente per comprendere la chiamata thread_yield, dove un thread volontariamente rilascia la CPU a thread fratelli. Tutto ciò non è garantito da interrupt gestite dal clock, come nei processi, ma semplicemente da chiamate alla funzione della libreria **phtread**.

ESECUZIONE CON UN SOLO CORE

T1	T2	T3	T4	T5	T6	T7	T8

ESECUZIONE CON PIÙ CORE

T1	T3	T5	T7
T2	T4	T6	T8

Mediante l'esecuzione parallela, l'**hyperthreading**, e l'utilizzo dei thread, si ottiene una vera e propria programmazione parallela.

La programmazione multicore è l'insieme delle tecniche che permettono ad un programma di sfruttare più core fisici e logici della CPU, per eseguire attività in parallelo.

I principi base della programmazione multicore sono i seguenti:

- **separazione dei task** -> prevista una suddivisione equa, dove non accade che ad esempio T1 svolge il 95% del programma e T2,T3,T4 il restante 5%;
- **suddivisione dei dati** -> dobbiamo stabilire la suddivisione dei dati tra i thread assegnando ad ogni thread la struttura dati privata e condivisa(da minimizzare);
- **dipendenze dei dati** -> coordinare l'esecuzione a seconda delle dipendenze tra le operazioni sui dati;
- **test e debugging** -> l'interlacciamento tra i thread è fuori dal controllo del programmatore, dato che dipende dal SO e dall'architettura harware; sono utili fanno test ripetuti al fine di verificare la concorrenza;

I thread si possono implementare in due luoghi: spazio utente e spazio kernel. La scelta è fortemente influenzata dalle esigenze che si hanno.

I **thread a livello utente**(modello 1:n) sono interamente gestiti nello spazio utente, attraverso l'utilizzo di librerie specifiche. Essi vengono creati a run-time e ad ogni processo viene associata una tabella dei thread, con le informazioni di stato.

I thread a livello utente sono **molto efficienti**, perché non viene effettuata alcuna trap, tutto è quindi limitato alle funzioni delle librerie. Possiamo utilizzare i thread a livello utente in sistemi dove non sono previsti, infatti, il sistema che reggerà i thread conosce solo i processi.

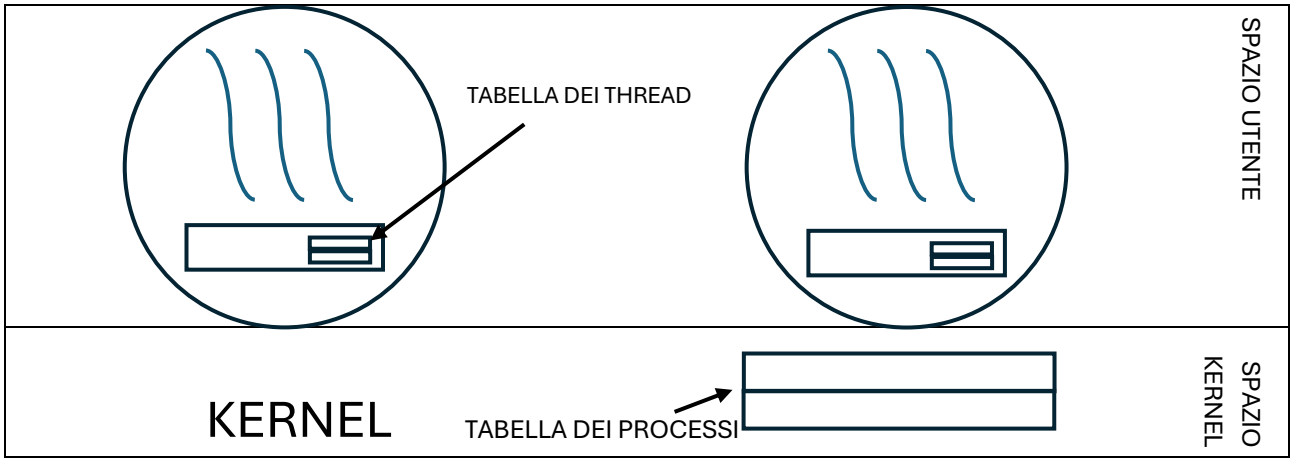
Un altro beneficio di usare i thread livello utente è che lo **scheduling può essere personalizzato**, utile in sistemi dove è presente garbage collection.

Nonostante le prestazioni, dato che il sistema non sa dell'esistenza di questi threa, una read, che è una chiamata bloccante per un processo, potrebbe **compromettere l'esecuzione sia del thread chiamante e sia di tutti i thread fratelli.**

Poiché in realtà la chiamata la fa il processo, si bloccano tutti i thread, perdendo i vantaggi della cooperazione e del parallelismo dei thread.

È un problema fondamentale che si pone e difficilmente può essere aggirato. Solitamente la soluzione è verificare, attraverso una **select**(chiamata di processo che verifica se si può leggere o scrivere un file senza bloccare il processo), se la chiamata read è bloccante; in tal caso, si passa ad un altro thread del processo.

Il vero problema è che le chiamate bloccanti sono causate da errori di pagina o eventi interni al sistema operativo, non rilevabili dal select. Quindi in contesti dove il numero di chiamate bloccanti è imprevedibile o frequente.



I **thread a livello kernel**(modello 1:1) possono essere implementati solo se il kernel li supporta.

Essi sono contenuti nella **tabella dei thread del kernel**, **senza distinzione per processo**. L'utilizzo di un'unica tabella permette quindi di mandare in esecuzione parallela thread di processi diversi e favorire di un'esecuzione più equa; diversa da quella dei thread a livello utente dove venivano eseguiti thread dello stesso processo, finché lo scheduler dei processi non lo destituisce dalla fase di esecuzione.

Uno dei vantaggi dei thread a livello kernel è che **una chiamata bloccante su un thread non intralcia il resto**.

Sebbene venga risolto il problema dei thread bloccanti, non è da trascurare che il cambio di contesto tra i thread è più lento, dato che richiede syscall trap che vanno ad **aumentare l'overhead**. Inoltre, la riprogrammazione dell'MMU potrebbe aggravare la latenza di elaborazione.

Tutte le chiamate sono chiamate di sistema e ciò si traduce in overhead in più, la cui soluzione potrebbe essere il non distruggere i thread, ma bensì si creano **thread che poi si riciclano**.

Spesso il numero di thread creati è pari al numero dei core della macchina, dato che si vogliono ridurre i costi di scheduling.

Oltre ad avere thread a livello kernel o utente possiamo avere un **modello ibrido**(modello n:n), supportati dal kernel, ma che possono essere gestiti anche a livello utente.

Si usano thread kernel solo quando è necessario, evitando overhead, e thread utente quando è possibile usarli senza che si blocchi il processo.

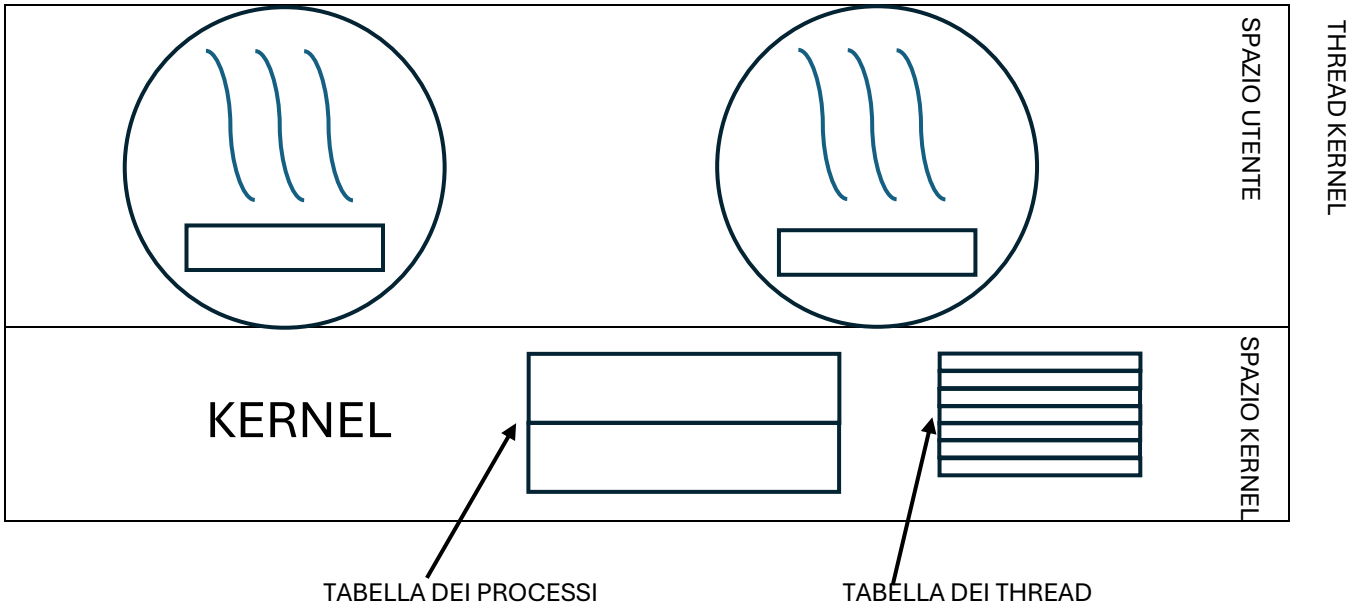
Si prendono quindi i benefici delle tipologie precedenti, che sono:

- processi non bloccati in presenza di chiamata bloccante(thread kernel);
- assenza di chiamate TRAP(thread utente).

Modello 1:n -> ad un solo thread kernel corrispondono più thread a livello utente;

Modello 1:1 -> ad un solo thread kernel corrisponde un thread utente;

Modello n:n -> a più thread kernel corrispondono più thread a livello utente;



I processi, al fine di evolversi, necessitano continuamente di comunicare con altri processi. Un esempio tipico è rappresentato dalla pipeline nella shell, dove l'output di un processo è l'input di un altro.

Questa necessità porta all'**Inter-Process Communication(IPC)**, cioè la comunicazione tra processi, che deve avvenire in modo strutturato e non deve incorrere a interrupt.

La comunicazione presenta però tre problemi fondamentali, quali:

- passaggio di informazioni da un processo ad un altro;
- evitare interferenze tra processi(vale pure per i thread);
- corretta sequenzialità nell'esecuzione(vale pure per i thread).

```
santolobianco@MBP-di-Santo ~ % cmd1 | cmd2 | cmd3
```

In questo modo vengono eseguiti sequenzialmente i comandi e l'output del precedente è l'input del successivo.

Il coordinamento è dato dalla natura bloccante dei processi. Se cmd2 non può procedere per dipendenza di dati, aspetta che cm1 venga eseguito.

Le problematiche prima esposte, legate allo scambio, si possono risolvere mediante l'uso di un **buffer condiviso**, dove vengono conservati i dati prodotti da un processo. Il buffer in questo caso funge da intermediario nello scambio di dati.

Tuttavia, quando più processi contemporaneamente accedono alla stessa area di memoria, si possono verificare situazioni di **Race Condition**, dove l'ordine di accesso ai dati può influenzare i risultati.

Nel Kernel soprattutto sono particolarmente gravi le Race Condition, dato che potrebbero causare la compromissione o il crash del sistema.

Per evitare queste situazioni, si deve garantire la mutua esclusione all'accesso della risorsa, ovvero l'accesso di un solo processo per volta alla risorsa. Questa è una delle problematiche che deve affrontare il sistema operativo,

Nella programmazione concorrente si deve fare una netta separazione tra:

- sezione critica, la parte di codice del processo, che accede alla memoria condivisa;
- sezione non critica, tutto il resto del codice.

Per garantire un'ottima mutua esclusione, si tiene in conto che:

- solo un processo deve essere nella sua zona critica
- non si conoscono né l'architettura e né le prestazioni dell'elaboratore(numero core e velocità);
- l'accesso deve essere limitato al solo processo;
- un processo non deve aspettare un tempo di accesso eccessivo per la risorsa.

Il programmatore definisce le zone critiche sulla base delle strutture dati utilizzate dei processi. L'individuazione di zone critiche in più, che potrebbero essere non critiche, può portare ad una perdita di efficienza.

Per la realizzazione della mutua esclusione, esistono diverse strategie generali, che poi vengono implementate nel dettaglio con modalità diverse a seconda del contesto:

- **disabilitazione dell'interrupt**: viene inibito il meccanismo di interrupt in modo tale che il processo che è nella zona critica non venga interrotto. Non funziona nei sistemi multicore, perché l'interlacciamento è diverso in caso di programmazione parallela. Inoltre, è pericoloso che i processi possono disabilitare gli interrupt, perché se un processo non li riabilitasse, il sistema cesserebbe di funzionare.
- **introduzione variabile di lock**: si utilizza una variabile che funge da guardia e specifica se la zona critica è accessibile. Il problema che si presenta è che esisterà una zona critica per l'accesso alla variabile di lock.

- **alternanza stretta**: è una tecnica facilmente implementabile per due processi e prevede un'alternanza rigida tra di essi. Un processo che ha precedentemente acquisito l'accesso alla propria sezione critica bloccherà l'altro processo finché non uscirà da quella sezione, anche se non è più interessato ad accedervi. Nel frattempo, il processo bloccato rimane in esecuzione, effettuando **busy waiting** (attesa attiva), effettuando un controllo continuo sul valore della variabile di lock. Questo monitoraggio costante viene chiamato **spinlock**.

PSEUDO-CODICE ALTERNANZA STRETTA
int n = 2;
int turn;
function enter_region(int process){
  while(turn != process) do nothing
  //.... zona critica
}
 	
function leave_region(int process){
  turn = 1-process;
}
	Il codice in questione garantisce mutua esclusione per l'accesso alla zona critica, seppur con problemi di busy waiting.
	È pensato per due processi, ovvero 0 e 1, e nella leave region si alterna il turno dei processi.
	Il problema grave è che un processo potrebbe bloccare gli altri, nonostante non acceda alla propria zona critica.

Sulla base dell'alternanza stretta si sono create soluzioni più robuste, una tra queste è la soluzione di Peterson.

La soluzione di Peterson, anch'essa pensata per due processi, ha risolto l'inconsistenza grave dell'alternanza stretta, ovvero che anche se un processo non intende accedere alla zona critica, ma è il suo turno, allora gli altri saranno bloccati.

int N = 2;
int turn;
int interested[N];
function enter_region(int process){
  int other = 1-process;
  int turn = process;
  interested[process] = true;
  while(interested[other] == true && turn == process)
  do nothing
}
function leave_region(int process){
  interested[process] = false}
	Il busy waiting è presente, ma almeno si evitano blocchi inutili e migliora l'efficienza.
	Se un processo non intende accedere alla sua zona critica, l'altro non viene bloccato.
	Può essere esteso al caso N, ma non è preferito, data la presenza di algoritmi più robusti.
	Può avere problemi su sistemi multi-processore a causa del riordino delle istruzioni e degli accessi alla memoria centrale.

I computer multiprocessore solitamente offrono una specifica istruzione hardware utile ad implementare la mutua esclusione, ovvero l'istruzione **TSL**(test and set lock). Questa istruzione prevede un intervento da parte dell'hardware, infatti, al momento che viene eseguita, dovranno essere disabilitati i bus di memoria(per la scrittura o lettura in memoria limitata al solo processore) agli altri processori presenti nel sistema.

Si preferisce la disabilitazione dei bus anziché degli interrupt, perché in ambienti con più processori, la disabilitazione degli interrupt è circoscritta solo al processore che esegue l'operazione, senza che si impedisca l'accesso concorrente da parte di altri processori.

Nel dettaglio l'operazione TSL svolge un insieme di operazioni, indivisibili, che prevedono:

- disabilitazione dei bus di memoria;
- lettura del valore della variabile lock e scrittura in un registro apposito;
- scrittura di un valore diverso da 0 nella variabile lock
- riabilitazione dei bus di memoria.

enter_region:
TSL REGISTER, LOCK
CMP REGISTER,#0
JNE enter_region
RET
leave_region:
MOVE LOCK,#0
RET
	Il problema principale della TSL è la presenza di busy waiting causata dal ciclo che si può incorrere prima di entrare nella zona critica.
	Tuttavia, attraverso l'intervento dell'hardware si evitano i problemi legati alla concorrenza tra processori per l'uso della memoria condivisa, e l'eventuale inconsistenza dei dati.

In alternativa alla TSL si può utilizzare l'istruzione **XCHG**(exchange) che permette di effettuare lo scambio del contenuto di due operandi in modo atomico, senza disabilitazione di interrupt o bus.

In tutte le soluzioni proposte prima è presente busy waiting(attesa attiva), infatti, si è pensato di usare le primitive **sleep** e **wakeup** dando la possibilità al processo di bloccarsi in modo passivo.
Il processo si bloccherà in modo passivo attraverso la rimozione dai processi pronti e poi la reintroduzione quando verrà svegliato.

Questo serve anche a limitare anche situazioni di proprietà inversa di priorità, in sistemi dove esiste la priorità.
Infatti, se si considerano due processi, con priorità alta H e bassa L, H verrà eseguito prima di L se tutti e due sono presenti nella coda dei processi pronti.
Se H non è presente nella coda dei processi pronti ed L viene eseguito per primo, H sarà costretto ad aspettare che L esca dalla sua sezione critica, non rispettando la priorità.

Per comprendere meglio l'applicazione delle primitive sleep e wakeup, si analizzerà il problema dei produttori-consumatori.

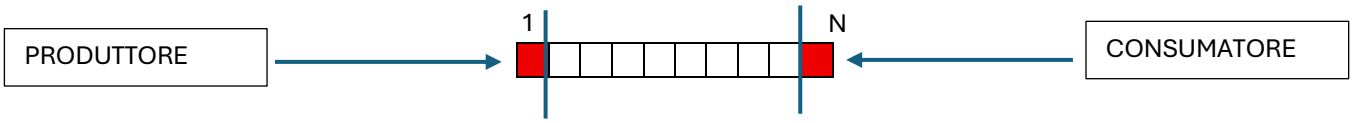
I casi oggetto presentati durante tutto il corso serviranno a comprendere al meglio le situazioni di race condition e altri problemi tipici legati alla programmazione multiprocesso, mostrando anche delle soluzioni valide per affrontarli.

Nel caso prima citato, ovvero il caso dei **produttori-consumatori**, si esamina il problema di **bounded-buffer** o di buffer limitato, attraverso il semplice esempio di produttori che immagazzinano informazioni nel buffer e di consumatori che le prelevano.

Nonostante il paragone semplice, è importante capire la logica dei problemi legati ad un buffer limitato e le soluzioni. Infatti, potremmo avere casi dove il produttore, nonostante il buffer sia pieno, tenti di inserire oppure che il consumatore, con il buffer vuoto, cerchi di prelevarne. A fronte di questi problemi si usano dei contatori e le primitive sleep e wakeup: quando si raggiunge uno dei due estremi del buffer, i produttori o i consumatori vengono sospesi(sleep) e riattivati(wakeup) quando le condizioni lo permettono.

Un altro problema fondamentale legato all'uso di queste primitive si verifica quando entrambe le parti agiscono a poca distanza di tempo, trovandosi in condizioni estreme.

Ad esempio, se il buffer è vuoto, il consumatore entra in sleep; tuttavia, se nel frattempo un produttore incrementa il buffer e invoca wakeup, ma il consumatore non è ancora effettivamente entrato in stato di sleep, la chiamata a wakeup andrà persa.
Per affrontare questo problema si utilizza un bit di wakeup, in modo tale che, se i consumatori non sono ancora dormienti e i produttori lanciano una wakeup, essa non andrà persa.



Una generalizzazione che si può fare delle primitive sleep e wakeup, è attraverso l'utilizzo dei **semafori**. I semafori sono delle variabili intere condivise che permettono la sincronizzazione tra più processi, attraverso l'utilizzo delle primitive **down()** e **up()** (wait e signal). Se la variabile diventa negativa, allora il processo verrà sospeso; invece, se ci sono processi bloccati, uno tra questi verrà risvegliato.
I semafori sono importanti perché sono implementati attraverso TSL/XCHG che interrompono il bus dati, durante una transizione di stato, garantendo atomicità dell'operazione.

```
#define N 100
typedef in semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void){
    int item;
    while(true){
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item); //zona critica
        up(&mutex);
        up(&full);
    }
}

void consumer(void){
    int item;
    while(true){
        down(&full);
        down(&mutex);
        item = remove_item();//zona critica
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

Il semaforo mutex è un semaforo binario(assume due valori), che garantisce la mutua esclusione: assumendo il valore 1 quando si può accedere alla zona critica e 0 quando già è occupata . Il semaforo empty garantisce quanti posti liberi sono presenti all'interni del nostro buffer, invece, il full indica quanti posti occupati.
È importante l'ordine esplicitato, perchè ad esempio nel caso producer:

1. se non vi sono posti vuoti(empty == 0), si attiverà la sleep e quindi non si entrerà nella zona critica;
2. solo quando un consumatore rimuoverà un elemento dal buffer(empty incrementato), il produttore potrà inserire un nuovo elemento
3. dopo l'inserimento si esce dalla zona critica;
4. e si incrementerà il numero di posti occupati.

Viceversa, per il consumer, accade questo:

1. se non vi sono posti occupati(full == 0), il consumatore non potrà prelevare la risorsa e si attiverà la sleep;
2. solo quando un produttore inserirà un nuovo elemento nel buffer(full incrementato), il consumatore potrà rimuovere l'elemento, entrando nella zona critica;
3. dopo l'inserimento si uscirà dalla zona critica;
4. e si incrementerà il numero di posti vuoti.

Un ulteriore motivo, per il quale si deve seguire l'ordine esplicitato nell'esempio, è che **non si deve mai mettere un'operazione che potrebbe risultare bloccante all'interno della zona critica, perché si blocca tutto in cascata**.

Il semaforo mutex, prima citato è un ottimo per gestire la mutua esclusione, garantendo l'accesso alla zona critica ad un processo/thread per volta. Esso ha due stati **unlocked(0)/locked (other)** ed è molto efficiente nei thread a livello utente.

Se si utilizza enter_region, il thread fa busy waiting finchè il thread non entra nella zona critica; invece, con l'utilizzo del mutex, quando un thread non può entrare nella zona critica rilascia la CPU(thread_yield) ad altri processi e viene sospeso, finché non avrà la disponibilità di entrare nella zona critica.

Il busy waiting può essere deleterio nei thread utente: se un processo viene sospeso, vengono sospesi anche gli altri thread dello stesso processo, impedendo ad esempio a un thread fratello già nella zona critica di completare l'operazione.

Per questo motivo è stata ideata una soluzione ibrida: il **futex (fast user-space mutex)**. Si tratta di un mutex implementato sia a livello kernel sia in user space, che gestisce la maggior parte delle operazioni in user space rendendolo molto veloce. Solo se il futex è già occupato viene effettuata una syscall, che sospende il thread richiedente e lo risveglia quando il lock diventa disponibile.

Come visto precedentemente, l'ordine dei down e up per rimuovere o aggiungere un elemento all'interno del buffer è importante. Quindi, quando si programma un sistema con i semafori, bisogna prestare attenzioni a tanti problemi che possono causare deadlock. Per semplificare la scrittura di programmi, Hansen e Hoare, proposero una primitiva di sincronizzazione di livello superiore chiamata monitor.

Un **monitor** è una raccolta di procedure, variabili e strutture dati, raggruppati in un tipo speciale di modulo. Attraverso la proprietà del monitor, ovvero che al suo interno deve essere attivo un solo processo, si garantisce la mutua esclusione. Le chiamate di procedure dei monitor sono diverse da quelle standard, perché attraverso il compilatore si verifica che non vi sia un altro processo attualmente attivo. Essi sono costrutto del linguaggio di programmazione, infatti, l'utente non deve preoccuparsi dell'implementazione della mutua esclusione, il compilatore si occupa di essa.

La sincronizzazione, ossia il quando e come un processo/thread può accedere alle risorse del monitor, è implementata mediante le **variabili condizione**. Le variabili condizioni (sono dei segnali logici(etichette)) sono dei meccanismi di sincronizzazione che serve ai thread di aspettare una condizione, consentendo ad un altro thread di notificare la condizione verificata. L'implementazione è consentita anche attraverso le primitive wait e signal, che rispettivamente bloccano un thread e lo riattivano. Così si elimina il busy waiting, perché i thread si addormentano e vengono risvegliato quando c'è qualcosa da fare.

I monitor possono avere signal con semantiche diverse:

MONITOR HOARE(TEORICO): signal & wait	T1 attiva la signal T2, passando subito il contorollo a T2, sospendendosi.
MONITOR MESA(JAVA): signal & continue	T1 attiva una signal per T2, continuando la sua esecuzione. T2 attende il lock.
COMPROMESSO(CONCURRENT PASCAL): signal & return	T1(terminando la sua esecuzione) attiva T2 e passa il lock, T2 viene messo nella coda dei threa pronti. Una via di mezzo.

I vantaggi che si hanno con l'uso dei monitor sono:

- mutua esclusione automatica;
- astrazione di alto livello
- integrazione con il linguaggio

Gli svantaggi che limitano i monitor sono i seguenti:

- poca flessibilità(non adattabili a sistemi complessi);
- dipendenza dal linguaggio, es. C e C++ non li prevedono;
- data la memoria privata(prevista dalla struttura) si possono avere problemi legati alla condivisione della memoria.

monitor pc_monitor
condition full, empty;
integer count = 0;
 	
function insert(item){
if (count == N) then **wait**(full);
insert_item(item);
count = count +1;
if(count == 1) then **signal**(empty);
}
 	
function remove(){
if (count == 0) the **wait**(empty);
item = remove_item();
count = count -1;
if (count = N-1) then **signal**(full);
}
 	
function producer(){
while(true){
item = produce_item();
pc_monitor.insert(item);
}
}
 	
function consumer(){
while(true){
item = **pc_monitor**.remove();
consume_item(item);
}
}
	In questo pseudo-codice che imita la funzione dei monitor, possiamo vedere come attraverso di funzioni e strutture private legate ad esso, si possono gestire problemi come il problema del produttore-consumatore.
	Si creano le due condizioni, full e empty, che serviranno a verificare se possiamo continuare ad agire con un thread oppure no.
	Infatti, nel caso in cui si raggiunga la capienza massima(N) viene bloccato il thread incaricato ad effettuare le insert. Il thread verrà risvegliato quando ci sarà almeno un produttore che consuma una risorsa e che quindi liberi un posto.
	Invece, nel caso in cui si consumino tutte le risorse, il thread incaricato a rimuovere risorse viene sospeso. Verrà risvegliato quando verrà aggiunta almeno una risorsa.
	Tutte le modifiche in memoria, vengono effettuate univocamente, perché stiamo operando attraverso i monitor.

Un'alternativa molto valida sia ai monitor e sia ai semafori è lo **scambio di messaggi**. Attraverso l'uso di primitive(non appartenenti allo standard POSIX) di alto livello quali **send**(destinazione, messaggio) e **receive**(sorgente, messaggio) è permesso la comunicazione tra processi, ergo la sincronizzazione.

Operazione send:

- manda un messaggio al destinatario;
- se il buffer dei messaggi del destinatario è pieno, il mittente non può inserire un nuovo messaggio;
- se il buffer dei messaggi del destinatario è pieno, si può restituire un errore(se implementato).

Operazione receive:

- legge un messaggio inviato da un processo sorgente;
- se non ci sono messaggi disponibili, il chiamante può bloccarsi o restituire un errore, finché non arriva un messaggio.

Queste primitive si possono usare su più macchine tramite librerie tipo MPI, permettendo la comunicazione distribuita.

Il mittente per essere sicuro della ricezione del messaggio da parte del destinatario aspetta un **messaggio di acknowledgement** entro un certo intervallo di tempo, altrimenti ritrasmette.

Si può intraprendere una comunicazione diretta, dove mittente e destinatario si conoscono, oppure preferire l'uso di una **mailbox**.

La comunicazione diretta è semplice da realizzare però non è bufferizzata ed è molto rigida, quindi se un processo e in receive, deve aspettare che una send arrivi. La cardinalità tra i processi è 1:1, quindi ad un destinatario corrisponde una sorgente.

Diversa è la comunicazione attraverso la mailbox poiché si utilizza un buffer dove vengono contenuti temporaneamente tutti i messaggi. I messaggi all'interno della mailbox sono accessibili da tutti i processi, che vi possono accedere alla coda dei messaggi, e inoltre questa modalità di inter process comunication rende bloccante la chiamata send(se il buffer è pieno, la sorgente dovrà aspettare che si liberi). La cardinalità tra i processi è 1:molti, quale sia 1 sorgente e molti destinatari(broadcast) e quale siano molte sorgenti e 1 destinatario.

Indipendentemente dalle implementazioni usare lo scambio di messaggi, come IPC non è vantaggioso, dato che il contenuto del messaggio verrà esaminato e copiato varie volte, inoltre, le chiamate send e receive dato che sono syscall causano overloanding in più.

Lo scambio di messaggi è efficiente nel caso in cui ci siano più macchine coinvolte, dato che per singole macchine sono più efficienti monitor e semafori.

```
function producer(){
        while(true)do
                item = produce_item();
                build_msg(m,item);
                send(consumer,msg)
}

function consumer(){
        while(true)do
                receive(producer,msg);
                item = extract_msg(msg);
                consum_item(item);
}
```

Durante il capitolo, si sono esaminate le varie strategie applicate al problema consumatori-produttori(bounded-buffer), ma vi sono altri problemi/casi oggetto che ci fanno comprendere i problemi legati alla condivisione di dati ecc. Tali problemi sono:

- problema dei 5 filosofi;
- problema lettori-scrittori.

Il **problema dei 5 filosofi**, è un grattacapo ideato da Dijkstra che riguarda l'accesso esclusivo ad un numero limitato di risorse da parte di processi in concorrenza.

"Ci sono 5 filosofi seduti in un tavolo circolare, con davanti un piatto di pasta e una forchetta. Essi possono o meditare o mangiare, ma per mangiare hanno bisogno di due forchette, quella a destra e sinistra."
Si utilizzano 5 filosofi, perché è il numero minimo dove al massimo possono mangiare 2 filosofi; quindi, il problema nasce quando più di due filosofi vogliono mangiare.
Come possiamo garantire che non ci sia starvation di alcun filosofo nell'attesa delle forchette per mangiare?

Possibili soluzioni pratiche:

- **Uno alla volta mangia** -> assenza di conflitti ma perdita di efficienza;
- **Controllo con rilascio, riprova tempo preimpostato** -> filosofi si svegliano prendono la forchetta alla loro sinistra, e se quella a destra e libera potranno mangiare, altrimenti rilasceranno la forchetta presa e torneranno a meditare -> si può incorrere in un ciclo se tutti prendono le forchette contemporaneamente;
- **Controllo con rilascio, riprova tempo randomizzato** -> soluzione simile alla precedente, solo che attraverso la randomizzazione del tempo per il nuovo tentativo, si evitano loop -> possiamo avere forchette inutilizzate ovvero perdita di efficienza;
- **Utilizzo di un semaforo mutex/monitor** -> 3 stati ovvero: **thinking(0)**, **hungry(1)**, **eating(2)** -> rischio starvation se la scelta dei risvegli non è equa.

```
int N = 5;
int thinking = 0;
int hungry = 1;
int eating = 2;
int state[N];
semaphore mutex = 1;
semaphore s[N] = {0,0,0,0,0}

function left(int i){
        return i-1 mod N;
}
function right(int i){
        return i+1 mod N;
}

function test(int i){
        if(state[i] == HUNGRY  && state[left(i)] != EATING &&
        state[right(i)] != EATING){
                state[i] = EATING;
                up(s[i]);
        }
}
function take_forks(int i){
        down(mutex);
        state[i] = HUNGRY;
        test(i);
        up(mutex);
        down(s[i]);
}

function put_forks(int i){
        down(mutex);
        state[i] = THINKING;
        test(left(i));
        test(right(i));
        up(mutex);
}

function philosopher(int i ){
        while(true) do{
                think();
                takefork();
                eat();
                put_forks();
        }
}
```

Soluzione con i semafori

Come citato prima vi sono tre stati:

- Thinking(0)
- Hungry(1)
- Eating(2)

Si utilizza un semaforo mutex, che permette di accedere univocamente alla zona critica, e un array di semafori che bloccano/sbloccano i filosofi.

La funzione **test** verifica se i filosofi accanto al filosofo interessato stanno mangiando, se non è così ed esso è affamato, gli è concesso di prendere le forchette e mangiare.

La funzione **take_forks** entra nella zona critica e chiama la funzione test. Se il filosofo può mangiare esso sarà sbloccato e mangerà, altrimenti alla fine della funzione stessa verrà bloccato finché un filosofo vicino lo riattiva.

La funzione **put_forks** entra nella zona critica, cambia lo stato del filosofo e verifica se i filosofi che sono vicini al filosofo in questione possono mangiare.

Infine, tutte le chiamate di funzione vengono gestite dalla funzione **philosopher**.

```
int N = 5;
int thinking = 0;
int hungry = 1;
int eating = 2;

monitor dp_monitor{
	int state[N];
	condition self[N];

	function take_forks(int i){
		state[i] = hungry;
		test(i);
		if(state[i] != eating){
			wait(self[i]);
		}
	}

	function put_forks(int i){
		state[i] = thinking;
		test(left[i]);
		test(right[i]);
	}

	function test(int i ){
		if(state[i] == hungry && state[left(i)] !=
		eating && state[right(i)] != eating){
			state[i] = eating;
			signal(self[i]);
		}
	}

	function philosopher(int i){
		while(true) do{
			think();
			dp_monitor.take_forks();
			eat();
			dp_monitor.put_forks();
		}
	}
}
```

Soluzione con i monitor

Si stabiliscono gli stati quali sono thinking, hungry ed eating. Dentro al monitor si dichiara l'array contenente gli stati e l'array di variabili di condizione(analogamente ai semafori, permettono di accedere alla zona critica e di bloccare altri processi). La funzione take_forks permette di cambiare lo stato in hungry, e attraverso la chiamata alla funzione test si verifica se il filosofo può accedere alla zona critica. Se non è possibile allora il processo verrà bloccato. Attraverso la funzione put_forks si cambia lo stato del filosofo che rilascia le forchette, e si verifica se i filosofi accanto possono mangiare, se è così verrano risvegliati e potranno mangiare.

L'array di variabili di condizione, permette di gestire la sincronizzazione, permettendo ad un processo piuttosto che ad un altro di accedere.

La differenza tra le due soluzioni sta nel fatto che l'esclusione per l'accesso della risorsa nel primo caso viene gestita dal programmatore, invece, nel secondo caso viene gestita dal compilatore. L'efficienza è anche un importante valore per effettuare un confronto, infatti, dato che con i monitor tutto è gestito a runtime si ha un maggiore overhead, ciò vuol dire che l'utilizzo dei semafori è più efficiente.

Un ulteriore problema che si presenta nella progettazione multi-processo è il **problema lettori-scrittori**. L'accesso allo stesso buffer di dati è esclusivo agli scrittori e condiviso tra i lettori. Più lettori possono leggere dati contemporaneamente invece, per gli scrittori è consentito l'accesso ad un solo scrittore(in assenza di lettori). La soluzione, se in prima approssimazione potrebbe sembrare facile, perché basta avere un counter di lettori che quando è 0 consente ad eventuali scrittori di scrivere, presenta un problema più vasto.

Nella soluzione si deve fare una scelta: garantire un accesso privilegiato agli scrittori oppure ammettere nel nostro sistema la **starvation**. Ebbene si, perché se non si concede un accesso privilegiato agli scrittori, possono accedere lettori ripetutamente, dato che la lettura condivisa, impedendo così gli scrittori di entrare e bloccandoli indefinitamente.

Utilizzando i semafori mutex, si ha starvation, dato che non viene esplicitata una priorità per gli scrittori.

Di seguito viene proposta una soluzione per evitare che i processi di scrittura attendano un tempo indefinito, garantendogli una priorità.

```
monitor rw_monitor{
        int rc = 0;
        boolean busy_on_write = false;
        condition read,write; //variabili condizione
        function start_read(){
                if(busy_on_write || in_queue(write)){
                        wait(read);
                }
                rc++;
                signal(read);
        }

        function end_read(){
                rc--;
                if(rc == 0){
                        signal(write);
                }
        }

        function start_write(){
                if(rc > 0 || busy_on_write){
                        wait(write);
                }
                busy_on_write = true;
        }

        function end_write(){
                busy_on_write = false;
                if(inqueue(write)){
                        signal(write);
                }else{
                        signal(read);
                }
        }
}
```

```
function reader(){
        while(true) do{
                rw_monitor.start_read();
                read_database();
                rw_monitor.end_read();
                use_data_read();
        }
}

function writer(){
        while(true) do{
                think_up_data();
                rw_monitor.start_write();
                write_database();
                rw_monitor.end_write();
        }
}
```

Il monitor gestisce l'accesso concorrente a una risorsa condivisa, permettendo a più lettori di leggere contemporaneamente e garantendo accesso esclusivo agli scrittori. La sincronizzazione tra lettori e scrittori evita conflitti e garantisce sicurezza dei dati.
I principali punti critici riguardano possibili situazioni di blocco o starvation: se non gestita correttamente, la priorità tra lettori e scrittori può portare a lettori o scrittori che rimangono troppo a lungo in attesa. Inoltre, un errore nel conteggio dei lettori attivi o nella gestione delle variabili di condizione potrebbe creare deadlock. Il monitor, se implementato correttamente, bilancia però accesso concorrente e mutua esclusione, minimizzando i rischi di attesa indefinita.

Anche se viene garantita priorità agli scrittori, i lettori agirano sempre in lettura condivisa, quindi la disparità tra le due entità non è visibile, nonostante è garantito un accesso privilegiato ad una delle due entità.

Quando un computer è multi-programmato, processi e thread competono tra loro per ottenere la CPU. La parte del sistema operativo incaricata a scegliere i processi che dovranno andare in esecuzione è chiamata scheduler e l'algoritmo che si utilizza è chiamato algoritmo di scheduling. Molti problemi che riguardano lo scheduling dei processi riguardano anche lo scheduling dei thread.

Lo scheduler deve garantire equità nell'utilizzo della CPU da parte dei processi e al contempo deve fare un uso efficiente della CPU. Ogni qual volta che c'è un cambio di contesto(context switching o process switch) si devono considerare delle operazioni di routine che vengono svolte, che appesantiscono il lavoro della CPU e incrementano l'overhead. Queste operazioni sono:

- Salvataggio dei dati del processo in transizione;
- Salvare la mappa della memoria;
- Riprogrammare l'MMU;
- Avviare il nuovo processo;

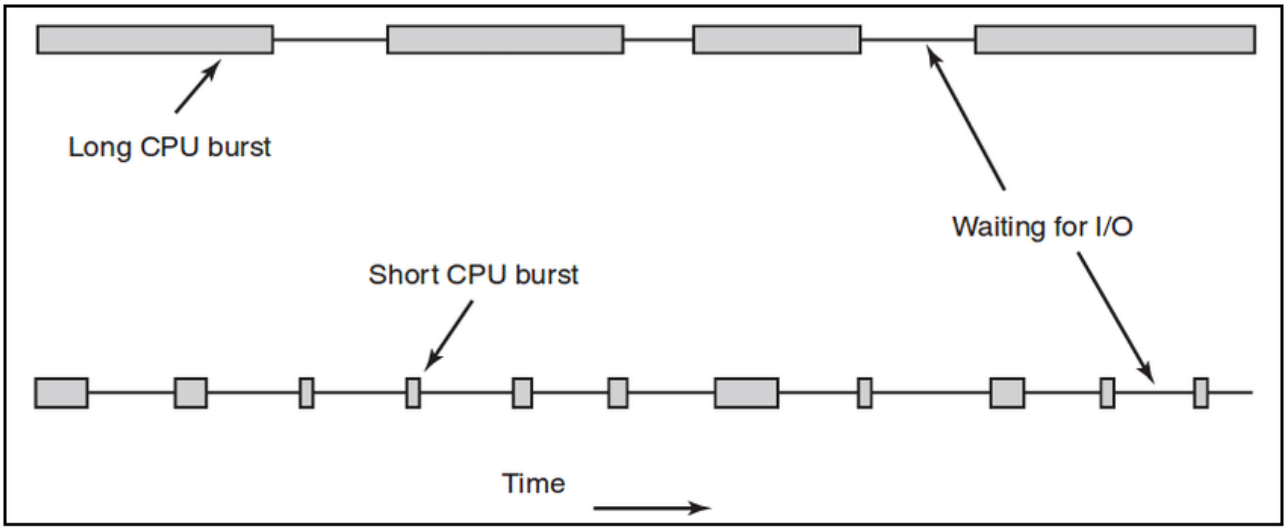
Il componente del sistema operativo che si occupa di gestire queste operazioni è il **dispatcher**. L'overhead che si crea a causa del cambio di un processo si chiama **latenza di dispatch**.

Fare troppi cambi di processo potrebbe consumare una quantità considerevole di tempo della CPU, quindi è opportuno fare attenzione.

Lo scheduler è utile principalmente in main-frame o server web, che gestiscono più processi contemporaneamente, situazione diversa nei personal computer, dove un utente gestisce un'attività alla volta e che quindi è impossibilitato ad avere più processi attivi contemporaneamente.

Quasi tutti i processi alternano una fase di burst di elaborazione a richieste di I/O, quindi, si può fare una distinzione tra i processi, ovvero:

- **CPU-bounded;**
- **I/O-bounded;**



Come si evince dall'immagine i processi che sono prettamente CPU sono quelli che occupano più tempo di esecuzione, ma hanno attese di I/O poco frequenti, invece, i processi I/O sono i più veloci da eseguire ma hanno attese frequenti di I/O. Inoltre, aumentando la velocità di esecuzione di una CPU si riducono i tempi di esecuzione di un processo, e quindi con il tempo si potrebbe verificare la condizione dove lo scheduling dei processi I/O-bounded potrebbe rivelarsi più importante.

Le situazioni in cui è necessario schedulare sono varie, infatti si schedula quando:

- Processo termina la propria esecuzione;
- Processo non può terminare perché si blocca sull'I/O.

Possiamo avere situazioni di scheduling **non preemptive**(senza prelazione) e **preemptive**(prelazione), ovvero:

- **non preemptive** -> i processi in esecuzione o terminano oppure si bloccano in autonomia(a causa della mancanza di risorse);
- **preemptive** -> lo scheduler utilizza l'interrupt che fornisce il clock hardware, per stipulare un tempo di esecuzione del processo, se esso non è stato terminato verrà sostituito ugualmente con un altro processo appartenente alla coda dei processi pronti. Questo permette di non monopolizzare la CPU e quindi garantire un'esecuzione equa.

La prelazione non è rilevante solo per le applicazioni, ma anche per i kernel, dato che un driver mal funzionante oppure una chiamata di sistema molto lenta potrebbe intasare la CPU.

In ambienti diversi sono necessari algoritmi di scheduling diversi, dato che diverse applicazioni hanno obiettivi diversi. Tre ambienti che si devono distinguere sono:

- **batch** -> eseguono una mole di job senza l'intervento dell'utente;
- **interattivo** -> interazione continua tra utente e computer durante l'esecuzione;
- **real-time**- > devono rispettare dei tempi precisi, l'operazione deve essere eseguita entro un tempo massimo.

Gli obiettivi comuni:

- **equità nell'assegnazione della CPU** -> dare priorità a processi IO bounded è un primo passo per l'equità, dato che CPU-bounded tende a causare starvation per gli altri tipi di processi;
- **bilanciamento nell'uso delle risorse** -> distribuisce il carico di lavoro tra risorse disponibili, evitando sovraccarichi su alcune parti del sistema.

Obiettivi nei sistemi batch:

- **massimizzare il throughput**-> numero di processi eseguiti in un lasso di tempo;
- minimizzare il tempo di tournaround-> tempo di attesa che venga eseguito il processo e l'esecuzione stessa; conseguenza ->**minimizzare il tempo di attesa**.

La massimizzazione del throughtput non è l'obiettivo principale, altrimenti si potrebbero preferire processi più brevi da eseguire, causando una starvation nei confronti di processi che richiedono più tempi di esecuzione.

Obiettivi nei sistemi interattivi:

- **minimizzare il tempo di risposta** -> il sistema è in continua comunicazione con l'utente, quindi, si potenzia lo scambio dei messaggi , con la minimizzazione dei tempi di risposta;
- **adeguatezza** -> l'utente si aspetta lassi di tempo brevi o lunghi a seconda dell'operazione, è bene quindi soddisfare le aspettative, quindi, per soddisfare delle aspettative il sistema deve rispettare delle scadenze.

Obiettivi nei sistemi real-time:

rispetto delle scadenze -> azione eseguita correttamente in un tempo prestabilito(airbag della macchina);

prevedibilità -> il tempo di esecuzione deve essere noto e costante-> non ci devono essere variazioni nel tempo di esecuzione.

Scheduling nei sistemi batch

Vedendo gli obiettivi che si devono raggiungere nei sistemi batch, sono proposti i seguenti algoritmi di scheduling:

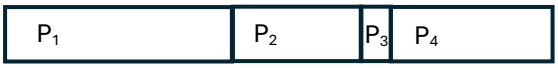
- **First-Come First-Served**(FCFS) -> semplice coda FIFO dove il primo processo che arriva viene servito, non c'è prelazione, quindi, se un processo viene iniziato a meno che non viene bloccato da una richiesta I/O, esso deve terminare. I problemi di questo algoritmo sono legati alla rigidità che lo caratterizza, infatti, abbiamo che: un processo lungo può bloccare tutti quelli corti e un tempo medio di attesa molto alto.
- **Shortest Job First(SJF)** -> presuppone che si conosca già in anticipo la durata di esecuzione(burst time) di ogni processo in coda, e si preleva tra tutti quello più breve. L'ordine si decide prima che avvenga l'esecuzione, e se arriverà un processo con tempo di esecuzione minore del processo che è attualmente in esecuzione, sarà costretto ad aspettare. Non c'è prelazione anche in quest'algoritmo ed è per questo che è ottimale se tutti i processi arrivano in tempo 1. I problemi principali sono: la conoscenza richiesta del tempo di esecuzione(è per questo molto teorico), possibile starvation per i processi lunghi e la scarsa adattabilità negli scenari dinamici.
- **Shortest Remaining Time Next(SRTN)** -> versione con prelazione dello SJF, quindi, se un processo arriva in un tempo n e ha tempo di esecuzione minore rispetto al processo in esecuzione, si sostituirà il processo in corso con il nuovo arrivato. Si avrà un maggiore overhead causato dal frequente context switch e possibile starvation per processi lunghi.

Processo	Arrivo	Durata
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

Facendo riferimento a questa tabella, dove sono espressi arrivo e il tempo di esecuzione(supponendo che il nostro sistema in anticipo conosca quanto tempo verrà impiegato), possiamo confrontare gli algoritmi sopraccitati.
Si calcola il tempo di attesa medio e il tempo di completamento(si ricorda che nel tempo di completamento è previsto anche il tempo di attesa) attraverso le seguenti formule:

- **tempo di attesa medio** -> somma tempi di attesa/numero processi;
- **tempo di completamento medio** -> ((somma tempi di attesa)+(somma tempo di esecuzione))/numero di processi;

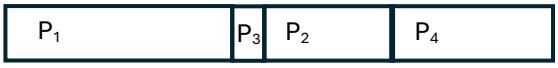
FCFS



Tempi di attesa: $P_1 = 0$, $P_2 = 5$, $P_3 = 7$, $P_4 = 7$ somma = 19 media tempi di attesa = 19/4 = 4,75

Tempi di completamento: $P_1 = 7$, $P_2 = 9$, $P_3 = 8$, $P_4 = 11$ somma = 35 media tempi di completamento = 35/4 = 8,75

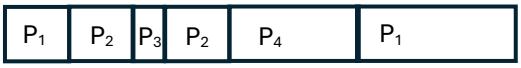
SJF



Tempi di attesa: $P_1 = 0$, $P_2 = 6$, $P_3 = 3$, $P_4 = 7$ somma = 14 media tempi di attesa = 14/4 = 3,5

Tempi di completamento: $P_1 = 7$, $P_2 = 10$, $P_3 = 4$, $P_4 = 11$ somma = 32 media tempi di completamento = 32/4 = 8

SRTN



Tempi di attesa: $P_1 = 9$, $P_2 = 1$, $P_3 = 0$, $P_4 = 2$ somma = 12 media tempi di attesa = 12/4 = 3

Tempi di completamento: $P_1 = 16$, $P_2 = 5$, $P_3 = 1$, $P_4 = 6$ somma = 28 media tempi di completamento = 28/4 = 7

Confrontando gli algoritmi proposti, quello che riduce il tempo di attesa e di conseguenza il tempo di completamento è lo short remaining time next. È da considerare anche il prezzo dei context switch pagati in overhead in più, infatti, si hanno 2 context switch in più.

Scheduling nei sistemi interattivi

Gli algoritmi di scheduling proposti per i sistemi interattivi sono i seguenti:

- **Scheduling Round-Robin**(RR) -> versione con prelazione del FCFS, quindi si rispetta una coda FIFO dove un processo può essere interrotto se ha esaurito il tempo di esecuzione messo a disposizione dello scheduler. Si stipula un **quanto di tempo**(**timeslice**(solitamente 20-50 ms)) dopodiché il processo dallo stato in esecuzione viene riportato alla coda dei processi pronti. Un quanto di tempo piccolo aumenta il numero di prelazioni che implicano context switch e maggiore overhead, invece, un quanto di tempo maggiore avvicina l'algoritmo al FCFS e alla sua inefficienza. Si ricorda che il quanto di tempo non si divide tra i processi, ma è il tempo di esecuzione assegnato ad ogni processo.
- **Scheduling a priorità** -> si etichettano i processi e a seconda dell'etichetta si offrono dei vantaggi. L'etichettatura può variare a seconda della natura del processo, infatti può essere svolta: se ci sono condizioni esterne(es. privileggio dato a chi paga), favorendo una tipologia di processo(I/O bounded) oppure come nel SJF a seconda del tempo di esecuzione. Si favoriscono quindi processi che hanno priorità più alta nella coda dei processi e poi man mano quelli a priorità più bassa. Le priorità possono essere: statiche, quindi, una volta assegnate rimangono le stesse durante tutta la vita del processo, oppure dinamiche che cambiano durante l'evoluzione del processo, aumentando se il processo ha un tempo di attesa alto o diminuendo se esso ha consumato molta CPU. Una valida alternativa è lo scheduling a code multiple(classi di priorità)-> dove si hanno al posto di una sola coda dei processi, più code, ognuna con un tipo di priorità diversa(dalla più alta alla più bassa). Quando si hanno più code, le priorità possono essere:
  priorità fisse-> le code non si influenzano tra loro e a cascata verranno eseguiti i processi. I processi a bassa priorità possono soffrire starvation;
  con feedback(retroazione)-> processi suddivisi in code, ma possono cambiarsi a seconda del loro comportamento, es. processo usa troppa CPU -> si abbassa la priorità.
  L'algoritmo in questione può prevedere la prelazione, infatti, se previsto il nostro sistema operativo diventerà più reattivo, ma ciò può aumentare l'overhead. La prelazione può essere gestita con un round robin per le varie code, assegnando un quanto di tempo diverso per le code. Nota quindi anche la gestione delle code di priorità possiamo fare una distinzione, infatti abbiamo: lo scheduler verticale ovvero l'algoritmo modellato per la scelta dell'ordine delle code da eseguire e lo scheduler orizzontale ossia l'algoritmo utilizzato per la scelta dei processi da eseguire all'interno della singola coda. Si possono utilizzare anche algoritmi di scheduling orizzontali diversi.
  Inoltre, utilizzando un sistema a priorità si rischia di ottenere la starvation per processi a bassa priorità, questo problema si può risolvere con un algoritmo di aging che, all'aumentare del tempo di attesa aumenta anche la priorità di processo. Se non si vuole utilizzare l'aging che implica complessità all'algoritmo, si può programmare lo scheduler verticale per un'esecuzione più equa nonostante la priorità, infatti potremmo imporre che si preferisca l'elaborazione dei processi di una coda se questa è stata soggetta di starvation.
- **Short process next**(SPN) -> l'idea è di esaminare il CPU-burst dei processi in coda e garantire l'uso privilegiato ai processi I/O bounded. Persegue il principio di favorire processi brevi(SJF ai processi interattivi), ma come facciamo a sapere a priori il burst di un processo?
  La previsione viene fatta sulla base dei comportamenti assunti in passato dai processi stessi(analisi burst precedenti). Questo è possibile perché raramente ci sono cambiamenti repenti che cambiano il burst dei processi.
  Dobbiamo quindi effettuare una stima e questa stima(parametrica) va fatta attraverso una media pesata, i cui membri analizzati sono stime precedenti e tempi effettivamente impiegati.

  STIMA PROSSIMA ESECUZION $S_{n+1} = S_n(1-a) + T_n*a$

  a è un parametro che serve a rendere più o meno elastica la stima(più alta è e più è facilmente influenzabile da rumori, più bassa è più la stima tende ad essere costante)
  sn è la stima precedente
  tn è il tempo effettivamente impiegato

- **Scheduling garantito** -> si fa una promessa ad ogni processo, ovvero che se ci sono n processi ad ognuno di loro verrà garantito 1/n di uso della CPU. Questa percentuale di utilizzo poi viene fatta rispettare. Si deve tenere traccia della percentuale di CPU già utilizzata dai processi e confrontandole si determina il prossimo processo. La scelta avviene in modo equo, garantendo a tutti i processi un trattamento al pari degli altri, ma il problema principale sta nel fatto che è difficile tenere traccia del tempo impiegato dai processi.
- **Scheduling a lotteria** -> ipoteticamente ad ogni processo viene assegnato un biglietto, con cui potrà partecipare all'estrazione randomica che effettuerà lo scheduler nello scegliere il prossimo processo da eseguire. Maggiore è il numero di biglietti che si hanno in possesso, maggiori sono le probabilità di essere scelti, quindi si possono modellare le priorità. È semplice nel realizzarlo e si garantisce equità non deterministica ma probabilistica.
- **Scheduling fair-share**(giusta-condivisione) -> nei modelli precedenti si è pensato a schedulare i processi in modo equo, indipendentemente dall'appartenenza all'utente. Se ci fossero più utenti sarebbe giusto ripartire equamente la CPU tra gli utenti. È per questo che lo scheduling fair-share raggruppa i processi e thread per utenti e divide la CPU equamente. Di seguito è mostrato un esempio:
  utente1 processi: ABCD
  utente2 processi: E
  un modo per coordinare le esecuzioni sarebbe dividere a metà la CPU, allora si avrebbe il seguente flusso di esecuzione AEBECEDE.

Lo scheduling nei thread varia a seconda della natura del thread stesso ossia utente o kernel.

Nei thread utente si possono utilizzare tutti gli algoritmi di scheduling non pre-emptive sopraccitati. I thread utente non possono essere preemptati dal kernel, quindi non vengono sospesi forzatamente, ma devono essere essi stessi a rilasciare la CPU attraverso la chiamata thread.yield(). Lo scheduler dei thread utente non fa parte del kernel, ma è implementato tramite librerie; i programmatori possono scegliere l'algoritmo di scheduling.

Nei thread kernel si possono utilizzare tutti gli algoritmi presentati. Non è possibile alcuna personalizzazione da parte del programmatore, dato che lo scheduler è integrato nel sistema operativo. Il vantaggio nei thread, piuttosto che nei processi, è che si ha meno overhead, nel caso che due thread sono fratelli. Quando non sono fratelli si deve riprogrammare l'MMU e in alcuni casi svuotare cache e registri della CPU.

Concetti come il time slice, l'aging e altre tecniche di scheduling tipiche dei processi possono essere applicati solo ai thread kernel, perché solo questi supportano la preemption.

Scheduling su sistemi multiprocessore

Possiamo seguire due alternative per lo scheduling nei sistemi multiprocessore:

- **Multielaborazione asimmetrica** -> un processore assume il ruolo del **master server** ossia si incarica di effettuare lo scheduling dei processi, facendoli poi elaborare a tutto il resto dei processori slave. Questo permette maggiore coordinazione tra i processi, specie quando si agisce in memoria condivisa, ma un'eccessiva quanti di task da schedulare potrebbe congestionare lo slave e rallentare l'elaborazione. Il master server diventa un collo di bottiglia per i processi da eseguire.
- **Multielaborazione simmetrica (SMP)** -> tutti i processori sono gli stessi e hanno lo stesso ruolo. In autonomia schedulano i processi e li eseguono. Si ha una coda unificata dei processi pronti e memoria, e si deve utilizzare una variabile di lock per la lettura e scrittura unica. Si possono anche avere delle code separate, per n core si hanno n code. Avere una variabile di lock sia per le code globali e sia per le code dei singoli core è importante per non incorrere in race condition
  Un processo può avere una **predilezione** (affinità) per un core si ha un vantaggio per l'accesso in memoria. Infatti, nella cache della CPU dove si è eseguito recentemente il processo interessato, molto probabilmente saranno presenti i dati generando un cache hit.

È anche importante avere un bilanciamento del carico tra le varie unità elaborative, dato che non si possono avere solo poche CPU che gestiscono un carico eccessivo mentre le altre sono sotto-utilizzate.
Per evitare di avere uno sbilanciamento del carico, si può avere una predilezione minore, e lasciare che più CPU elaborino un processo in tempi diversi, senza perdere il vantaggio ottenuto dalla predilezione stessa.
Attraverso la migrazione dei dati da CPU a CPU, si evita di accedere in memoria centrale e passare i dati da cache a cache. Un core, che ha un carico minore, può scegliere tra la coda dei processi pronti di un altro core un processo pronto e i dati necessari. Questa tecnica è una forma di **stealing**, che i sistemi operativi permettono.
La **migrazione** può essere **guidata** dal sistema operativo oppure **spontanea**, le due tipologie non si escludono all'interno di un sistema operativo. La differenza sta appunto nell'intervento del kernel, che implica un tempo di migrazione superiore nel caso in cui se ne occupa il sistema operativo. In assenza della guida da parte del SO si può incorrere comunque in uno sbilanciamento.

Ma quali scheduler si utilizzano nei vari sistemi operativi?

Su windows viene utilizzato uno scheduler basato sulle priorità, pensato per un sistema interattivo dove è importante la velocità di risposta. Sono inoltre presenti euristiche che evitano l'inversione di proprietà.

Su MACOS lo scheduler è basato sul **Mach scheduler**, che applica algoritmi pre-emptive, con l'applicazione di code di priorità con euristiche.

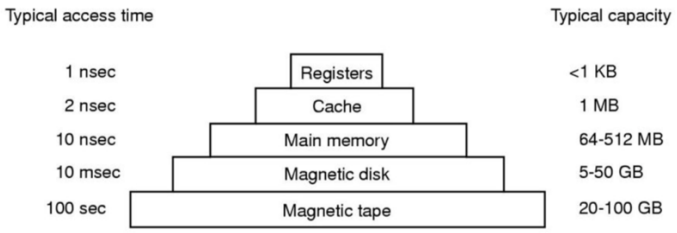
Infine, su Linux lo scheduler è pensato per i task, e non thread e processi che sono casi particolari dei task stessi. I task vengono raggruppati per utente e si utilizza uno scheduler Completely Fair Scheduler, che è una versione moderna dello scheduler garantito.
Le chiamate **thread.create()** e **process.fork()** sono chiamate particolari della funzione **task.clone()**.

CAPITOLO 3 GESTIONE DELLA MEMORIA

La legge di Parkinson, legge formulata ironicamente nel campo economico, può essere riadattata in ambito informatico per capire quanto è importante la memoria nei nostri computer: “I programmi si espandono in modo da riempire tutta la memoria che hanno a disposizione per contenerli”. Nonostante la natura ironica, evidenzia un concetto importante, infatti: più grande è la memoria a disposizione degli elaboratori e molto più ricchi e complessi possono diventare i nostri programmi.

Ogni programmatore desidera avere una memoria privata, infinitamente grande, molto veloce, non volatile e a basso costo. Con le tecnologie moderne questo non è possibile, ed è per questo motivo che si ricorre al concetto **di gerarchia di memoria**, ossia la possibilità di avere: pochi megabyte di memoria veloce ma volatile(cache), qualche gigabyte di memoria abbastanza veloce, volatile, e a basso costo(RAM) e qualche terabyte di memoria non volatile, lenta e a bassissimo costo(dischi).

La gerarchia di memoria rappresenta un compromesso necessario, capace di bilanciare velocità, capacità e costo e costituisce la base del funzionamento degli elaboratori moderni.



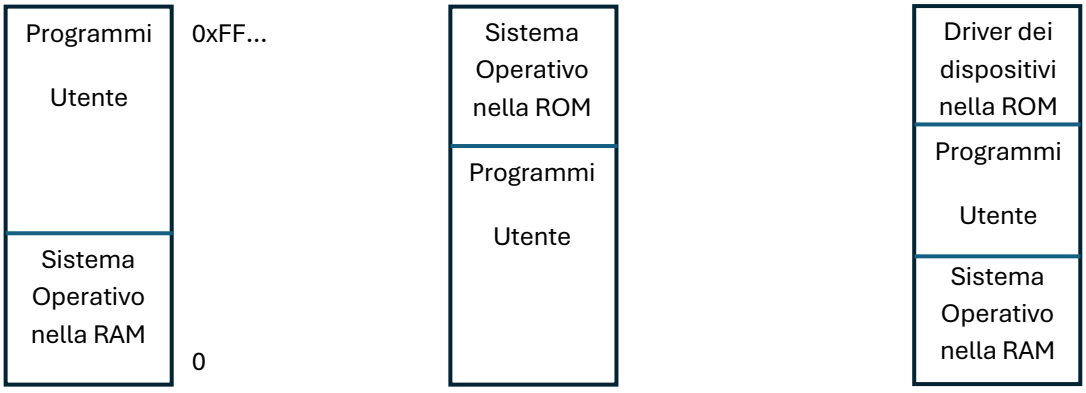
La RAM o memoria centrale è il fulcro della gerarchia di memoria, dato che è il livello più basso accessibile dal processore e di fatto tutti i dati la devono attraversare. L’astrazione ci permette questa gerarchia come una memoria unificata, con tempi di accesso diversi, a seconda del dato che si sta cercando. Un esempio fondamentale è il funzionamento della cache(memoria nascosta) che attraverso i cache hit, ottenuti attraverso vari algoritmi, fa apparire molto veloce tutto ciò che è al di sotto dei registri del processore.

La parte del sistema operativo che gestisce(in parte) la gerarchia della memoria è chiamata **gestore della memoria**.

La più semplice astrazione della memoria è l’**assenza di astrazione**. I primi mainframe, minicomputer e personal computers non avevano un’astrazione della memoria, quindi ogni programma vedeva direttamente gli indirizzi fisici.

Era difficile eseguire due programmi contemporaneamente, infatti se un programma scriveva in una determinata posizione, sovrascriveva ciò che prima aveva scritto un altro programma. Da qui si evince un problema legato alla sicurezza dei programmi, dato che la sovrascrittura della memoria da parte di un programma, poteva corrompere i programmi restanti.

Inoltre, il problema nasce sulla separazione da fare tra lo spazio da riservare ai programmi utenti e al sistema operativo. Si hanno diversi modelli, di seguito sono rappresentati:



Il primo e il terzo modello hanno lo svantaggio che il sistema operativo condivide la memoria con i programmi utente, che può rendere il sistema operativo corruttibile dato che un programma utente lo può cancellare. La differenza tra i due modelli sta nei driver dei dispositivi, infatti, visto che nel terzo modello risiedono in ROM, l'avvio del SO è garantito.

Nel secondo modello si ha più robustezza del sistema operativo, ma meno flessibilità, dato che la ROM è una memoria di sola lettura.

L'unico pseudo-parallelismo che si può ottenere è quello dato dall'utilizzo dei thread, noto che può essere eseguito un solo programma alla volta. Per effettuarlo tra processi era possibile soltanto attraverso context switch continui.

Ma con il tempo si trovò una soluzione hardware che divideva la memoria in blocchi, ognuno con una chiave di accesso, contenuta nel PSW(Program status word). Nonostante ciò, esistevano problemi in fase di load del programma, infatti, non potevano essere caricati in posizioni arbitrarie, ma solo negli indirizzi per cui erano stati compilati. Per affrontare questo problema si utilizzò la **rilocazione statica**, che prevedeva di applicare un offset ai programmi in fase di loading, rendendo possibile l'allocazione in qualunque area della memoria libera.

I passaggi della rilocazione statica sono i seguenti:

- Scansione degli indirizzi;
- Offset applicato agli indirizzi(modifica nel codice);
- Loading del programma.

Queste intuizioni furono notevoli, ma il caricamento di più programmi in memoria contemporaneamente era impossibile, considerando anche che il loader è rallentato dalla rilocazione statica.

La soluzione fu astrarre la memoria, come già avviene per le CPU, facendo percepire a ciascun processo di avere a disposizione l'intero spazio di indirizzamento in modo privato. Da qui nacque il concetto di spazio degli indirizzi, cioè l'insieme di tutti gli indirizzi che un processo può utilizzare per accedere alla memoria.

La dimensione dello spazio degli indirizzi dipende dal numero di bit utilizzati per rappresentare gli indirizzi. Le architetture moderne sono tipicamente a 32 bit o a 64 bit:

- con **32 bit** si ottengono 2^{32} indirizzi distinti (circa 4 GB di spazio di indirizzamento),
- con **64 bit** si ottengono 2^{64} indirizzi distinti (uno spazio teorico enorme, nell'ordine degli exabyte).

Questo significa che i processi dispongono di una quantità di memoria astratta molto più ampia.

Questa soluzione si basa su una versione particolarmente semplice della rilocazione dinamica: ovvero che si mappa lo spazio degli indirizzi di ogni processo su una parte diversa della memoria fisica in modo semplice. Si utilizzano due registri speciali ovvero **registro base** e **registro limite**.

Il registro base contiene l'indirizzo fisico dove comincia il programma in memoria e nel registro limite viene caricata la lunghezza del programma. L'indirizzo logico scritto nel programma poi viene tradotto in indirizzo fisico per l'hardware e questo potrebbe implicare somme ed eventuali confronti che causano delle latenze nell'esecuzione.

L'**MMU** contiene il registro base e limite e si occupa della traduzione degli indirizzi logici in fisici. Per ogni context switch si deve riprogrammare l'MMU, operazione che causa overhead in più.

Inoltre, nel caso che un indirizzo logico tradotto, non rientra nel range [base, base + limite] allora verrà effettuata una trap per **addressing error**.

SWAPPING

Se la memoria fisica del computer è abbastanza grande da contenere tutti i processi, allora tutti gli schemi descritti funzioneranno. Nella realtà non è così, poiché la quantità di RAM richiesta è molto più grande da quella che fisicamente esite.

Un esempio potrebbe essere il programma photoshop, che per essere avviato ha bisogno di circa 1 gb e svariati giga in esecuzione. Tenere tutti i programmi in memoria potrebbe richiedere una quantità di memoria elevata e la memoria potrebbe non essere sufficiente.

La soluzione a questo problema è sospendere e parcheggiare sul disco rigido il processo(non eseguibile) perché non pronto. Viene fatta un'ibernazione in virtù di esigenze -> **processo ibernato**.

Questa tecnica prende il nome di **swapping dei processi**, e prevede che processi non pronti, di backup o routine o che non hanno diretta interazione con l'utente, vengano temporaneamente sospesi e registrati sui dischi, in modo tale che la RAM sia libera per altri processi in esecuzione.

Viene utilizzato uno **scheduler di medio termine**, diverso dallo scheduler normale dei processi, perché la frequenza con cui si applica è minore, poiché le operazioni di trasferimento sul disco sono molto più costose in termini di tempo rispetto ad un semplice cambio di contesto in RAM.

Swap in e swap out potrebbero causare problemi con **I/O pendenti**, infatti, dato che le operazioni non vengono gestite solo dalla CPU ma anche dal **DMA**(direct memory access), che accedere direttamente alla memoria fisica.

Un esempio classico è il seguente: supponendo di avere due processi P1 e P2. P1 sta effettuando operazioni di scrittura in memoria, mentre che il sistema operativo decide di effettuare lo swap con P2. Allora, i dati che sta scrivendo P1 potrebbero poter esser corrotti dalla scrittura di P2.

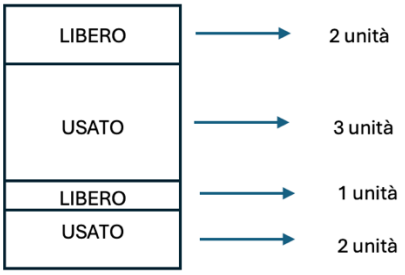
Inoltre, un ulteriore problema potrebbe essere la dimensione pensata per il programma. Se si utilizza una dimensione di memoria fissa per ogni programma, si può incorrere nei seguenti problemi:

- programmi più grandi della partizione -> la memoria non può crescere dinamicamente e può essere insufficiente;
- programmi più piccoli della partizione -> durante la fase di swapping si vengono salvate e ricaricate anche aree di memoria non utilizzate.

In questo tipo di allocazione la memoria utilizzata e contigua, ossia che ogni processo deve occupare un blocco unico di memoria, se da un lato semplifica la gestione, dall'altro amplifica alcuni problemi. Questi problemi di consumo futile di memoria sono chiamati **frammentazione**.

La frammentazione può essere di due modi:

- frammentazione interna -> spazio sprecato dentro un blocco di memoria allocato;
- frammentazione esterna -> spazio disponibile per un processo, ma non utilizzato perché non contiguo.

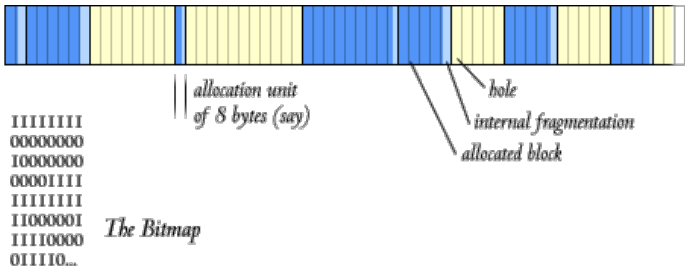


Supponendo di avere un processo che abbia bisogno di tre unità di memoria per essere allocato, con l'allocazione contigua non potrebbe essere allocato, nonostante che la memoria richiesta sia effettivamente disponibile.

Una soluzione alla frammentazione esterna potrebbe essere la **memory compaction**, ossia il compattare la memoria libera, muovendo in basso tutti i processi ed usare lo spazio libero rimasto per allocarne di nuovi. Sebbene inizialmente sembri una tecnica efficace, non è così, dato che per effettuare questo spostamento si impiegherebbero molti secondi.

Quando la memoria è assegnata dinamicamente, il sistema operativo deve gestirla. La gestione della memoria avviene attraverso la **bitmap**, che rappresenta un intero blocco e i singoli bit rappresentano un'unità di allocazione. La dimensione dell'allocazione di memoria è un problema progettuale, dato che avere un'unita di allocazione molto grande potrebbe equivalere ad avere spreco all'interno dell'unità e un'unità più piccola potrebbe rendere enorme la bitmap.

La bitmap contiene informazioni sulle unità di allocazione all'interno di un blocco, indicando con 0 il blocco libero e 1 il blocco occupato.

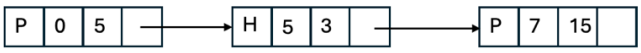


L'obiettivo di quando si alloca la memoria è cercare delle celle di memoria contigue libere.

La ricerca di una serie di 0 di una certa lunghezza è un'operazione lenta, elemento a sfavore della bitmap.

Per gestire meglio l'allocazione contigua in memoria, si potrebbe usare la **lista doppiamente concatenata**.

L'impiego di una lista doppiamente concatenata per indicare se la memoria è occupata dai dati di un processo o se è spazio tra due processi, potrebbe essere più facile da consultare. La lista è ordinata per indirizzo da 0 a MAX-1. Quando si trova la memoria libera sostituiamo il buco(vuoto) con i dati del processo da allocare, e l'eventuale buco che si crea è causato dalla frammentazione interna. Quando rimuoviamo la memoria di un processo, avremmo due buchi contigui, e se sono ordinati per indirizzi e non per dimensione si facilita la **coalescenza**, ovvero l'unione dei blocchi adiacenti per ridurre la frammentazione esterna.



Per la ricerca di blocchi liberi per l'allocazione abbiamo svariati esempi e a seconda dell'algoritmo si hanno benefici e problemi diversi. Di seguito, sono citati alcuni algoritmi per la ricerca del blocco.

1) **First fit** -> Il primo blocco libero che permette di allocare n byte -> È il più semplice da attuare, ma non è equo(parte sempre dal primo blocco), dato che blocchi liberi in fondo potrebbero non essere allocati, inoltre, la selezione non si cura di cercare un blocco adatto, quindi si potrebbe avere frammentazione interna.
2) **Next fit** -> Il prossimo blocco libero che permette di allocare n byte -> A differenza del primo algoritmo la ricerca riprende dall'ultimo blocco dove si è fermato nell'ultima allocazione -> maggiore equità, anche se non vengono risolti i problemi legati alla frammentazione.
3) **Best fit** -> Il blocco più adatto che permette di allocare n byte -> È più lento a cercare il blocco, dato che deve cercare in tutta la lista, infatti, ha complessità O(n). Inoltre, occupando sempre i blocchi migliori, si creeranno buchi piccoli, inutilizzati.

4) **Worst fit** -> Il blocco più grande che permette di allocare n byte -> Cerca il blocco vuoto con capacità maggiore ,in modo che la differenza tra il blocco trovato e l'allocazione effettiva sarà maggiore, in modo che il buco poi potrà essere riutilizzato da altri processi.

Si possono ordinare i blocchi anche in base alla dimensione. Ordinandoli così dal blocco più piccolo al più grande, si può trovare più velocemente il blocco appropriato. Applicare il best fit risulta meno complesso, ma, nonostante ciò, si incorre nella creazione di buchi piccoli non allocabili.

Quindi, dobbiamo abbandonare l'idea dell'allocazione contigua per passare ad un'**allocazione non contigua**, permessa attraverso la virtualizzazione della memoria. Attraverso la memoria virtuale possiamo quindi oltrepassare i vincoli dell'allocazione contigua e questo è reso possibile dall'astrazione.

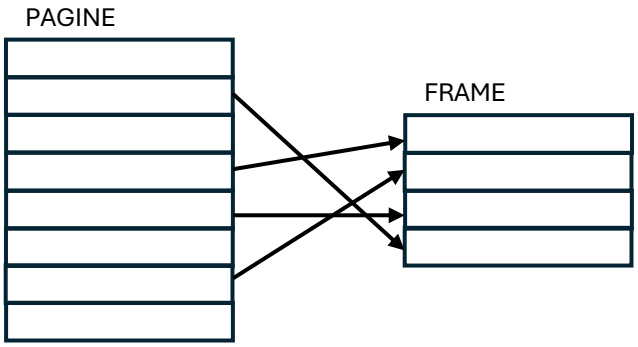
Lo **spazio di indirizzamento diventa virtuale** e può superare di gran lunga la memoria fisica. Lo spazio di indirizzamento è determinato dai bit riservati agli indirizzi.

I computer odierni sono a 64 bit, quindi, genereranno uno spazio di indirizzamento di circa 18,4 exabyte(troppo praticamente). Ma come si implementa tutta questa memoria se non c'è realmente?

Nel modello di memoria virtuale, lo spazio di indirizzamento virtuale, viene suddiviso in blocchi di pochi KB, chiamati **pagine**.

Ogni pagina è un intervallo di indirizzi logici contigui, e per essere utilizzata essa deve essere mappata in memoria fisica. Per eseguire un programma non è necessario che tutte le pagine siano contemporaneamente nella memoria fisica.

Ma non solo lo spazio di indirizzamento virtuale viene suddiviso in blocchi, anche la memoria fisica viene divisa in frame, dove la dimensione delle pagine corrisponde a quella dei frame.



La dimensione delle pagine è fissa solitamente, e in una pagina si scrivono dati di un solo processo. In alcuni sistemi la dimensione può essere variabile, risolvendo così la frammentazione interna ma complicando la gestione delle pagine.

La suddivisione del processo in più pagine contigue è una tecnica chiamata **paginazione** o **paging**. Quest'ultima tecnica risolve definitivamente la frammentazione esterna, dato che queste pagine potranno essere riscritte in memoria anche in modo non contiguo.

Per colmare il grande divario tra spazio di indirizzamento virtuale e memoria fisica, il sistema operativo utilizza anche la memoria secondaria (tipicamente il disco) per allocare le parti dei processi non presenti in RAM.

Il processo, quando è in esecuzione, non è consapevole di quali parti siano effettivamente in memoria. Se accede a una parte non caricata in RAM, si verifica un **page fault** (errore non critico). A quel punto, il sistema operativo carica la pagina richiesta dalla memoria secondaria alla RAM, permettendo al processo di continuare l'esecuzione senza interruzioni.

Le informazioni che il sistema operativo, intuisce che non saranno utili al processo nel breve periodo, le sposta in memoria secondaria, tenendo in RAM solo le parti utili ai processi affinché si evolvano.

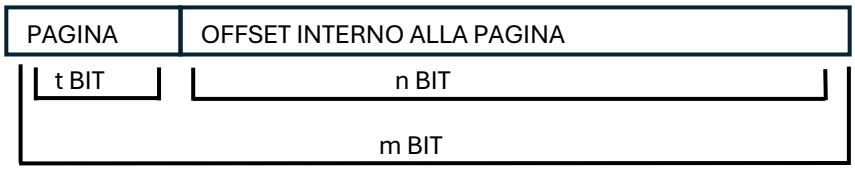
Le intuizioni possono anche essere anche errate, in quel caso si paga con il tempo di accesso alla memoria.

Con un sistema multi-programmato e la paginazione possiamo sfruttare tutte le risorse senza starvation, dato che possiamo mantenere molti più processi in esecuzione, caricando solo le pagine essenziali. C'è una protezione implicita tra processi perché vengono assegnati spazi di indirizzamento virtuali diversi tra loro. P1 non può accedere allo spazio virtuale di P2 e viceversa.

La frammentazione interna è presente ma non è un problema, visto i limiti che si superano, inoltre, in futuro quelle mezze(quarti,ecc.) potrebbero essere usate.
La paginazione è gestita dall'MMU, che si occupa principalmente della traduzione degli indirizzi. La tabella delle pagine è una struttura dati presente nel sistema operativo, ed è accessibile dal MMU. Essa serve a mappare le pagine virtuali sui frame oltre contenere bit utili a descrivere la pagina stessa.

Se la pagina desiderata è presente in memoria centrale l'MMU traduce direttamente l'indirizzo, altrimenti si attiverà un page fault. La traduzione avviene nel seguente modo:

si ha il seguente indirizzo: **1100000010100100(*)** ad m bit e la dimensione della pagina di **n** bit.



Si effettua lo shift a destra di n posizioni e si otterrà numero espresso in **t** bit con **t** < m, ossia **1100**. Il numero trovato sarà il numero della pagina da consultare nella tabella, che conterrà il frame associato. Trovato il frame, si applica uno shift a sinistra di n posizioni, si fa l'or logico con l'offset interno della pagina e finalmente si ottiene l'indirizzo logico(questo è come opera l'MMU). Si ricorda che nella consultazione della tabella delle pagine gli indici partono da 0.

Gli shift sono poco intuitivi per l'uomo, dato che si lavora principalmente con i bit, per questo bisogna ricordare che ad ogni shift corrisponde una moltiplicazione per 2^n(con n numero di posizioni) se a sinistra e una divisione per 2^n se a destra.

Nel nostro caso avevamo l'indirizzo 1100000010100100 che in decimale equivale a 49316. Lo shift effettuato a destra corrisponde a dividere per 2^{12} o 4096, quindi, 49316 / 4096 = 12 r = 164.

Nella pagina 12 della tabella delle pagine, sarà contenuto il frame associato. Supponiamo che sia 6, lo shift a sinistra equivale moltiplicare 6*4096 che è pari a 24576, a cui si somma l'offset che è 164. L'indirizzo fisico sarà 24576+164 ossia 24740.

Prima, si è citato il **page fault** causato dall'assenza di una risorsa in memoria centrale. Questa trap non è altro che un'eccezione simile a eventi fatali, con la differenza che non si blocca permanentemente l'esecuzione.
Il sistema operativo, quindi, prende un frame poco utilizzato e se è sporco(ha ricevuto modifiche) lo parcheggia nell'**area di swap**. Il contenuto desiderato viene cercato in memoria secondaria e verrà scritto nel frame soggetto ad eliminazione, solo quando il vecchio contenuto di quest'ultimo verrà scritto in memoria.

Ora che si è visto, in generale come funziona la connessione tra pagine e frame, possiamo parlare più nel dettaglio della singola voce della pagina presente all'interno della tabella.

L'esatto layout dipende molto dalla macchina, ma le informazioni sono più o meno le stesse per tutto le macchine.

ALLOCAZIONE	CACHING	RIFERIMENTO	DIRTY BIT	RWX	PRESENZA	NUMERO FRAME

Come già detto, il numero di frame si riferisce al frame fisico in cui è caricata la pagina. Gli altri bit presenti nella tabella delle pagine (come bit di presenza, di protezione o il dirty bit) servono a definire il contesto della pagina e a indicare le operazioni da fare nel caso in cui essa debba essere consultata o eventualmente sostituita.

Ogni bit assume quindi un significato, ossia:

- **Bit di presenza** -> indica se la voce è in memoria centrale e può essere utilizzata(1), se non è così(0) deve essere riportata in memoria centrale;
- **Bit RWX(protezione)** -> indica se il frame è di lettura, scrittura ed esecuzione. Nei casi elementari esiste un solo bit dove 1 indica solo lettura e 0 lettura e scrittura.
- **Bit dirty** -> indica se sono state fatte modifiche nella pagina. Se si vuol dire che il frame è stato sporcato e deve essere necessariamente scritto in memoria prima di essere sostituito. In caso diverso può essere direttamente sovrascritto in fase di sostituzione.
- **Bit riferimento** -> viene posto ad 1 quando si fa qualsiasi riferimento alla pagina(lettura o scrittura), ed ogni tot viene azzerato. Questo bit è utile in algoritmi, quali di aging per capire quali pagine sono state utilizzate di recente.
- **Caching disabilitato**-> viene posto ad 1 quando la cache non deve essere utilizzata per accedere a quella pagina. Se inizialmente può sembrare insensata, nel caso in cui si hanno registri delle periferiche è necessario che essi siano raggiunti attraverso gli indirizzi di memoria.

Il sistema operativo tiene traccia di ogni frame fisico attraverso la tabella dei frame, la cui dimensione è corrispondente alla dimensione della memoria RAM. Essa contiene le informazioni riguardanti i frame, ovvero se sono allocati o no e a quale processo corrispondono.
La tabella delle pagine viene consultata:

- **Viene creato un processo e la tabella delle pagine**-> viene consultata e assegna eventuali frame iniziali;
- **Allocazione di nuove pagine per un processo** -> vengono cercati frame liberi;
- **Page fault** -> per individuare frame vittima o liberi.

La traduzione dell'MMU avviene per ogni indirizzo e ci sono 3 accessi alla memoria: 2 per ogni traduzione(accesso alla tabella delle pagine e alla tabella dei frame) e una per prendere il dato in memoria.

La tabella delle pagine inizialmente era presente nell'MMU stesso, in un set di registri dedicati. Il sistema operativo manipolava i registri dell'MMU soprattutto durante i context switch. Il problema principale è la non scalabilità con l'uso dei registri.

Oggi, la tabella è allocata in RAM con opportuno registro integrato, ovvero **PAGE-TABLE BASE REGISTER(PTBR)**, ovvero il registro che punta al primo record della tabella delle pagine del processo corrente.

Si perde l'efficienza perché l'MMU per tradurre gli indirizzi deve fare il primo accesso alla memoria RAM(per la tabella delle pagine) e poi tutto il resto per raggiungere la risorsa.

Nella progettazione di una tabella delle pagine, dato che risiede in **RAM**, bisogna prestare attenzione a:

- **Velocità nella consultazione** → quando preleviamo un'istruzione dalla memoria, essa ha solitamente uno o due operandi. Se questi operandi possono essere tradotti rapidamente tramite la tabella delle pagine e la memoria centrale, si risparmia molto tempo nella ricerca degli operandi.

- **Dimensione** → nonostante i nostri computer siano a 64 bit, si utilizzano al massimo 48 bit per gli indirizzi, che già generano trilioni di posizioni di memoria accessibili. È importante che la tabella delle pagine sia ridotta, rispetto al generabile, dato che deve essere conservata in memoria.

La maggior parte dei programmi tendono a fare riferimenti ad un piccolo numero di pagine e non il contrario. Per aumentare le prestazioni e quindi ridurre l'accesso alla memoria, i progettisti hanno creato la **TLB**(translation lookaside buffer), spesso chiamata anche memoria associativa.

La TLB è un piccolo dispositivo hardware per mappare indirizzi virtuali su indirizzi fisici, senza passare dalla tabella delle pagine. È una memoria cache che sta tra la RAM e l'MMU(sta al suo interno) contenente la copia di alcune pagine del processo in corso. Si diminuisce l'accesso in memoria richiesto per la traduzione degli indirizzi.

Le poche voci della TLB contengono:

- Il numero della pagina virtuale;
- Bit di presenza;
- Codice di protezione(RWX);
- Dirty bit;
- Numero di frame.

La ricerca della pagina non è lineare quindi con complessità O(n) ma bensì attraverso particolari algoritmi è costante O(1). Se il contenuto è stato precedentemente copiato nella TLB allora faremo una **TLB-HIT** (non accediamo in memoria -> meno overhead), altrimenti una **TLB-MISS**(si accede allora alla RAM, si cerca il frame e poi si preleva la risorsa).

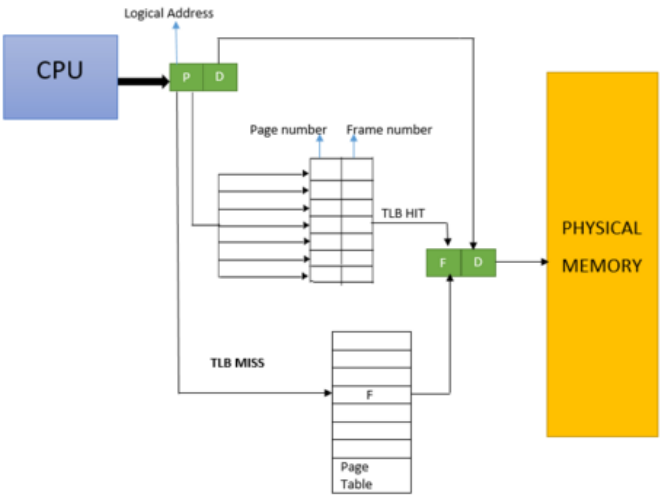
La TLB-MISS, come nel caso di un page fault, genera un errore gestito dal Sistema Operativo, che si occupa di cercare la risorsa nella tabella delle pagine, in memoria centrale e infine nelle memorie secondarie.

Le TLB-MISS sono molto più frequenti dei page-fault perché le voci all'interno della TLB possono variare da 64 a 1024 e invece le pagine nella tabella delle pagine sono migliaia. Quindi è importante la gestione in caso di TLB-MISS.

Quando c'è context switch dobbiamo azzerare la TLB, operazione che purtroppo fa perdere l'efficienza ottenuta con la TLB. La causa è legata al sistema di etichettatura delle pagine, che potrebbe assegnare a due processi diversi la stessa pagina, anche se il riferimento agli indirizzi fisici è differente.

Si può evitare questo svuotamento attraverso l'assegnazione univoca di pagine, resa possibile grazie all'**Addres-space identifiers(ASID)**. Oltre al numero di pagina, il frame e altre informazioni legate alla voce all'interno della TLB, il sistema operativo genera anche un identificatore, per ogni processo(tipo il pid), attraverso il quale ogni processo accede esclusivamente alla propria regione di TLB assegnate.

Nonostante il numero di voci della TLB per ogni processo siano ridotte, si ha un guadagno in termini di minor overhead, perché non si devono svuotare le voci che si riferiscono al processo precedente.



La TLB cura l'aspetto prestazionale in termini di velocità, resta risolvere il problema delle dimensioni della tabella delle pagine.

Nel caso di sistemi a 32 bit(es. pentium), si hanno 2^{32} indirizzi e sistemandoli in pagine avremmo 1 milione di pagine da 4kb. Avere una tabella di pagine monolitica e allocata contiguamente potrebbe richiedere molta memoria per la sua allocazione.

Per aggirare il problema si sono pensate delle tecniche di allocazione diverse, che vanno a snellire il peso occupato dalla tabella delle pagine.

Una tra queste tecniche è avere delle **tabelle multilivello** che permettono di dividere le tabelle delle pagine in livelli gerarchici.

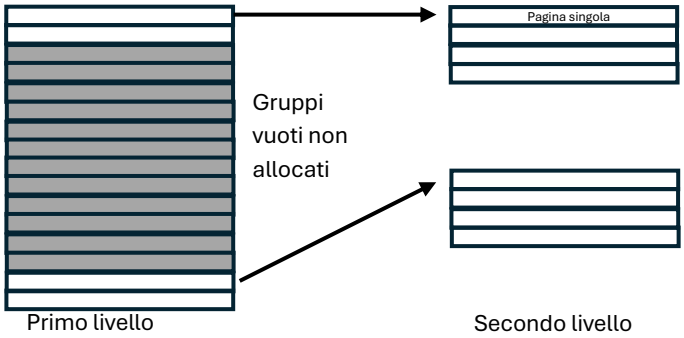
Il segreto delle tabelle multilivello sta nel non mantenere tutte le pagine in memoria, ma solo una parte di esse.

Avendo ad esempio un sistema a 32 bit si può spezzare l'indirizzo in 3 parti:

- **10 bit per le Page Directory Index** -> individua univocamente una tabella delle pagine;
- **10 bit per le Table Page** -> individua le entry all'interno della tabella;
- **12 bit per l'offset** -> posizione dentro la pagina;

Quindi si hanno 2 livelli: il primo livello composto da una tabella che individua tutte le altre tabelle(limitato al caso 1024 voci) e il secondo livello dalle tabelle delle pagine effettive(1024 voci) contenenti le pagine poi associate ai frame.

In questo modo lo spazio degli indirizzi è coperto completamente, ma vengono allocate soltanto le pagine interessate.



Possono esistere anche più di due livelli e la suddivisione dei bit per gli indirizzi è arbitraria. Aumentando il numero di livelli, si ha un guadagno in termini di memoria, ma una perdita in termini prestazionali, dato che la traduzione sarebbe più complessa da gestire.

Se non utilizzate, non si allocano le pagine nel secondo livello. Così si evita un eccessivo spreco di memoria.

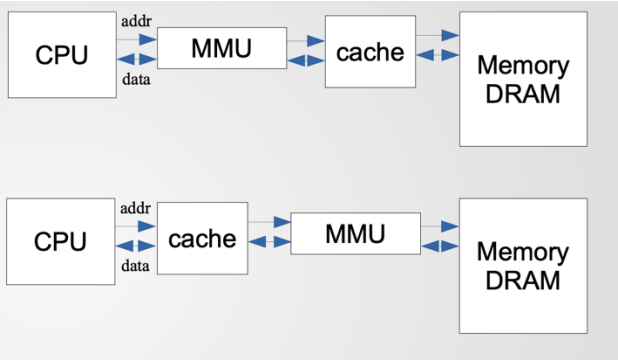
Un'alternativa alle tabelle multilivello, sono **le tabelle delle pagine invertite**. Esse prevedono l'uso della tabella dei frame arricchita con delle informazioni riguardanti il numero di pagina virtuale e il pid del processo. Quindi si sostituiscono le n tabelle dei processi con una sola tabella(quella dei frame).

La ricerca è molto complessa, dato che si deve cercare la coppia numero di pagina virtuale e processo associato. Un'intuizione è stata l'utilizzo di una funzione hash, che velocizza il processo di ricerca. Inoltre, si utilizza una TLB che conserva le pagine che sono state referenziate di più.

Le tabelle delle pagine però non vengono sostituite del tutto, dato che a causa dei page fault devono essere consultate. Esse, invece di risiedere in RAM, vengono mantenute sul disco. Durante l'esecuzione, il sistema utilizza la tabella dei frame (tabella delle pagine invertita) per le traduzioni. In caso di page fault, il sistema operativo consulta le tabelle delle pagine sul disco per recuperare le informazioni necessarie e caricare la pagina richiesta in memoria, aggiornando di conseguenza la tabella dei frame.

Ma dati gli indirizzi virtuali, la memoria cache che collaborava con la RAM, lavorerà con indirizzi fisici o logici?

Dipende dove posizioniamo le memorie cache, esse lavoreranno con gli indirizzi logici altrimenti fisici.



Nel primo caso, l'MMU è posto prima della cache. Come ben sappiamo, l'MMU è lo strumento che si occupa della traduzione degli indirizzi, quindi, se la cache è posta dopo lavorerà con indirizzi fisici.

In questo caso non si avrà invalidazione della cache ad ogni context switch, cioè non dovrà essere svuotata, perché anche se due indirizzi virtuali di due processi sono uguali, essi punteranno ad indirizzi fisici diversi.

Lo svantaggio principale è il catching che non è efficace, dato che il sistema operativo può mappare la memoria in posizioni sparse e il principio di località può venire meno.

Nel caso in cui si pone la cache prima dell'MMU, il catching è più efficace, dato che sfrutta il principio di località sugli indirizzi logici. Lo svantaggio principale è che durante ogni context switch la memoria dovrebbe essere invalidata. Questo si può risolvere attraverso gli ASID, che associano gli indirizzi al processo.

In pratica si utilizza la cache L1 prima dell'MMU, permettendola di lavorare con indirizzi virtuali e la cache L2 verrà posizionata dopo, lavorando con indirizzi fisici.

In caso di page fault ed in assenza di frame liberi è necessario scegliere una **pagina vittima**. La sostituzione della pagina da rimuovere nella memoria e l'inserimento della nuova pagina avviene come nella gestione della cache.

Se l'algoritmo di sostituzione delle pagine è efficiente allora il disservizio nei confronti della pagina vittima, non causerà ulteriori page fault. L'obiettivo principale è quindi minimizzare il numero di page fault in futuro.

L'**algoritmo ottimale OPT**(optimal page replacement) è una soluzione teorica addotta per effettuare dei confronti con gli algoritmi realmente usati. Esso tenterà di rimuovere solo le pagine che verranno referenziate in un futuro più lontano. È irrealizzabile perché non abbiamo la possibilità di sapere quando verrà fatto il riferimento ad una pagina.

Per gli algoritmi che poi si implementano nei sistemi operativi, ci serve sapere delle informazioni, contenute già nelle voci delle pagine. In particolare, sapere se una pagina è stata referenziata o modificata è utile nella scelta. Il bit di referenziazione e il dirty bit sono largamente usati negli algoritmi, in particolare un algoritmo li usa entrambi.

L'**algoritmo di NRU**(Not Recently Used) è un algoritmo molto semplice, che attraverso l'uso di questi bit stabilisce delle classi delle pagine. Le quattro classi sono le seguenti:

- **Classe 0**: **non referenziata**, **non modificata**;
- **Classe 1**: **non referenziata**, **modificata**;
- **Classe 2**: **referenziata**, **non modificata**;
- **Classe 3**: **referenziata**, **modificata**;

Viene scelta una pagina appartenente alla classe più bassa disponibile. All'interno della classe stessa, per scegliere la pagina vittima possiamo utilizzare un principio fisso, ad esempio la più vecchia.

Si ha una predilezione maggiore a mantenere pagine modificate in RAM, perché la sostituzione comporterebbe una scrittura in memoria. Si preferisce mantenere in RAM le pagine modificate finché possibile, così da ridurre le scritture su disco e ottimizzare le prestazioni di accesso in memoria.

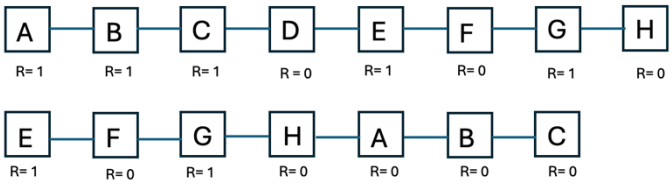
I pro di questo algoritmo sono dati dalla semplicità di implementazione e dal prolungare l'eventuale scrittura in memoria, invece, una dei problemi è che non distingue pagine poco usate e pagine molto usate.

Un altro algoritmo molto semplice è l'**algoritmo FIFO**(First-In First-out) che rimuove la pagina più vecchia, e non tiene conto della natura della pagina. Non per forza la pagina più vecchia è quella meno utilizzata, un esempio è la tabella dei simboli del compilatore, che è una delle prima pagine ma è quella più usata.

Una variante dell'algoritmo FIFO è l'**algoritmo della seconda chance**, dove oltre a seguire lo stesso principio sopraccitato si consulta anche il bit R(referenziazione).

Quando si deve scegliere la pagina vittima si procede così:

- Se la pagina presa in considerazione ha il bit di referenziazione uguale a 1, viene messa in coda e il bit di referenziazione viene posto uguale a 0;
- Se la pagina presa in considerazione ha il bit di referenziazione uguale a 0 allora la pagina trovata sarà la pagina vittima.



Se tutte le pagine, nell'ultimo intervallo di clock, sono state referenziate, allora l'algoritmo degenera in algoritmo di tipo FIFO.

Usare una lista circolare piuttosto che una coda è molto più efficiente, perché invece di spostare pagine nella coda si utilizza scorrere semplicemente la lista circolare. Questa diversa implementazione è detto **algoritmo di clock** e funziona esattamente come l'algoritmo di seconda chance, con la sola differenza che la struttura dati impiegata è diversa.

Un'osservazione che si è fatta è che le pagine che non sono state usate di recente non saranno usate neanche in futuro. Per avvicinarci all'obiettivo dell'algoritmo ottimale è quindi rimuovere le pagine che non sono state usate di recente. Questa strategia viene utilizzata dall'**algoritmo LRU**(Last Recently Used), che è fattibile però molto dispendioso economicamente.

Una possibile implementazione potrebbe essere l'impiego di un contatore a 64 bit, a livello hardware, e di un eventuale campo per ogni voce della tabella delle pagine. Quando una pagina viene referenziata, il valore del contatore verrà conservato all'interno della voce della pagina. Quando si deve effettuare una sostituzione, la pagina vittima sarà la pagina che avrà il contatore più basso.

Dato che è molto dispendioso realizzarlo a livello hardware, esso può essere simulato a livello software. Un'implementazione via software dell'algoritmo sopraccitato è l'**algoritmo NFU**(Not Frequently Used). Esso utilizza una strategia basata sul conteggio delle referenze e non contando l'ordine di utilizzo. Quest'algoritmo prevede l'uso di un contatore in ogni voce della tabella delle pagine e periodicamente il bit R, prima di essere aggiornato dal Sistema Operativo, viene sommato al contatore.

La pagina vittima sarà la pagina con il contatore più basso. È molto facile da implementare anche se il problema principale è che non tiene conto della **recency**. Questo problema potrebbe privilegiare pagine usate spesso in passato e non più usate nell'intorno della sostituzione(pagini non più utili).

Queste pagine rimarranno in memoria fintantoché non saranno le pagine con il contatore più basso. Si effettua una vera e propria starvation per il resto delle altre pagine.

Per raggiungere l'efficienza dell'algoritmo LRU, basta effettuare delle semplici modifiche all'algoritmo NFU.

Le modifiche sono:

- Shift a destra di una posizione prima che sia sommato il bit R;
- Scrittura del bit R nella posizione più a sinistra.

Così si ottiene un algoritmo che tiene conto della **recency** che spesso viene chiamato **algoritmo aging**. Esso enfatizza di più gli usi recenti, infatti, rimuoverà la pagina che ha il contatore più basso.

PAGE	T1	T2	T3
0	10000000	11000000	01100000
1	00000000	10000000	11000000
2	10000000	11000000	11100000
3	10000000	10000000	~~01000000~~
4	00000000	00000000	10000000

Queste soluzioni rispetto alle soluzioni hardware adottate sono meno efficienti, dato che il taglio di bit è limitato, ma sono economicamente quelle meno dispendiose.

Dovremmo scegliere una metrica per determinare se un algoritmo è prestante. La metrica adottata è il numero di fault di pagina. Si decide il numero di frame e si stabilisce una sequenza di indirizzi virtuali a cui accedere.

Il numero di fault di pagina ottenuto dall'algoritmo ottimale è il numero minimo che un algoritmo può produrre, e a cui deve aspirare.

Notiamo che l'algoritmo FIFO e altri algoritmi riducibili(second chance, clock, NRU) ad esso soffrono dell'**anomalia Belady** ossia che aumentando il numero di frame si aumenta il numero di page fault. Quest'anomalia seppur controintuitiva, è plagiata dal problema di questi algoritmi cioè che non considerano se una determinata pagina è ancora usata spesso. Aggiungendo più frame l'ordine con cui le pagine entrano ed escono cambia, aumentando talvolta anche i page fault.

NB: l'algoritmo NRU si riduce a FIFO, perché all'interno delle classi la scelta avviene secondo il principio FIFO.

L'algoritmo LRU(least recently used) e algoritmi riconducibili(aging, NFU), non soffre di questa anomalia perché gode della **proprietà di inclusione**, ovvero che: l'insieme delle pagine in memoria al tempo t con n frame è sempre contenuto all'interno dell'insieme delle pagine in memoria con n+1 frame al tempo t.

ALLOCAZIONE DEI FRAME

Come allochiamo i vari frame per i vari processi? Si alloca in modo equo? Si partizionano i frame in modo equo, ad esempio avendo 100 frame e 10 processi, ogni processo otterrà 10 frame a disposizione?

La quantità di frame allocati per ogni processo ha già un range stabilito per funzionare:

- **Minimo strutturale teorico** -> soglia minima sotto il quale non può essere effettuata una separazione, dato che il processo non può essere eseguito -> non è 0, 1, ecc. ma è legata alla natura del processo stesso, infatti, il numero di frame dipenderà dal set di istruzioni. Per un'istruzione ADD tra due operandi, con indirizzamento diretto, se avessimo a disposizione un numero di frame massimo 1, l'istruzione non può essere eseguita.
  Il numero minimo sarà quindi 3, dato che: una pagina conterrà l'istruzione, una pagina il primo operando, una pagina il secondo operando. Si deve stare lontano dal minimo strutturale, altrimenti si incorre in un'esecuzione inefficiente.
- **Massimo** -> tutti i frame disponibili.

Nella forma più pura di paginazione, quando i processi sono avviati nessuna delle loro pagine è in memoria. Appena la CPU prova a prelevare la prima istruzione genera un page fault e fa si che il sistema operativo prelevi la pagina. Rapidamente seguono altri page fault, in modo che le prime azioni del processo decidano quali pagine portare sulla RAM. Questa forma di paginazione è detta demand paging e nella sua prima fase si ha un overhead alto. Si può ottimizzare?

Sì, si può ottimizzare guardando allo storico dei processi, caricando le pagine utili individuate dalle statistiche precedentemente stimate. L'operazione che coinvolge il caricamento delle pagine prima di lasciare eseguire i processi è detto anche **pre-paginazione** o **prepaging**, e si basa sulle statistiche o euristiche. Ebbene si caricheranno solo le pagine utili, dato che ogni processo usa solo una piccola porzione di indirizzi e il cambiamento non è mai repentino.

Ma la suddivisione come avviene?

Si possono scegliere diversi modi per suddividere il numero di frame tra i processi, e questi sono i seguenti:

- **Allocazione equa** -> si segue il principio che tutti i processi hanno lo stesso numero di pagine. L'equità però non è sempre efficiente, dato che processi piccoli potrebbero avere a disposizione molti più frame rispetto a quelli che servono realmente.
- **Allocazione proporzionale** -> più grande è il processo, più frame gli verranno concessi. Se riuscissimo a stimare le taglie dei processi, potremmo sapere quanti frame assegnare, il problema è sapere le taglie. Oltre al suddetto problema, potremmo assegnare pochissime pagine ai processi più piccoli e andare al di sotto del minimo strutturale. Si verificheranno page fault continui ed un programma che causa page fault ogni poche istruzioni è definito **thrashing**. Quando il thrashing riguarda tutti i processi allora si parla di sistema sovraccarico.
  In caso di sistema sovraccarico, il sistema operativo potrebbe decidere di limitare la multiprogrammazione, sospendendo molti processi. Il processo non dovrebbe stare al di sotto della **località di riferimento**, ovvero l'insieme di indirizzi di memoria di cui ha bisogno(dimensione variabile nel tempo) in un dato momento. La località di riferimento nasce dal concetto di località: un comportamento che mostrano i processi a non accedere alla memoria in maniera casuale.
- **Allocazione per priorità** -> si stabiliscono delle priorità per i processi, i processi con priorità più alta avranno a disposizione più frame. Questo può essere vantaggioso perché si riduce il numero di page fault per processi che hanno priorità più alta, liberandoli subito. Per processi a bassa priorità e di grandi dimensioni lo svantaggio è un possibile thrashing continuo.

Quando si incorre in un page fault, la rimozione di pagine può essere conseguita nei seguenti modi:

- **Rimozione di frame allocati localmente**, frame precedentemente allocati per il processo che ha causato page fault. È considerato l'approccio più giusto da seguire, facendo rispettare il principio di partizione,ma non è flessibile. Infatti, se un processo ha bisogno di frame dato che è in thrashing, allora sarà costretto a non cambiare la propria situazione.
- **Rimozione di frame allocati globalmente**, frame allocati per tutti i processi. È molto flessibile, dato che si potrebbero sistemare problemi legati al thrashing, ma non garantisce l'isolamento tra processi. Possono essere sottratte pagine a processi meno importanti o più piccoli, mandando essi stessi in thrashing.

Per ogni processo manteniamo un **working set**, che è l'insieme delle pagine utilizzate durante un intervallo di tempo. Per definirlo si utilizza **Δ**, un arco temporale recente che specifica quali pagine sono state referenziate recentemente. Il working set rappresenta un'implementazione concreta del concetto di località.

Si deve tracciare il working set per assegnare al processo un certo numero di pagine. Attraverso il working set, se si fa una sommatoria per tutti i processi si può calcolare il numero di frame richiesti da tutti i processi per essere elaborati.

Si può esaminare la situazione attraverso la richiesta globale di frame, infatti:

- se la richiesta globale di frame è maggiore della memoria disponibile, allora si incorre in situazioni di thrashing e di competizione per i frame.
- se la richiesta globale dei frame è minore della memoria disponibile, allora si procede con un'elaborazione priva di un eccessivo numero di page faults.

Nel caso negativo, dove la richiesta globale è maggiore della memoria disponibile, allora si limita la multiprogrammazione e si sospendono e si effettua swap-out di alcuni dei processi in esecuzione.

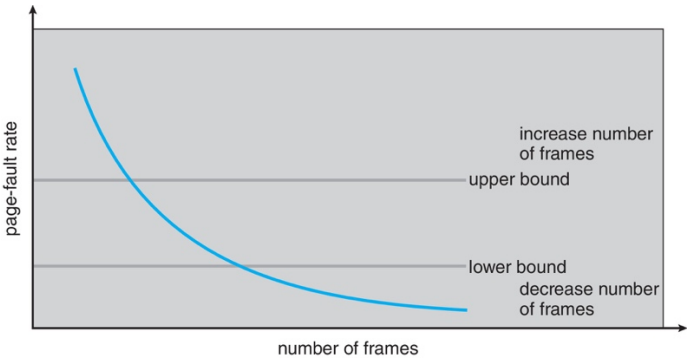
Il working set non si può calcolare precisamente, per questo si approssima attraverso all'uso di strumenti e tecniche già impiegate. Infatti, possiamo sfruttare i bit di referenziazione e l'interrupt periodici per stabilire quali pagine sono state utilizzate, come nell'algoritmo NFU o di aging.

Useremo un log o contatore che ci dice quanto frequentemente è stata usata la nostra pagina. Se la parte alta ha bit uguali a 1 vuol dire che la pagina è stata usata nell'ultimo periodo.

Il **Page Fault Frequency** affronta direttamente il thrashing, considerando il rapporto di page fault di processi per il numero di frame disponibili. Possiamo monitorare la frequenza e in tal caso assegnare più frame se il processo è in sofferenza.

Superato l'**upper bound** equivale a dire che il numero di frame per processo sono pochi. È per questo che vengono sottratti i frame ai processi che stanno sotto la **lower bound**.

Se il processo sta sotto la **lower bound**, allora vuol dire che il processo ha a disposizione più frame di quanti ne richiede la località.



Il **PFF** è collegato al working set, dato che entrambi descrivono l'andamento dinamico dei processi. Essi hanno l'obiettivo comune di garantire il giusto numero di frame, in modo tale che un processo non vada in thrashing.

Se tutti i processi stanno sopra l'upper bound, si ha un sovraccarico sistemico ed è inutile che il sistema provi a trovare un equilibrio con la sola ridistribuzione della memoria.

I page fault sono inevitabili ma si possono limitare con gli algoritmi. Per velocizzare la gestione dei page fault bisogna scegliere pagine pulite, quindi non modificate, cosicché non si paghi in termini di tempo la scrittura in memoria.

Per mantenere pagine libere, i sistemi di paginazione dispongono solitamente di un processo in background detto **paging daemon**(daemon di paginazione), che viene risvegliato periodicamente per ispezionare lo stato della memoria. Se i frame liberi sono pochi, inizia a scegliere le pagine che verranno sfrattate, anticipando le scelte di sostituzione. Questo lavoro viene svolto asincronamente rispetto ai page fault e ciò permette ad un minor overhead.

I frame liberi vengono inseriti nel **pool dei frame liberi**. Se la pagina è pulita, il contenuto resta nel frame: così, se la stessa pagina viene richiesta, può essere riutilizzata senza doverla ricaricare da disco. Quando si verifica quest'ultimo caso si parla di possibilità di ripescaggio.

La scelta della dimensione della pagina è importante. Essa avrà la stessa dimensione dei frame e a seconda della dimensione potremo avere più o meno benefici. Questa scelta non è arbitraria del produttore del software, perché viene molto influenzata dall'hardware.

Possiamo quindi avere:

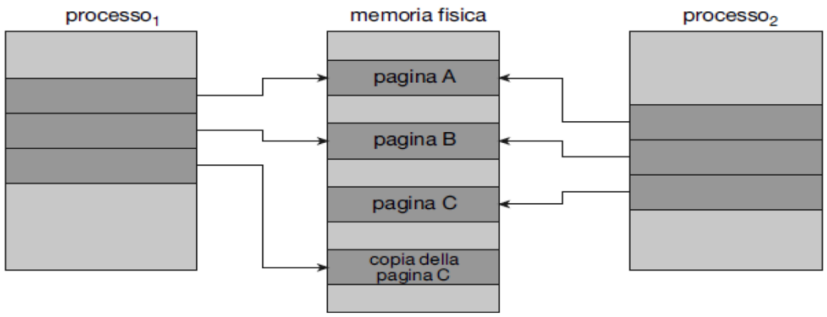
- **pagine più grandi** -> si ha tabelle delle pagine più piccole e quindi una gestione più semplice. A seguire il trasferimento è più efficace, dato che sono più costosi le fasi preliminare che il prelievo dei dati. Il numero dei page fault diminuisce dato che è proporzionale al numero di pagine. Avendo pagine più grandi, si ha la possibilità di poter referenziare la stessa pagina più volte.
- **pagine più piccole** -> si ha una minore frammentazione interna, a differenza delle pagine grandi che per metà sono vuote. Le informazioni presenti sono minimali, quindi si riduce la probabilità di caricare informazioni inutili, inoltre, questa proprietà garantisce una migliore risoluzione nel definire il working set in memoria.

La scelta della dimensione della pagina è sempre un compromesso e la scelta ottimale cerca di minimizzare il numero di page fault e numero di TLB miss.

Solitamente i sistemi operativi riservano pagine grandi per lo spazio kernel e pagine piccole per lo spazio utente, in modo che durante l'elaborazione delle istruzioni in modalità kernel non si hanno molti page fault.
Si consiglia di utilizzare una dimensione delle pagine multipla alla dimensione di un blocco su disco, in modo che si hanno calcoli più semplici e un allineamento garantito in memoria. Se si soddisfa la condizione allora l'allineamento è molto naturale e si evita l'overhead aggiuntivo causato dalle frammentazioni e ricomposizioni di pagina.

I processi possono condividere alcune pagine di sola lettura, ad esempio porzione di codice uguale e condiviso.

La condivisione è sicura solo in modalità di sola lettura, perché eventuali modifiche si propagano in modo non trasparente ad altri processi.



Storicamente esiste un modello, ormai in disuso, dove due processi condividevano lo stesso segmento logico e vi ci accedevano mediante indirizzi logici diversi. Il sistema operativo doveva essere informato di questa condivisione.

Questo tipo di condivisione non è implementabile soprattutto quando si utilizzano tabelle invertite. Infatti, nelle pagine invertite si parte dal frame e si arriva alle diverse pagine associate. In questo modello un frame può essere riferito a più processi, rendendo complessa la gestione della memoria.

Per la scrittura conviene usare l'IPC tramite memoria condivisa. Mappare le stesse pagine fisiche nelle page table di più processi è molto efficiente, sia con tabelle mono livello sia multilivello. Lo svantaggio principale è la necessità di sincronizzazione esplicita, già trattata nel capitolo sull'IPC.

Quando noi poniamo la cache prima dell'MMU, essa lavora con indirizzi virtuali. L'approccio non è molto efficiente dato che la cache non sa gestire i casi di **aliasing**, ossia quando attraverso indirizzi virtuali diversi si accede alla stessa posizione di memoria.

Esempio: si suppone di avere due processi, P1e P2, che hanno indirizzi virtuali diversi ma accedono alla stessa porzione di memoria condivisa.
Se inizialmente lavorassimo con P1, attraverso determinati indirizzi virtuali potremmo caricare la memoria in comune. Immaginando di avere bisogno di accedervi alla memoria già caricata con il processo P2, ci sarà un cache miss. Giacché sia stato caricata l'informazione in memoria, siccome con un indirizzo virtuale diverso essa non è riconosciuta dalla cache. Verrà quindi caricata la linea di memoria che ha provocato il cache miss, avendo in memoria la stessa informazione ma ripetuta in due posizioni della RAM.

Si causerà incoerenza perché se si manipolano le due linee con i due processi, uno sovrascriverà l'altro. Lo stesso problema si riscontra pure con l'uso degli ASID.

Per evitare ciò abbiamo due soluzioni:

- disabilitare la cache sulle pagine condivise-> l'accesso alla memoria condivisa verrà operata solo dalla RAM, non caricando le linee di memoria in cache -> accessi più lenti ma assenza di incoerenza e aliasing;
- uso di una ricerca basata su indirizzi virtuali e tag fisici-> la cache ricerca in parallelo sia attraverso l'indirizzo virtuale e sia attraverso la TLB. L'indirizzo virtuale sarà composto da TAG VIRTUALE e OFFSET. Il tag virtuale verrà passato alla TLB, che lo confronterà con le voci contenute, e se il confronto è positivo verrà trasmesso il TAG FISICO associato.
  Il physical address sarà composto da TAG FISICO, INDEX e OFFSET.
  Prima che si trovi il tag fisico, attraverso index e offset si vede se il dato è presente in cache -> potenziale cache hit -> ma si aspetta comunque il confronto con il tag fisico trasmesso dalla TLB.

TAG VIRTUALE(NP)	OFFSET
numero di pagina	

TAG FISICO(NF)	INDEX	OFFSET
numero del frame	set della cache	

L'index corrisponde al set di cache, ossia il gruppo di linee posizionato all'interno della cache.

Quando si hanno due processi, uno padre e uno figlio(dato da fork) allora la memoria dovrebbe essere diversa. Questo occuperà molto spazio, specialmente se ci dati comuni ai due processi.

Il **copy-on-write** permette di far condividere la stessa memoria inizialmente, fin quando nessuno dei due processi non scrive in memoria. Alla modifica, il sistema operativo:

- fa scattare un page fault perché la pagina è di sola scrittura;
- viene allocata la pagina o frame e fa una copia della pagina condivisa, modificando i permessi per la scrittura;
- aggiorna la page table del processo che ha modificato la pagina.

Così si evitano: sia l'incoerenza dei dati data dalle modifiche e sia l'inutile consumo nell'usare memoria diversa.

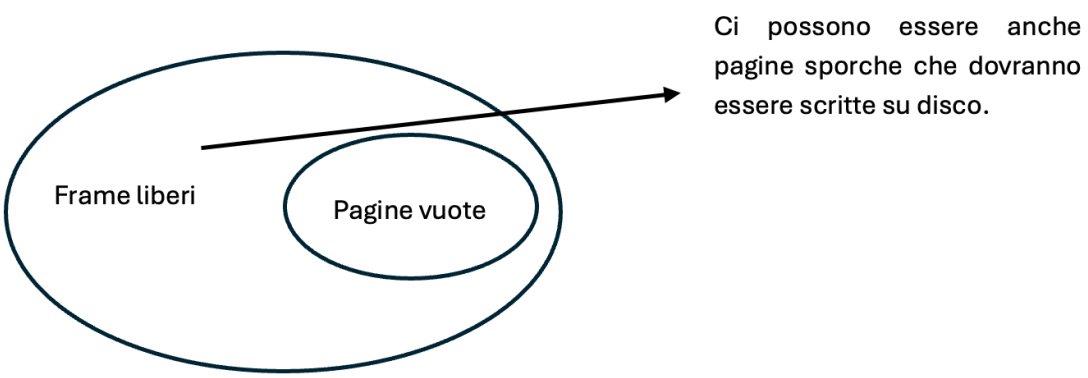
Inoltre, è utile perché avremmo delle parti del code segment e del data segment che in condivisione non verranno modificate. Si alloca quindi la stessa memoria una sola volta per diversi processi.

Un'altra possibile ottimizzazione del nostro SO è utilizzare la tecnica **zero-fill-on-demand**. Quando un processo richiede molti byte, tipo nel caso dell'allocazione dinamica(HEAP), il sistema operativo dovrebbe sottrarre al resto dei processi il numero delle pagine richieste e garantirle al solo processo richiedente.

Per questo motivo si è pensato di usare lo zero-fill-on-demand, ovvero far credere al processo di avere queste migliaia di pagine, ma in realtà si fanno migliaia di riferimenti ad una pagina chiamata **read-only static zero page**.

Viene poi applicato il processo di copy-on-write alla pagina read-only static zero page, ovvero che una volta che si tenta di sporcare la pagina, il sistema operativo: attiverà un page fault, si prenderà un frame dal **pool di pagine vuote** e infine lo si allocherà.

Il pool di pagine vuote è in realtà un sottoinsieme di frame liberi, con la sola differenza che questi frame non dovranno essere scritti in memoria quando sostituiti, perché non sono dirty.



Molti dei programmi utilizzano parti di codice che sono contenute all'interno di librerie standard dei linguaggi di programmazione. Questo codice è quindi comune a molti programmi che possono anche essere in esecuzione contemporaneamente.

Potrebbe essere il caso delle librerie che gestiscono I/O, che vengono usate da tutti i programmi. Se caricassimo le librerie per ogni singolo processo attivo, potrebbe essere ingombrante, per questo vengono condivise.

Questo codice che fa da base, negli anni viene modificata l'implementazione al suo interno ma non la sua interfaccia e il suo obiettivo. L'essere indipendente dall'implementazione è un motivo valido per cui si usa e a seconda delle esigenze si può avere un **linking statico** o un **linking dinamico**.

Il **linking statico** semplicemente si occupa di incollare il binario della funzione richiamata attraverso l'inclusione della libreria. Così viene generato un eseguibile molto più esteso e inutile perché è presente in memoria dato che lo usano anche altri programmi di esecuzione.

Per evitare questo spreco di memoria si ricorre al **linking dinamico**, che permette di caricare in memoria una sola volta il file da cui attingiamo le funzioni. Il collegamento viene fatto attraverso i riferimenti che in fase runtime sono utili poi a caricare ed eseguire il codice.

Quindi, attraverso il solo uso di riferimenti alla memoria(dove si trova il codice), si evita di riscrivere più volte lo stesso codice.

Oltre a risparmiare memoria però, si ha la possibilità di aggiornare la libreria senza ricompilare il codice che è già compilato.

Una delle piccole rivoluzioni che si sono avute in informatica nell'ultimo periodo è lo sviluppo dei **file mappati**.

Normalmente per scrivere o leggere su un file, in C si seguono i seguenti passaggi:

- si apre il file con la direttiva open che ritornerà un descrittore al file;
- si effettuano letture o scritture con read/write;
- per ogni operazione si effettua una syscall ergo viene incrementato eccessivamente l'overhead;

Quindi per scrivere o leggere sul file, si ha un overhead alto, dato che viene gestito tutto chiamate di sistema.
Per evitare ciò, attraverso opportune librerie, si assocerà tutto il file ad una parte dello spazio virtuale del processo ed esso apparirà semplicemente come un array di caratteri.
Grazie al **demand paging**, il kernel carica in RAM solo le pagine a cui si accede; se queste vengono modificate, si attiverà un page fault e il kernel gestirà la modifica

Mappare file in memoria sta anche alla base delle librerie condivise, del caricamento del codice eseguibile e dei dati statici.

Nel caso della memoria condivisa, l'eseguibile delle funzioni chiamate verrà riportato una sola volta alla memoria, ma verrà mappato tante volte nello spazio virtuale dei processi quante volte sono essi stessi.

La memoria per i processi utente può essere virtuale e può avere frammentazione interna. Per i processi kernel è diverso, date le esigenze; infatti, si ha una porzione di memoria riservata e lo spazio di indirizzamento deve essere fisico. Non ci sono indirizzi virtuali e l'allocazione deve essere contigua e non frammentata, questa rigidità è imposta perché non si passa dall'MMU, tipo quando si lavora con il DMA.

Per l'allocazione esiste un gestore di memoria interna del kernel, ossia lo **slab allocator**. Uno **slab** è un insieme di pagine contigue, che viene contenuto insieme ad altri slab all'interno di una **Cache**(non la memoria) che serve ad una specifica struttura dati omogenea. Sotto la stessa cache si raggruppano quindi più slab, in questo modo si riduce la frammentazione esterna.

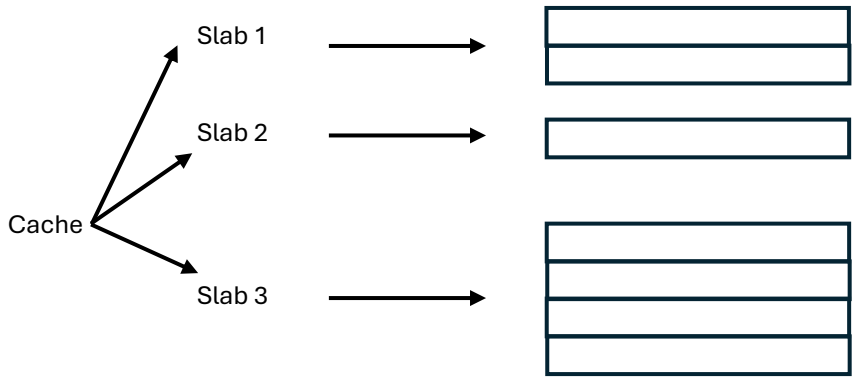
Si ricorda che la distinzione tra una Cache e le altre è fatta solo sulla base della struttura dati che ospita, e non per il processo che la utilizza. Quindi possiamo avere informazioni di più processi all'interno dello stesso slab però in frame diversi.

Lo stato di uno slab può essere:

- **vuoto** -> tutti i frame sono liberi;
- **pieno** -> nessun frame è allocabile;
- **parziale** -> alcuni frame sono liberi e allocabile e altri no;

La situazione migliore è che lo slab sia parziale, così si riempono eventuali buchi che sono presenti in memoria.

Utilizzando gli slab non si ha spreco di memoria ed è molto efficiente perché la ricerca di un blocco, dei frame disponibili o simili, si fa in tempo costante.



CAPITOLO 4 FILE SYSTEM

Tutte le applicazioni per computer devono poter memorizzare e recuperare informazioni. Si è visto nei capitoli precedenti che durante l'esecuzione un processo può conservare un numero limitato di informazioni in RAM. Quando il processo termina, le informazioni in RAM andranno perse, dato che è una memoria di tipo volatile. Oltretutto, se un'informazione deve essere condivisa tra più processi, non dobbiamo assegnarlo a un solo processo, ma essa deve essere indipendente da qualunque processo.

I requisiti per salvare informazioni a lungo termine quindi sono tre:

- si deve poter salvare una grande quantità di informazioni;
- le informazioni devono permanere oltre la fine del processo che le usa;
- le informazioni devono poter essere condivise tra più processi.

Per la memorizzazione di grandi quantità di informazioni a lungo termine si impiegano i dischi magnetici, anche se stanno prendendo piede i dischi a stato solido che sono più affidabili, dato che non hanno parti mobili che si possano rompere.

Le operazioni supportate sono:

- lettura del blocco k;
- scrittura del blocco k;

Tuttavia, ci sono dei problemi su come trovare le informazioni, proteggerle o interagire con blocchi liberi.

È per questo motivo che come per i processi e lo spazio di indirizzamento della memoria centrale, s'introduce un'ulteriore astrazione per manipolare i blocchi all'interno della memoria secondaria: i **file**.

I file si possono pensare come uno spazio di indirizzamento fornito dal modello del disco. Nel dettaglio sono **unità logiche di informazioni** create dai processi.

Le informazioni salvate sul file saranno persistenti, non influenzate dalla creazione e terminazione del processo.

Il sistema operativo si adopera a: strutturarli, denominarli, accedervi, utilizzarli e proteggerli. La parte del sistema operativo che gestisce i file si chiama **file system**.

I dettagli di implementazione e gestione costituiscono i file system. Essi si occupano di tenere traccia della memoria libera ed eventualmente gestire i settori all'interno di un blocco logico.

I dettagli di un file sono i seguenti:

- **nomenclatura** -> la nomenclatura è fortemente influenzata dal file system utilizzato. Infatti, ad esempio i file system FAT non sono case sensitive e quindi nella stessa directory non potremmo conservare due file quali: file.txt e FILE.txt. Inoltre, anche la dimensione del nome del file è decisa dal sistema operativo, infatti, nel file system di MS-DOS era permesso un massimo di 8 caratteri. Molti sistemi operativi supportano nomi di file composti da due parti: **nome** ed **estensione**. Solitamente l'estensione dopo il punto può assumere un significato, come in Windows, e quindi determina come il file deve essere aperto o quale programma è proprietario di quell'estensione. Nel caso contrario come nei sistemi operativi UNIX like, l'estensione è solo un promemoria per il proprietario e sono solo i programmi come il compilatore di C a richiedere l'estensione che indichi che il file è da compilare.
- **tipi di file** -> la maggior parte dei file sono file normali(S_ISREG()) usati dall'utente, ma possiamo avere diversi tipi di file. Le **directory**, ad esempio, sono file di sistema usati per mantenere la struttura dei file

system. Possiamo avere **file speciali a carattere**, spesso utilizzati per l'input e output, e **file speciali a blocchi** usati per modellare i dischi. A seconda del tipo di file la struttura è diversa.

- **tipi di accesso** -> i primi file system prevedevano sono l'**accesso sequenziale** ai file, ovvero che dovevano essere letti byte per byte in ordine, senza accessi sparsi. Si poteva riavvolgere la lettura e rileggere tante volte quante richieste. Con la sostituzione dei nastri e l'incentivo uso di dischi magnetici si è sperimentato un altro tipo di accesso, l'**accesso casuale**, che permette di leggere i byte da qualsiasi posizione all'interno del file. Inoltre, il posizionamento e la lettura sono possibili anche con una chiave e non soltanto con la posizione, utile nei database.
- **metadati** -> i metadati o attributi sono ulteriori voci del file (che possono essere consultabili attraverso funzioni quali stat/fstat) che elargiscono informazioni: sulla protezione del file, eventuali flag di lettura/scrittura, data di creazione modifica o ultimo accesso e la dimensione.
- **operazioni supportate sul file** -> supportate le operazioni quali: creazione, rimozione, chiusura, lettura, scrittura, riposizionamento e ridenominazione. Inoltre, indica se un processo ha **accesso esclusivo** o **condiviso** su un file. Per garantire un accesso condiviso al file si utilizza un **meccanismo di lock**, per sincronizzare gli accessi concorrenti. Il meccanismo di lock varia a seconda del sistema operativo e solitamente può assumere due stati: **shared** per la lettura da parte di più lettori ed **exclusive** per la scrittura di un solo scrittore che impedisce ulteriori letture o scritture. Le implementazioni a seconda del sistema operativo sono: **mandatory**, ossia un meccanismo di lock gestito dal sistema operativo che blocca tutte le richieste e **advisory**, ovvero che il sistema operativo avvisa ad un processo che il file è già presieduto da un altro processo.

Ci sono diverse strutture dati per la gestione dei file, e possono essere di due tipi:

- globali -> vivono nel kernel e ci danno informazioni sui file -> gli inode che contengono metadati sui file e i puntatori ai blocchi; la open-file table che contiene traccia di tutti i file aperti nel sistema.
- per processi -> servono a gestire i file che un processo sta usando -> la tabella dei file descriptor che associa i file descriptor alle voci della tabella dei file aperti; current working directory ossia la directory corrente; umask ovvero i permessi che ha un processo per aprire i file e le directory.

Agli utenti interessa come si denominano i file, quali operazioni si possono fare e altre questioni legate all'interfaccia. Agli implementatori interessa il modo in cui sono memorizzati file e directory, com'è gestito lo spazio sul disco e come fare funzionare tutto in modo efficace e affidabile.

È giusto quindi vedere come vengono implementati i file system.

La maggior parte dei dischi può essere divisa in una o più partizioni, con file system indipendenti su ciascuna di esse.

Nei vecchi sistemi, il settore 0 è chiamato **MBR**(**Master Boot Record**) ed è usato per avviare il computer. La fine dell'MBR contiene la tabella delle partizioni che contiene gli indirizzi di inizio e fine di ciascuna partizione. Una tra queste partizioni è indicata come partizione attiva.

Quando si avvia il computer, il **BIOS(firmware BASIC INPUT/OUTPUT SYSTEM)**, legge ed esegue l'MBR, che localizza la partizione attiva ed esegue il primo blocco, chiamato **blocco di boot**. Per uniformità ogni partizione deve riservare il primo blocco al blocco di boot, anche se non contiene un sistema operativo avviabile. Oltretutto, il boot record contiene il sistema operativo che deve essere caricato e attraverso ciò si carica il kernel dei vari sistemi operativi.

Ormai l'MBR è storico e non si usa più ma, nonostante ciò, è ancora contenuto per retrocompatibilità.

Il layout di una partizione del disco cambia molto a seconda del file system utilizzato, ma nonostante le varie implementazioni, il primo blocco in tutti i file system è il **superblocco**. Esso contiene tutti i metadati chiave riguardanti il file system e viene letto in memoria all'avvio del computer. Le informazioni sono legate a: numero

che identifica il file system, il numero di blocchi totali e quelli liberi, dimensioni dei blocchi e puntatori al root-inode.

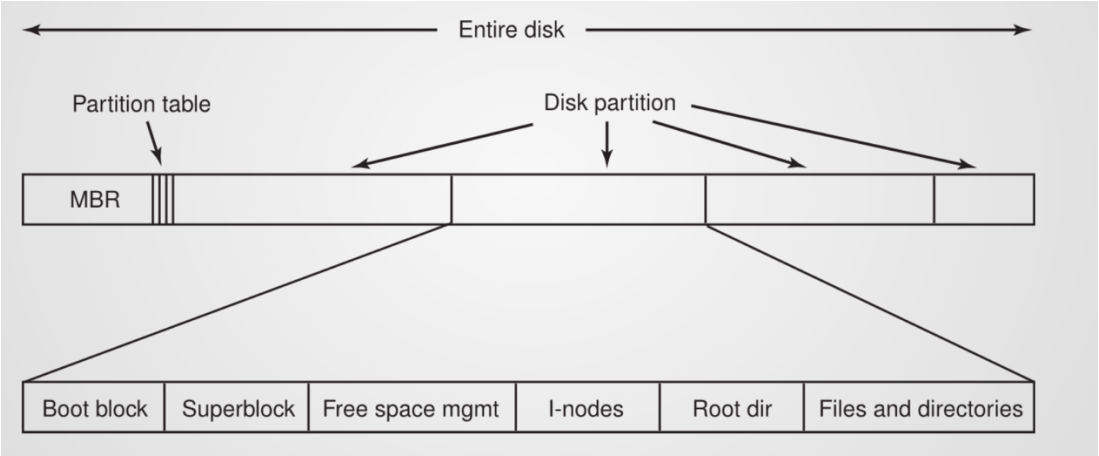
Come citato prima l'MBR non si utilizza più, quindi si è proposto di sostituirlo con **UEFI**(Unified Extensible Firmware Interface), che: avvia velocemente il sistema, risolve il problema di architetture diverse e supporta dimensione dei dischi molto elevate.

L'UEFI si trova in un chip della scheda madre e quando deve avviare una partizione consulta la tabella delle partizioni che contiene la **GPT**(GUID Partion Table) che ha presente l'inizio e la fine di ogni partizione.

Consultata la GPT, l'UEFI ha funzionalità sufficienti a leggere file system di tipi specifici, tipo FAT ecc.

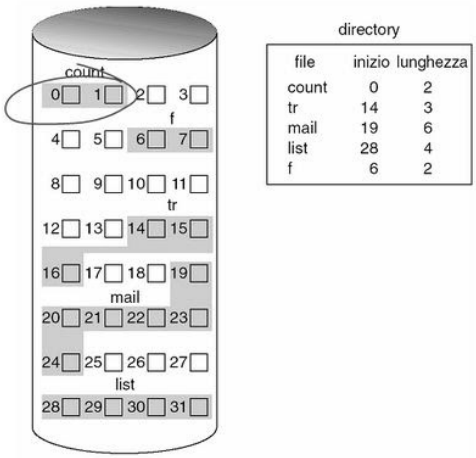
L'unità minima di allocazione del file system è il Cluster che può essere un insieme di blocchi di memoria o il singolo blocco di memoria offerto dal disco.

(NOTA BENE: NON SCAMBIARE LE POSIZIONI ALL'INTERNO DEL BLOCCO DI MEMORIA CON LE POSIZIONI ALL'INTERNO DEL FILE SYSTEM)



L'aspetto più importante dell'implementazione della memorizzazione dei file è tener traccia di quali blocchi del disco siano associati ad un determinato file. Abbiamo metodi diversi per implementarla, i quali sono:

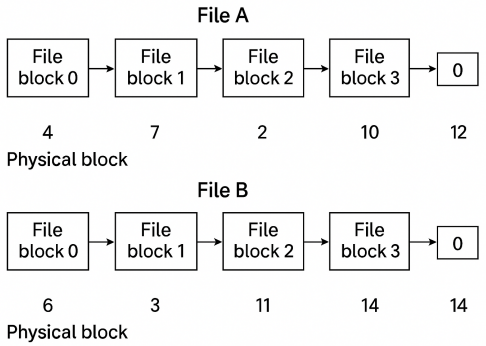
- **allocazione contigua** -> lo schema di allocazione più semplice è quello di memorizzare i file come una sequenza contigua di blocchi del disco. L'implementazione di questo tipo di allocazione è molto semplice da eseguire, infatti, basta ricordare l'indirizzo fisico del primo blocco e quanti blocchi successivi sono allocati. Inoltre, la lettura è molto efficiente dato che con una singola operazione di lettura si può leggere l'intero file. Si ha quindi una semplicità di implementazione e un'efficienza di lettura, dato che si deve cercare solo il primo blocco.



file	inizio	lunghezza
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Ma, la suddetta allocazione presenta un problema importante, la frammentazione che si genera nel tempo. Infatti, rimuovendo file si libera lo spazio sul disco, ma non verrà usato perché per allocare un nuovo file, non si può sapere la dimensione alla creazione e neanche si possono imporre limiti da rispettare. Nel caso che venga eliminato un file da 6kb non si può imporre che un file che si crea successivamente, abbia quella dimensione massima. Si dovrebbe deframmentare il disco e ciò ha un costo elevato e non deterministico.

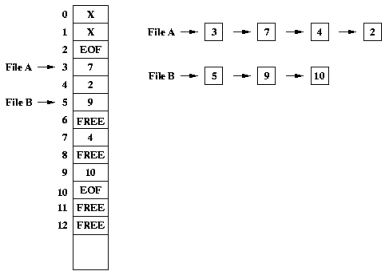
- **allocazione con liste concatenate** -> il file è configurato come una lista concatenata di blocchi del disco. La prima parte di ciascun blocco è usata come puntatore al successivo e attraverso questa tecnica si possono allocare tutti i blocchi del disco.



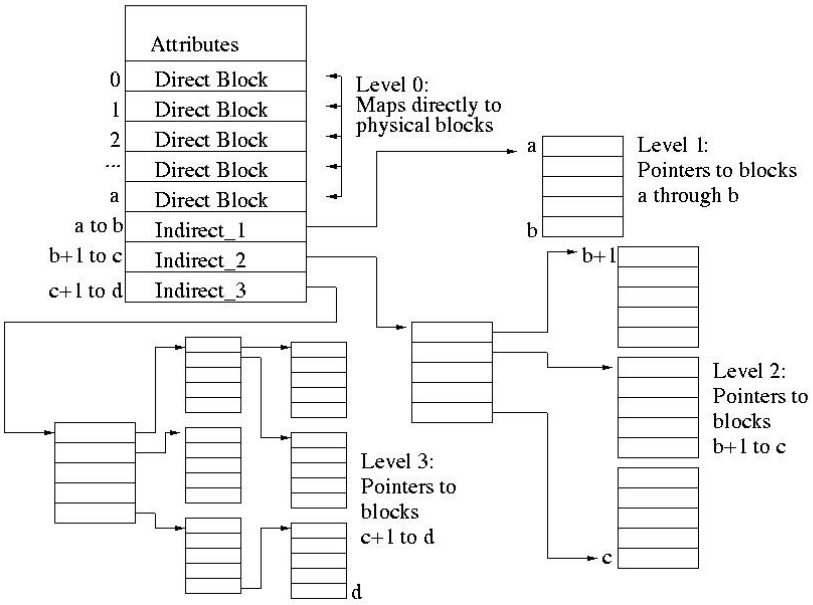
Leggere un file sequenzialmente è semplice, ma l'accesso casuale è lento, dato che per arrivare al blocco n, il sistema operativo deve partire dal principio e leggere n-1 blocchi. Inoltre, la quantità di dati in un blocco non è una potenza di due, dato che il puntatore sottrae alcuni byte. Alcuni programmi leggono e scrivono in blocchi la cui dimensione è una potenza di due, avere una dimensione diversa risulta meno efficiente.

- **allocazione a liste concatenate con una tabella di memoria** -> gli svantaggi dell'allocazione a liste concatenate possono essere eliminati, togliendo la parola puntatore ad ogni blocco e ponendola in una tabella in memoria principale.

Questa tabella è chiamata **FAT**(file allocation table), usando questa organizzazione si eviterebbe di non lavorare con blocchi di memoria la cui dimensione è la potenza di due. L'accesso casuale è sempre inefficiente come nell'allocazione con sole liste concatenate e inoltre il problema principale è che con dischi molto estesi, mantenere in RAM la suddetta tabella è costoso.



- **i-node** -> a differenza della tabella, è che l'i-node che contiene i metadati e l'indirizzi dei blocchi, deve essere presente in memoria solo quando il file è aperto. Esso contiene solo le voci dei k blocchi usati per il file. Invece, la tabella contiene n voci quanti sono gli n blocchi del disco. L'unico problema che si ha è che se l'i-node ha un numero fisso di indirizzi del disco, che cosa accade se un file supera tale limite? L'ultimo indirizzo del disco non punterà ad un blocco dati, ma ad ulteriore blocco che contenente ulteriori indirizzi di blocchi del disco(blocchi indiretti).



Per aprire un file, il sistema operativo utilizza il nome di un percorso fornito dall'utente per localizzare la voce della directory sul disco. Queste informazioni sono necessarie all'utente per trovare i blocchi del disco. A seconda del sistema, queste informazioni potrebbero riguardare l'indirizzo del file, il numero del primo blocco o il numero dell'i-node.

Una questione importante è la locazione degli attributi/metadati. Le possibili implementazioni prevedono che:

- **memorizzarli nella voce della directory**-> essa conterrà una lista di voci, una per ogni file, contenente:
  - nome del file;
  - struttura degli attributi di un file;
  - uno o più indirizzi che indicano dove sono i blocchi del disco;
- **memorizzare gli attributi negli i-node** -> le voci delle directory conterranno solo il nome del file e il numero di i-node;

È più efficiente memorizzarli negli i-node(se utilizzati), dato che leggere una directory contenente molti file potrebbe risultare molto lento. Infatti, sebbene più semplici da leggere, le directory occuperebbero molto spazio e alla rimozione del file si creerebbero dei buchi da rimuovere con la **deframmentazione**(compattazione della directory come nel caso dell'allocazione contigua).

Con gli i-node rimane la questione dell'allocazione dei nomi dei file. Le possibili strategie sono:

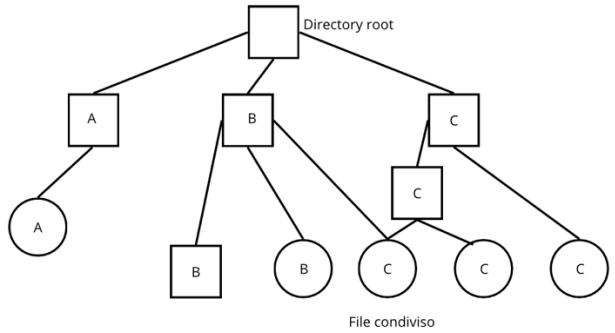
- **allocazione fissa con pochi byte**(MS-DOS) -> Non va in contro alle esigenze degli utenti;
- **allocazione fissa con tanti byte**(255) -> raramente si utilizza questa quantità di byte per un nome di un file -> spreco di memoria;
- **allocazione dinamica a lunghezza variabile** -> all'interno della directory è presente un puntatore, che punta a memoria allocata dinamicamente, sulla base delle esigenze.

Nonostante ciò, la scansione del nome di un file potrebbe essere costosa. Per migliorare si adottano:

- **tabelle hash** -> velocizzano la ricerca ma aumentano la complessità dei file system;
- **cache delle directory** -> ideale in sistemi dove ci sono pochi file e spesso la lettura è ripetuta -> cache posta in RAM, contenente il nome del file e gli i-node corrispondenti.

FILE CONDIVISI

Quando parecchi utenti lavorano insieme a un progetto, spesso hanno bisogno di condividere i file; quindi, risulta comodo un file che appaia in directory diverse, appartenenti a utenti diversi.



Come riportato nella figura il file è condiviso tra la directory C(proprietario) e la directory B. La connessione tra la directory B e il file condiviso è chiamata link(**hard link**).

La condivisione è comoda, ma introduce dei problemi. Se viene posto solamente l'indirizzo del file nelle due directory, nasce la situazione in cui, in caso di accodamento di nuovi blocchi da una delle due directory, non saranno visibile all'altra directory. Questo problema si può risolvere inserendo i blocchi in una struttura dati contenuta all'interno del file stesso(i-node). Le directory avrebbero quindi un puntatore alla struttura dati all'interno del file. Una seconda soluzione consiste nell'utilizzo di un **link simbolico**, che il sistema operativo userà per risolvere il nome del file e infine leggerne il contenuto.

Nel primo caso, se si utilizzassero gli hard-link agli i-node, potrebbe sorgere un problema durante l'eliminazione del file. Infatti, se il file venisse cancellato da C e l'i-node verrebbe pulito, avremmo comunque un puntatore in B che punterebbe ad uno spazio inesistente. L'alternativa al problema è cancellare la sola voce da C e mantenere intatto l'i-node finché c'è almeno un puntatore al file(come in quello B).

Nel secondo caso, se si utilizzassero i link simbolici al file, il file verrebbe eliminato nel caso in cui il solo proprietario lo elimina(C). Quindi, nel caso in cui C eliminasse il file, B non dovrebbe poterlo leggere più. Il problema sta nell'appesantimento della gestione, dato da letture extra nel disco. Il vantaggio fondamentale è che un file potrà essere condiviso tra più macchine, ovunque nel mondo.

Un ulteriore problema nasce in caso di backup. Quando un file ha più collegamenti, può essere raggiunto da percorsi diversi. Un programma che copia tutti i file in una directory e nelle sue sottodirectory può trovare lo stesso file più volte e, se non riconosce che sono link allo stesso i-node, creerà più copie sul backup. Su un'altra macchina, lo stesso file collegato potrebbe essere copiato due volte invece di rimanere un link.

Ma come si tiene traccia dei blocchi liberi sul disco?

Ci sono due implementazioni che possono garantire il tracciamento dei blocchi liberi e sono:

- **lista dei blocchi liberi**(free list) -> uno o più blocchi conterranno i puntatori ai blocchi liberi sul disco -> si utilizzano i blocchi liberi stessi per non occupare spazio.
  Ad esempio, supponendo di avere la dimensione del blocco di 1 KB e 32 bit(4 byte) utili per l'identificazione del blocco, si ha che un blocco può contenere 256 voci di blocchi liberi. Queste voci verranno ripartite così:
  - 255 per i blocchi liberi in memoria;
  - 1 per il blocco successivo(altro puntatore a blocchi liberi);

  Considerando di avere un disco da 1 TB con circa 1 miliardo di voci, si ha che per puntare ai blocchi liberi utilizzeremo circa 4 milioni di blocchi.
  Il vantaggio che si ha sta nel fatto che all'aumentare della memoria occupata diminuisce il numero di blocchi liberi, ergo viene utilizzata meno memoria per il tracciamento.
  Nel caso in cui ci sono lunghi tratti di blocchi consecutivi, il sistema della lista dei blocchi liberi può essere modificata per tenere traccia di serie di blocchi. Si punta al primo blocco e poi attraverso un contatore si specifica il numero di blocchi in serie.
- **bitmap** -> un disco con n blocchi richiede una bitmap con n bit. All'interno della bitmap il valore 1 indica il blocco libero, invece, il valore 0 indica il blocco occupato.
  Ad esempio, considerando un disco da 1 TB con circa 1 miliardo di voci, si occuperanno circa 130.000 blocchi da 1KB. Perché? Dato che si utilizzano i singoli bit, abbiamo che per ogni blocco possiamo indicare 1024(byte)*8(bit) = 8192 blocchi. Quindi, 1000000000/8192 ≈ 130.000 blocchi.

Queste strutture dati dovrebbe essere contenute in memoria centrale per minimizzare il numero di I/O. Soprattutto per file temporanei, consultare ripetutamente la lista dei blocchi libero o la bitmap potrebbe aumentare l'overhead per accesso al disco.

La soluzione sta nel caricare un solo blocco in memoria centrale, e nel caso che esso esaurisca i puntatori o i bit a blocchi liberi, verrà sostituito con il successivo.

Nel caso specifico, quando usiamo la free list carichiamo in memoria blocchi di puntatori, invece, quando usata la bitmap si caricheranno blocchi di bit.

Un'altra area dove l'affidabilità è importante è la coerenza del file system. I file system leggono blocchi, li modificano e poi li riscrivono. A seguito di un crash prima di aver riscritto i blocchi modificati, il file system si potrebbe trovare in uno stato incoerente. Diventa critico se i blocchi non scritti sono: inode, blocchi di directory o blocchi contenente la lista dei blocchi liberi.

Per questo si utilizzano delle tabelle dei blocchi referenziati da file e blocchi liberi. Se un file referenzia un blocco allora il contatore nella tabella dei blocchi referenziati verrà incrementato. Possiamo avere anche i casi in cui il blocco viene condiviso da più file; quindi, nella tabella avremmo un numero diverso da 0 e 1. Al contrario nella tabella dei blocchi liberi si ha 0 se il blocco è utilizzato e 1 se è libero.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	0	2	0	1	1	1	0	0	1	1	0	0	0	1
0	0	1	0	1	0	0	0	2	1	0	0	0	1	1	0

Nel caso del blocco 3, si ha una condivisione tra più file. Se questo blocco viene eliminato da un file, entrerà in uno stato dove è libero ma anche referenziato da un altro file. Per evitare ciò si copia il blocco all'interno di un blocco libero e si renderà libero il blocco eliminato, incrementando il contatore.

Nel caso del blocco 8, si ha un errore nel contatore del blocco libero. Questo viene semplicemente forzato dal sistema operativo a cambiare, ricostruendo la lista.

Nel caso del blocco 12, si ha un **blocco mancante**, cioè un blocco che non è né referenziato e né libero. Non è un problema, dato che verrà forzato ad essere libero, perché è la soluzione più semplice.

Oltre a contare correttamente i blocchi, il controllore del file system verifica anche il sistema delle directory.

Il controllore partendo dalla radice, scende lungo tutto l'albero e conta i riferimenti agli i-node che ci sono nelle directory. Ogni riferimento agli i-node è dato da hard-link, mentre i link simbolici vengono esclusi dal conteggio.

Si possono verificare due casi di incoerenza, infatti:

- se il valore del contatore è più alto delle voci delle directory -> eliminando tutti i file dalle directory, il contatore non sarebbe ancora zero e qualche i-node verrebbe mantenuto. Non è un caso grave, dato che spreca solo spazio. Si potrebbe infine impostare il contatore dei link dell'i-node al valore esatto.
- se il valore del contatore è più basso delle voci delle directory -> si può cancellare l'i-node anche se ci sono ancora riferimenti in alcune directory. L'i-node potrebbe poi essere assegnato ad altri file e questo causa inconsistenza. Per evitare questo problema si forza il valore del contatore alle voci delle directory.

Oltre ad applicare queste tecniche che limitano danni in caso di crash, se si vuole un file system molto coerente, basta sviluppare un **file system con journaling**. Un file system journaled prevede che prima di svolgere le operazioni tutto sia registrato in un log; in caso di crash: al riavvio se un'operazione è stata interrotta viene poi terminata. Ad esempio, per l'eliminazione di un file si seguono i seguenti passaggi:

- eliminare il file dalla sua directory;
- rilasciare l'i-node nel pool degli i-node disponibili;
- restituire tutti i blocchi del disco al pool dei blocchi liberi;

Se una delle sopraccitate operazioni venisse saltata, potremmo avere file in più nel disco, non liberati e non più accessibili.

Il file system scrive prima le tre operazioni in una voce di log, essa viene scritta sul disco e infine cominciano le operazioni. Quando tutte le operazioni saranno svolte senza problemi, la voce di log viene cancellata.

Per far sì che il journaling funzioni le operazioni devono essere **idempotenti**, ovvero che possono essere ripetute tutte le volte che servono senza che corrompano il sistema.

Assegnare il blocco 1000 all'i-node 25. **OPERAZIONE IDEMPOTENTE**

Aggiungere 512 byte al file. **OPERAZIONE NON IDEMPOTENTE**

CACHE DEL DISCO

L'accesso in memoria centrale per prelevare una parola di memoria è tipicamente di 10 ns. Rispetto al disco, l'accesso in memoria è dell'ordine di un milione di volte più veloce, dato che il tempo necessario per il movimento della testina e il posizionamento può richiedere fino a 5-10 ms.(nanosecondi = 10^{-9}s < millisecondi = 10^{-3}s). Per migliorare le prestazioni, i file system sono stati progettati con diverse ottimizzazioni.

La tecnica più utilizzata è la **block cache**(cache dei blocchi) o **buffer cache**. La cache, in questo contesto, è una raccolta di blocchi appartenenti al disco logicamente, ma che sono tenuti in memoria per ragioni di prestazioni.

Per ricercare tra migliaia di blocchi, si è deciso di adottare una struttura basata su tabelle hash. Ciò incrementa la velocità di ricerca del blocco. Come per la paginazione o la sostituzione dei blocchi della memoria cache, quando questa memoria è piena e si deve inserire un nuovo blocco, si hanno gli algoritmi di sostituzione.

L'algoritmo di sostituzione più utilizzato è l'algoritmo LRU, che presenta molti vantaggi, ma anche dei contro. Infatti, non scrivendo blocchi che sono stati modificati, in caso di crash si potrebbero perdere i dati. Per questo si utilizzano delle primitive, come **synch** in UNIX, che scrivono in memoria tutti i blocchi modificati. Questa primitiva può essere attivata ogni 30 secondi, garantendo molto coerenza.

Invece, in principio windows scriveva subito su disco i blocchi modificati(**write-through**), perché lavorava con i floppy disk. Con l'avvento dei dischi rigidi si è adattato inserendo una primitiva simile a synch.

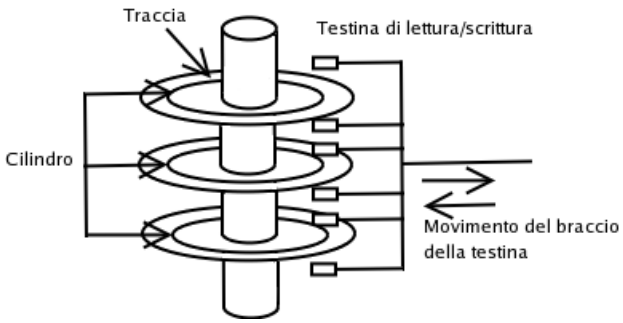
Inoltre, attraverso l'utilizzo della cache nel disco si possono adoperare strategie che riducono drasticamente il tempo di accesso nel disco. Le strategie applicabili per ridurre la latenza sono:

- **read-ahead**(lettura anticipata)-> Nel caso di un file sequenziale, quando preleviamo il primo blocco possiamo anche prelevare anche i seguenti. In questo modo quando il programma richiede il blocco successivo, esso è già in memoria.
- **free-behind** -> i blocchi già letti, vengono rimossi subito, mantenendo posizioni libere per i successivi blocchi.

HARD-DISK

I dischi rigidi magnetici sono caratterizzati dal fatto che le operazioni di lettura e scrittura sono egualmente veloci. Essi sono organizzati in:

- **piatti circolari**(vetro, ceramica) rivestiti da materiale magnetico -> attraverso una testina, spostata da un braccio attuatore, si scrivono o leggono dati -> i piatti girano a una velocità constante attraverso un motore;
- **tracce** -> cerchi concentrici che dividono la superficie;
- **settori** -> l'unità minima di memorizzazione indirizzabile;
- **blocchi** -> raggruppamento logico di più settori;
- **cilindro** -> insieme delle tracce disposte su tutti i piatti che sono equidistanti dal centro.



Se è presente un solo braccio attuatore, conviene disporre i dati dello stesso file sullo stesso cilindro, favorendo letture in parallelo, con un solo movimento del braccio.

Se ci sono più bracci attuatori, possiamo fare delle letture e scritture sovrapposte. Infatti, mentre il controller e il software sono in attesa che si completi la ricerca su un'unità, il controller può iniziare la ricerca su un'altra unità.

Per ottimizzare le richieste di lettura/scrittura si utilizza lo **scheduling del disco**, ossia l'insieme di politiche e algoritmi che utilizza il driver e il controller per gestire le richieste di accesso al disco.

Gli obiettivi da perseguire sono:

- massimizzare il numero di richieste soddisfatte in un'unita di tempo(througput), ad esempio raggruppare richieste vicine tra loro riducendo così gli spostamenti del braccio;
- minimizzare il tempo medio di accesso -> tempo che intercorre tra la richiesta e il completamento della lettura e scrittura;

Il tempo richiesto è determinato da:

- **il tempo di ricerca**, ovvero il tempo per spostate il braccio sotto il cilindro giusto(**seek time**);
- **il ritardo rotazionale**, ossia il tempo necessario al settore giusto di ruotare sotto la testina(**rotational latency**);
- **il tempo di trasferimento** vero e proprio dei dati.

In un sistema multi-programmato, più processi possono richiedere contemporaneamente letture e scritture del disco. Il disco fisico può servire una richiesta alla volta, per questa ragione tutte le richieste non servite vengono messe in coda nel driver del disco o nel controller.

Il **controller** che è la parte hardware integrata nel disco, gestisce direttamente le richieste di I/O. Esso può avere una piccola coda interna a cui sono applicate politiche di scheduling, per ottimizzare il movimento del braccio e la rotazione del piatto.

Il **driver** che è la parte software, che comunica con il controller, mantiene la coda globale delle richieste pendenti di I/O. Esso traduce gli indirizzi fisici in istruzioni per il controller, implementa politiche di scheduling sofisticate e coordina eventuali priorità tra processi.

Lo scheduling ottimale, si ottiene dalla collaborazione tra driver e controller, dato che il driver invia le richieste al controller in un ordine ottimizzato già a livello logico, che viene ulteriormente ottimizzato a livello fisico.

Di seguito sono proposti alcuni algoritmi di scheduling implementati nel driver.

FIRST-COME FIRST-SERVED(FCFS)

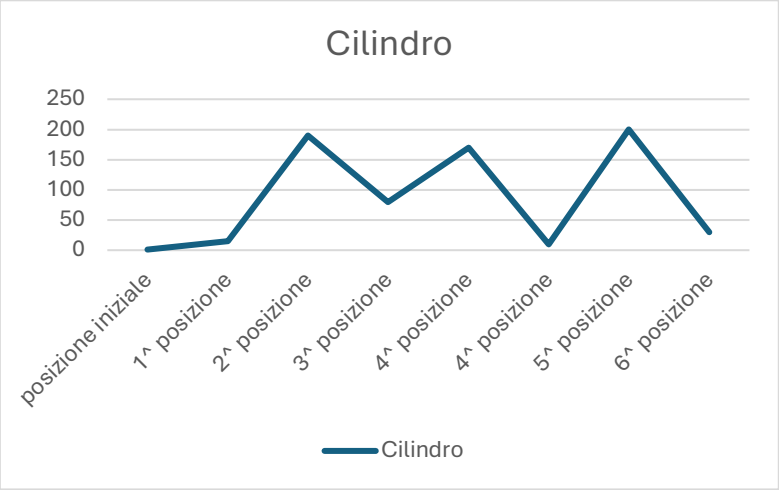
Se il driver del disco accetta una richiesta per volta e la segue nell'ordine di arrivo, cioè **Fist-Come First-Served**, c'è poco da fare per ottimizzare il tempo di ricerca. Esso è equo e semplice da realizzare, ma è molto inefficiente, perché la distanza percorsa tra tutte le tracce potrebbe risultare elevata, se esse sono lontane tra loro. Il seek time non viene ridotto, influenzando così il tempo medio necessario per completare un'operazione di I/O.

Ad esempio, se la lista delle richieste in ordine di arrivo per cilindro fosse la seguente:

15, 190, 80, 170, 10, 200, 30;

Avendo una posizione della testina, che corrisponde al cilindro 1, avremmo una distanza totale percorsa di 909 tracce.

Con una distanza totale così elevata, si impiega più tempo per accedere ai dati sul disco.



SHORTEST SEEK TIME FIRST(SSTF)

Per ridurre al minino il testo di ricerca(spostamento della testina) si potrebbe gestire sempre come richiesta la più vicina. L'algoritmo **Shortest Seek Time First** riduce lo spostamento totale del braccio della metà in confronto all'FCFS.

Sfortunatamente l'SSTF comporta un problema, dato che non è equo. Infatti, quando verranno aggiunte nuove richieste, saranno soddisfatte solo quelle che si trovano vicino alla testina e quelle agli estremi verranno ignorate. In questo caso il tempo di risposta minimo e l'imparzialità sono in contrasto.

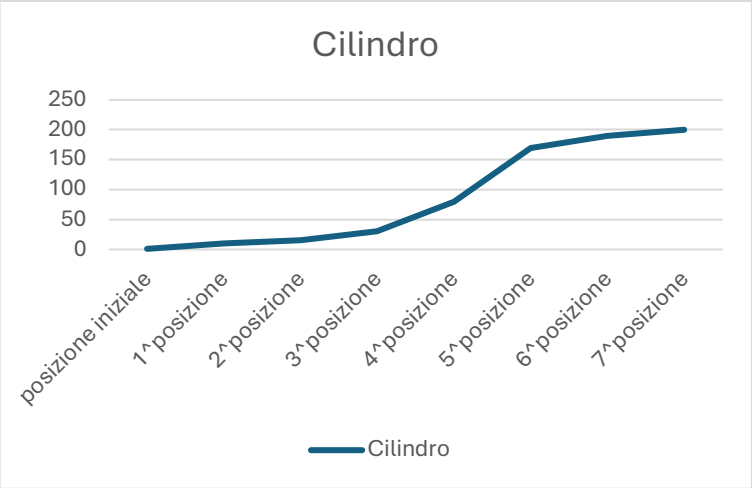
Ad esempio, se la lista delle richieste in ordine di arrivo per cilindro fosse la seguente:

15, 190, 80, 170, 10, 200, 30;

Avendo una posizione della testina, che corrisponde al cilindro 1, dovremmo riordinare le richieste. L'ordine è il seguente:

10, 15, 30, 80, 170, 190, 200;

La distanza totale attraversata è di 199.



(IN QUESTI GRAFICI LA DISTANZA PERCORSA È EFFETTUATA LUNGO L'ASSE VERTICALE, ANCHE SE IN REALTÀ LA TESTINA SI SPOSTA ORRIZONTALMENTE)

ALGORITMO DELL'ASCENSORE(SCAN)

Questo problema da affrontare con il braccio attuatore è simile a quello del braccio di un disco. Per trovare un punto d'incontro nel conflitto tra efficienza e parzialità, la maggior parte degli ascensori utilizza un algoritmo diverso. Essi continuano a muoversi in una direzione finché non ci sono richieste inevase per quella direzione.

Questo algoritmo che è presente sia nel mondo degli ascensori e sia nel mondo dei dischi è chiamato **algoritmo dell'ascensore**. Esso richiede al software di mantenere un bit: quello della direzione, **UP** o **DOWN**.

È meno efficiente dell'algoritmo SSTF, ma garantisce un'attesa massima per ogni richiesta; quindi, non ci sarà starvation per richieste che sono poste agli estremi.

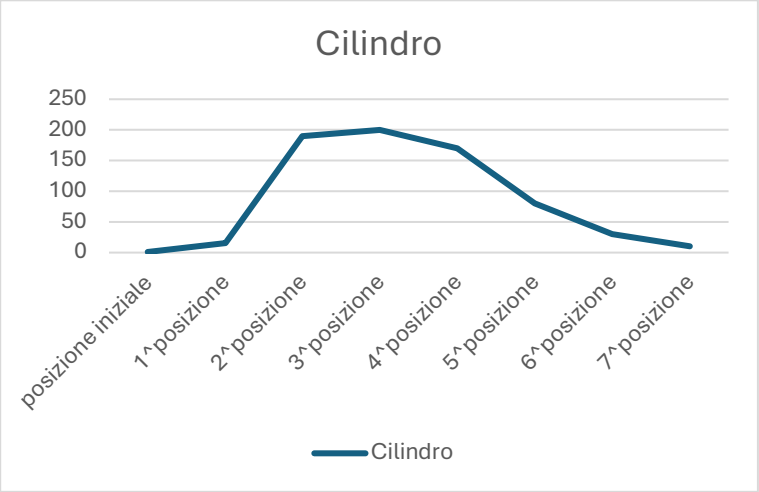
Ad esempio, se la lista delle richieste in ordine di arrivo per cilindro fosse la seguente:

15, 190, 80, 170, 10, 200, 30;

Avendo una posizione della testina, che corrisponde al cilindro 1, dovremmo riordinare le richieste. L'ordine è il seguente:

15,190,200,170,80,30,10

La distanza totale attraversata è di 389.



SCANSIONE CIRCOLARE(C-SCAN)

Una diversa implementazione della scansione o algoritmo dell'ascensore è la **scansione circolare**. La testina in questo caso serve in una sola direzione(ad esempio crescente). Quando arriva alla fine, torna all'inizio senza servire richieste sulla corsa di ritorno. Questo favorisce una maggiore imparzialità, dato che con l'algoritmo ascensore si potrebbero preferire richieste vicine alla direzione corrente. Così le attese sono più regolari.

Richieste che si trovano in posizioni centrali non vengono eseguite presto rispetto ad altre. Così si abbassa il tempo medio di attesa in presenza di tante richieste.

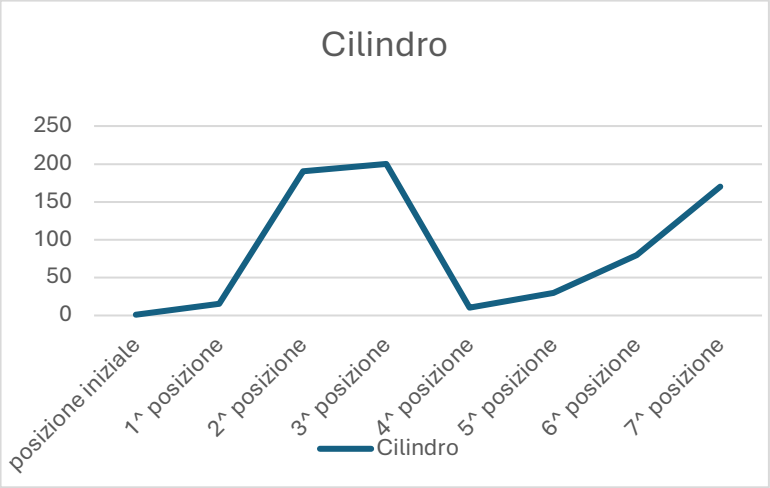
Ad esempio, se la lista delle richieste in ordine di arrivo per cilindro fosse la seguente:

15, 190, 80, 170, 10, 200, 30;

Avendo una posizione della testina, che corrisponde al cilindro 1, dovremmo riordinare le richieste. L'ordine è il seguente:

15,190,200,10,30,80,170;

La distanza totale attraversata è di 549.



Per un basso carico si sceglie l'algoritmo dell'ascensore o lo Short Seek Time First, dato che la starvation di alcune richieste non influenza molto il tempo di attesa medio. Per un alto carico si sceglie la scansione circolare per garantire maggiore equità.

RAID

Prima della diffusione degli SSD, le prestazioni delle CPU raddoppiavano circa ogni 18 mesi, mentre quelle dei dischi rigidi rimanevano sostanzialmente costanti, con tempi di accesso nell'ordine dei millisecondi. Questo divario crescente tra la velocità delle CPU e quella dei dischi ha rappresentato un problema sempre più evidente nel tempo.

Per sfruttare al meglio la potenza delle CPU, si è fatto ampio ricorso alla programmazione parallela, nella speranza di velocizzare l'elaborazione. Allo stesso modo, si è pensato che introdurre parallelismo anche nelle operazioni di I/O potesse migliorare significativamente le prestazioni complessive del sistema.

Sei specifiche organizzazione dei dischi, secondo Patterson, avrebbero potuto migliorare le prestazioni e affidabilità. Queste idee furono adottate dal settore e diedero vita ad una nuova classe di dispositivi chiamati RAID(Redunance Array of Inexpensive/Independent Disk) che sostituirono in parte i SLED(Single Large Expensive Disk).

L'idea alla base è installare un contenitore pieno di dischi, sostituire la scheda controller dei dischi con un controller RAID e continuare come al solito.

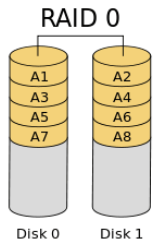
Oltre ad apparire al software come un disco singolo, un'altra proprietà di tutti i dischi RAID e i dati sono distribuiti su tutti i dischi. Le diverse configurazioni che indicano dal RAID 0 al RAID 5, non fanno parte di una gerarchia, ma solamente di diverse implementazioni.

Per migliorare l'efficienza si è adottata una tecnica chiamata **striping**, che prevede che un file sia diviso in blocchi e questi blocchi vengano conservati su un disco diverso. Ad esempio, un file che viene diviso nei blocchi A,B,C,D verrà conservato nella seguente maniera:

- disco 1 -> A
- disco 2 -> B
- disco 3 -> C
- disco 4 -> D

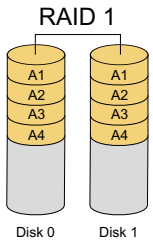
Seguendo la seguente disposizione le letture e le scritture possono avvenire in parallelo su più dischi, aumentando il throughput di sistema.

Il **RAID 0(striping)** prevede che i dati, all'interno dei dischi, vengano conservati in modalità **Round-Robin**, ossia in sequenza ciclica. Questa organizzazione è semplice da implementare e offre prestazioni elevate, specialmente nella lettura e scrittura di grandi quantità di dati.



Non c'è ridondanza nella conservazione dei dati, sebbene possa essere utile perché si risparmia spazio nei dischi, nel caso in cui un disco si guasta, si perdono tutti i dati. Quindi la vulnerabilità è maggiore.

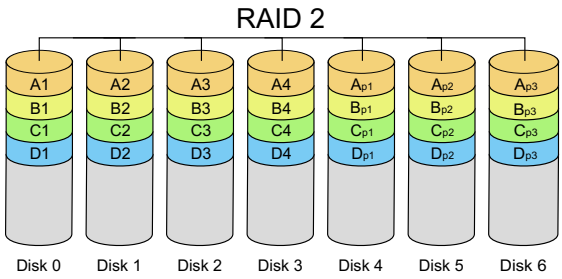
Diversamente invece accade nell'organizzazione **RAID 1(mirroring)**. Infatti, i dati vengono duplicati in più dischi, garantendo un'ottima tollerabilità ai guasti.



Si può anche fare a meno dello striping, utilizzando un singolo disco per conservare le informazioni. Le prestazioni in lettura sono raddoppiate, dato che si legge sia dal disco, contenente il file originale o file originale, e sia da quello di backup. Si possono usare quindi letture combinate per raddoppiare le prestazioni.

Lo svantaggio principale è che esso richiede il doppio dello storage e in scrittura l'overhead aumenta, dato che un dato si scrive due volte.

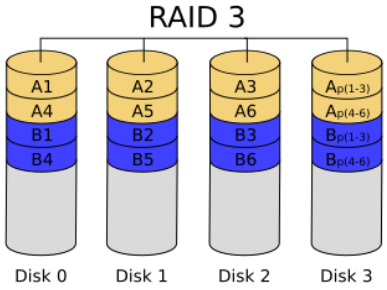
Il **RAID 2 (stripping a livello bit con ECC)**, diversamente dei RAID precedenti che operano con gli strips, lavora con i singoli bit, distribuiti su tutti i dischi. Ogni gruppo di bit è accompagnato da bit di ridondanza calcolati con un **codice Hamming**, che permette di rilevare e correggere errori singoli.



Un esempio di suddivisione dei bit è la seguente: 4 bit impiegati per i dati e 3 bit impiegati per la ridondanza.

C'è un'ottima tolleranza, perché se un disco si guasta, attraverso i bit per la ridondanza possiamo ricostruire i dati. Lo svantaggio principale è dato dalla rigidità della sincronizzazione precisa dei bit, perché i bit di una parola devono essere letti in parallelo. Data quindi l'elevata complessità, oggi questa tipologia di RAID è obsoleta e non più utilizzata.

Il **RAID 3(stripping a livello bit con bit di parità)**, è una configurazione che combina stripping a livello bit con un **disco dedicato alla parità**, in modo da poter recuperare dati, in caso di guasto di un singolo disco.

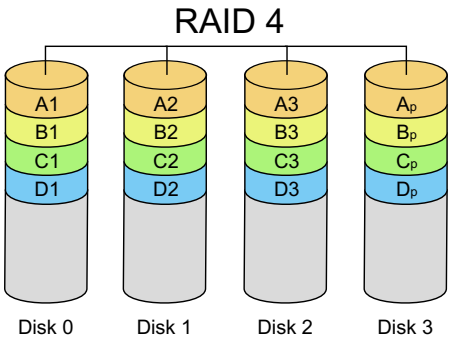


Un disco speciale sarà impiegato per contenere un bit di parità, calcolato attraverso operazioni XOR dei bit corrispondenti degli altri dischi.

La parità è molto più semplice da calcolare rispetto all'ECC, garantendo lo stesso livello di fault tollerance.

La sincronia durante la lettura è ancora richiesta. Inoltre, nei due RAID(2,3) lo stripping a livello bit può essere pesante se gestito solo a livello software, diventa più efficace solo se supportato a livello software.

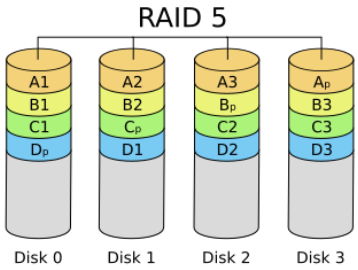
Il **RAID 4(stripping a livello blocchi con XOR sull'ultimo disco)**, riprende lo stripping in blocchi e dedica l'ultimo disco per lo XOR degli strips. Il calcolo della parità viene fatta per i bit di tutti i blocchi. Attraverso quest'operazione, in caso di guasto di un disco, possiamo ricavare il blocco corrotto.



A differenza dei RAID 2 e 3, il RAID 4 non ha bisogno della sincronia in fase di lettura del blocco. Inoltre, implementare lo stripping è semplice come nei RAID 0 e 1.

L'unico svantaggio è che in fase di scrittura o modifica si deve calcolare tutto il blocco XOR. Questo potrebbe aumentare l'overhead in fase di scrittura. L'ottimizzazione si può avere solo in fase di modifica, calcolando il blocco XOR, effettuando l'operazione tra il blocco sovrascritto e quello che stiamo sovrascrivendo.

Il **RAID 5(striping a livello di blocchi con informazioni di parità distribuite)** è un'evoluzione del RAID 4, infatti, le informazioni di parità non vengono conservato all'interno di un singolo disco(preposto per conservare solo informazioni del genere), ma vengono distribuite tra i vari blocchi. Mentre un disco sta ricevendo il blocco di parità (XOR), gli altri dischi possono già ricevere i blocchi dati successivi. Nessun disco diventa un collo di bottiglia: tutto procede in parallelo, perché la parità è distribuita. Questa evoluzione di fatti sostituisce il RAID 4.



UNITÀ A STATO SOLIDO(SSD)

Un **SSD**(solid state disk) è una memoria non volatile utilizzata come disco rigido, ma con funzionamento completamente diverso. Le celle al suo interno sono disposte in serie e formano una stringa di celle. La disposizione in serie ricorda la porta NAND ed è per questo che la tecnologia adoperata si chiama tecnologia NAND.

I dati non vengono conservati come negli hardisk, ma piuttosto l'immagazzinamento è simile a quello adoperato nelle chiavette USB o nelle schede SD, in **celle elettroniche**.

La struttura dell'SSD è divisa in blocchi e a sua volta ogni blocco è suddiviso in pagine. Le letture e scritture avvengono per pagina di ogni blocco, flusso più leggero, invece, le cancellazioni avvengono per singolo blocco. Se un blocco è pieno, non si può scrivere al suo interno, l'unica alternativa è:

- copiare le pagine che si vogliono mantenere;
- cancellare il blocco;
- riscrivere le pagine che non si devono modificare;
- sostituire la pagina che si vuole cancellare;

La strategia che poi si è attuata per migliorare le situazioni in caso di scrittura è stata aggiungere un particolare controller speciale nell'SSD, che:

- cerca un nuovo blocco per scrivere la pagina che va a sostituire quella indesiderata;
- marcare come non valida la pagina sostituita;
- il controller adopera una sorta di **garbage collector**, che prende tutti gli oggetti etichettati come non validi e li spazza via.

Nel caso invece della cancellazione, si hanno avuto problemi fino all'introduzione del commando TRIM. Infatti, quando un file veniva cancellato dal file system, quello spazio occupato dal file veniva cancellato(non c'erano più riferimenti), ma esso non comunicava al controller che i blocchi non erano più validi e che dovevano essere liberati.

Dato che non venivano considerate pagine spazzatura, si avevano delle pagine fantasma, considerate dal solo controller e non dal file system.

Attraverso l'operazione **TRIM**, alla cancellazione il file system avvisa il controller che le pagine associate al file non sono più necessarie. Il controller può così marcare le pagine come non valide, e successivamente il garbage collector le elimina, liberando spazio utile e riducendo le scritture inutili.

Per sfruttare al meglio le caratteristiche della memoria NAND, non basta solo il TRIM: anche il file system deve collaborare. I file system classici, progettati per HDD, non comunicano direttamente con il controller e possono generare scritture ridondanti o inefficaci.

Per questo sono nati i **flash-friendly file system**, come il **log-based file system**, che permettono di scrivere dati e metadati apponendoli sempre su spazio libero in maniera sequenziale (log). Questo approccio facilita anche la **recovery in caso di crash**, perché il log contiene la cronologia completa delle modifiche.

Il software all'interno del controller, chiamato **Flash Translation Layer** (FTL), si occupa di gestire dove scrivere fisicamente i dati nella NAND, mappando i blocchi logici del file system su blocchi fisici e ottimizzando le scritture.

I vantaggi ad usare le SSD sono:

- **maggiore affidabilità** -> non ci sono parti fisiche che sono in movimento;
- **maggiore velocità** -> non esiste il seek time e il tempo di latenza, dato da parti meccaniche;
- **minor consumo di energia** -> dato che non si devono muovere parti meccaniche;
- **tempi di lettura e scrittura costanti** -> non esiste una testina che si deve spostare sul piatto e che quindi i tempi di accesso variano a seconda della posizione dei dati.

Gli svantaggi sono legati a:

- **costi di produzione** -> con lo stesso budget un HDD può essere più capiente;
- **durata di vita** -> le celle NAND hanno una durata di vita limitata, infatti dopo eccessivi cicli di scrittura e cancellazione, le celle possono diventare inutilizzabili;
- **scritture lente** -> in caso di sovrascritture all'interno di blocchi pieni, abbiamo overhead dato che si devono svolgere le operazioni sopraccitate.